

LLM Evals — Interview Notes (Concept-Clear)

1. LLM Evals Kyun Zaroori Hain?

LLM Evals = process of properly testing/evaluating an LLM-based application before deploying it — sirf 4-5 questions poochh kar 'lag raha hai sahi kaam kar raha hai' bol dena eval nahi hai.

Vibe Testing — sabse common (galat) approach

- **Definition:** Casually kuch prompts try karke, sirf *feel* ke basis pe judge karna ki application sahi kaam kar raha hai ya nahi — koi metric nahi lagaya jaata.
- Example: 5–10 questions poochhe, answers achhe lage, so maan liya "it works".
- **Problems with Vibe Testing:**
 - **Informal** — koi structured process nahi.
 - **Subjective** — person to person judgement alag hoga.
 - **Not repeatable** — next version banate waqt same tarike se dobara evaluate nahi kar paoge.
 - Sirf **personal/portfolio projects** ke liye chal jaata hai; production-grade system ke liye nahi.

2. Real-World Case Studies (Bina Eval Deploy Karne Ke Nateeje)

(a) Air Canada — Bereavement Fare Hallucination

- Customer ne website chatbot se bereavement (death-in-family) discount policy poochhi.
- Chatbot ne **hallucinate** karke galat info di — bola pehle full fare pay karo, baad me refund milega (actual policy: discount pehle hi lena hota hai, baad me refund nahi milta).
- Customer ne case court me le gaya; Air Canada ne argue kiya chatbot ek "separate entity" hai.
- **Court verdict:** Website ki tarah chatbot bhi company ki property hai — chatbot jo bhi bole, company uski ownership legi. Air Canada case haar gaya aur poora refund dena pada.

(b) Chevrolet Dealer Chatbot — Jailbreak → \$1 Car Offer

- User ne emotionally chatbot ko convince/jailbreak kiya ki uski har baat maani jaaye.
- Fir poochha "\$1 me car doge?" — jailbroken chatbot ne agree karke ek **binding offer** likhit me de di.
- Poora conversation documented tha; screenshots social media pe viral hue → dealership/Chevrolet ki bahut badnaami (negative marketing).

(c) Colombian Airline — Fabricated Legal Case Citations

- Injured passenger ke lawyer ne ChatGPT se past similar court cases maangi (proof ke liye).
- Model ne confidently **fake cases fabricate** kar diye — fake names, dates, sab kuch banaya.
- Lawyer ne bina verify kiye court me present kar diya; opposition ne verify kiya to cases exist hi nahi karte the.
- **Result:** Lawyer/firm pe ~\$5000 ka fine, aur case bhi haar gaye.

Common Lesson: Teeno cases me developers ne LLM app banaya, sirf vibe-test kiya, aur direct deploy kar diya — proper evaluation hoti to ye incidents avoid ho sakte the.

3. Software Testing vs LLM App Evaluation — Core Differences

(1) Deterministic vs Probabilistic

- **Traditional software** = deterministic → same input ka output **hamesha same** aata hai (e.g. calculator: $2+2 = 4$, always).
- **LLM-based apps** = probabilistic (LLMs ke nature ki wajah se) → same input pe **different output** aa sakta hai (e.g. "overfitting kya hai" — har baar thoda alag phrased answer, koi bhi galat nahi).

(2) Single Correctness Check vs Multi-Dimensional Evaluation

- Software testing me sirf ek benchmark: **correctness** (sahi ya galat).
- LLM app evaluation me answer ko **multiple dimensions** pe judge karna padta hai, jaise:
 - Factuality
 - Completeness
 - Tonality
 - Groundedness
 - Latency
 - Cost (per answer generate karne ka cost)
- Ye dimensions **application-specific** hote hain — alag app ke liye alag important aspects honge.

4. LLM Evals — Playlist / Learning Roadmap

1. LLM Evals kya hote hain — concept ek example se samjhna
2. LLM Evals ka poora Landscape — techniques & tools ka high-level overview
3. Evaluating LLMs — different benchmark categories (model-level evaluation)
4. Evaluating LLM Applications — app-level evaluation kaise hoti hai
5. Apna khud ka Eval Pipeline banana — golden dataset curate karna + rubrics define karna
6. RAG-specific Evals
7. Agent-based Evals
8. Safety-based Evals
9. Operational Evals — deploy hone ke baad bhi evaluate karte rehna (latency, tokens/sec, time-to-first-token, system load, etc.)

Interview Tip: GenAI/AI Engineer interviews me commonly poochha jaata hai — "How do you evaluate your RAG application?" aur "How do you evaluate your agentic AI application?" — in dono ke liye upar wale concepts (multi-dimensional eval, golden dataset, RAG/agent-specific evals) clear hone chahiye.

LLM Evals — What Are They? (Concept-Clear)

1. LLM Evals — Formal Definition

Definition: LLM Evals are **systematic, repeatable tests** used to judge an LLM or an LLM-powered system against **clear criteria**.

3 Core Characteristics:

- **Systematic** — vibe testing nahi (random 5 questions poochh kar decide karna). Proper **dataset** banaya jaata hai jisme har tarah ka **edge case** cover ho — e.g. real 100 user-chats uthake ek test dataset banana.
- **Repeatable** — prompt, model, retriever, chunking strategy — kuch bhi change karo, same dataset pe wahi tarike se dobara evaluate kar sako. Isi se pata chalta hai ki **Version 1 vs Version 2** me kaun better hai.
- **Clear Criteria** — kis basis pe judge kar rahe ho, wo pehle se define hona chahiye (e.g. answer correct ho, simple explanation ho, sirf course content se aaye, safe ho — no unsafe/abusive content). Bina criteria = sirf vibe testing; criteria ke saath = proper evaluation.

2. Common Misconception — Eval Is Not Just a Metric

- Classic ML/DL background se aane walon ko lagta hai eval matlab sirf **metric** (accuracy, precision, recall jaisa) — ye galat hai.
- **LLM Eval = Entire Testing Setup**, sirf ek number nahi. Isme shaamil hota hai:
 - **Kya** test kar rahe ho (e.g. RAG ka retriever component)
 - **Kis criteria** pe test kar rahe ho (e.g. retriever ka accuracy)
 - Test ke liye use hua **dataset**
 - **Kab** test ho raha hai — offline ya production me deploy hone ke baad
 - Kaunse **tools** use ho rahe hain (e.g. RAG app ke liye Ragas)

3. LLM Evals Kis Tarah Ke Questions Answer Karte Hain?

- Can the model be used for a particular task/application?
- Is this system good enough to ship (production-ready)?
- Did Prompt v2 improve over Prompt v1?
- Is the RAG answer grounded in the retrieved context?
- Is the agent completing the task correctly?
- Is the chatbot safe for real users?
- Is the latency under control?

Eval ka goal ek score dena nahi hai — goal hai in tarah ke **practical questions** ka answer dena.

4. Two Types of LLM Evals

(Note: "Model Evals" aur "Application Evals" official industry terms nahi hain — simplify karne ke liye use kiye gaye hain. Industry me dono ko combined "LLM Evals" hi bola jaata hai.)

(A) Model Evals — Evaluate the Model Itself

- Goal: naya LLM release hote waqt uski **capabilities** test/benchmark karna.
- **8 Core Capabilities** jinpe LLMs test hote hain:
 - Reasoning

- Knowledge (general/world knowledge, cut-off date tak)
- Basic Math
- Coding
- Instruction Following
- Long Context Handling
- Multimodal Understanding
- Tool Use
- Example Benchmarks:
 - **MMLU** → knowledge & reasoning (science, history, law, medicine)
 - **GSM8K** → basic/grade-school math
 - **SWE-Bench / HumanEval** → coding
 - **IFEval** → instruction following
 - **Needle in a Haystack** → long context handling
 - **MMMU** → multimodal capability
- Ye kaam usually **frontier labs** karte hain (OpenAI, Anthropic, etc.) jab naya model release karte hain — AI Engineer ke roz ke kaam ka part nahi. Aapko bas **benchmarks read karna aana chahiye** ("literacy") — taaki project ke liye sahi LLM choose kar sako.

(B) Application Evals — Evaluate the Entire LLM-Based Application

- LLM ek application ka **sirf ek component** hai ("brain"), poora system nahi. Ek proper app me aur bhi bahut kuch hota hai:
 - User Interface
 - System Prompt
 - Tools / APIs
 - Orchestration code (LangGraph jaisa flow control, branching, parallel execution)
 - Guardrails (application-level)
 - Output Parsers
 - Memory & Context
 - Retrieval system (RAG ke liye — embedding model, vector database)
 - Deployment ke baad Monitoring & Feedback loop

Smartphone Analogy: chip maker (Snapdragon/MediaTek) apne processor ka benchmark score publish karta hai — par sirf strong processor hone se poora smartphone accha nahi ban jaata. Camera, OS, battery, screen, graphics — sab kuch achha hona chahiye. Same tarah, sirf LLM strong hone se poora application achha nahi ban jaata.

- Application Evals do level pe hote hain: **Component-level** (retriever accurate hai? embedding model sahi kaam kar raha? reranker sahi hai?) aur **System-level** (overall response quality, latency, cost-per-token).
- Question style: **"Can the model do this?"** nahi poochte — instead poochte hain **"Kya hamara product sahi se kaam karega?"**
- Example (CampusX chatbot): Student ka question sahi answer hua? Course material properly use hua? Answer faithful/grounded tha? Beginner ke liye easy tha? Hallucination hua ya nahi? Fast response mila? Chatbot safe hai?

Interview Tip: Jab bhi online/YouTube pe koi "LLM Evaluation" padhata hai, ~99% cases me wo **Application Evals** hi hota hai, Model Evals nahi — kyunki AI Engineer ka core kaam applications banana aur unhe evaluate karna hai, models banake benchmark karna nahi.

LLM Application Eval — Workflow (Concept-Clear)

1. The LLM Application Eval Workflow (9 Steps)

Ye workflow **universal** hai — simple LLM app ho ya complex agent/RAG system, same steps repeat hote hain.

1. **Define Task & Target** — Kya evaluate karna hai, wo clearly define karo (e.g. poor system/workflow).
2. **Define Success Criteria** — Kis metric se pata chalega system sahi kaam kar raha hai (e.g. classification task → Accuracy).
3. **Build a Dataset ("Golden Dataset")** — Real past data (chats/emails) uthake, manually correct labels ke saath ek dataset banao — 50–500 rows typical.
4. **Define Evaluation Method** — Result evaluate **kaun** karega — Automated (code/script), Human, ya LLM-as-judge.
5. **Run the Model** — System me dataset bhejo, system har input ka output/answer generate karega.
6. **Evaluate the Results** — Chosen method se score nikalo (e.g. Accuracy = correctly classified / total).
7. **Analyze the Results** — Errors ka pattern samjho — galti kahan ho rahi hai (prompt? model? data?).
8. **Improve the System & Iterate** — System prompt tweak karo ya model change karo → dobara Step 5 se run karo, jab tak satisfied na ho.
9. **Deploy → Monitor → Feed Back** — Deploy karne ke baad continuously monitor karo; production ke failures ko wapas golden dataset me add karo aur loop dobara chalao.

2. Example Walkthrough — Zomato Email Classifier

Problem: Zomato ko roz bahut saare customer emails aate hain — manually reply karna mushkil hai. Goal: ek LLM system jo email padhke usko **classify** kare — **Billing / Technical / General** — aur sahi team ko auto-route kar de.

System: Ek LLM ko prompt diya gaya — "You are this customer agent that reads the email and decides where to route it." (5–10 min me bana sakte ho — but directly deploy nahi karna!)

Workflow Applied:

- **Task/Target:** Poor classification system evaluate karna hai.
- **Success Criteria / Metric:** Accuracy — 100 emails me se kitne sahi route hue (e.g. 90/100 sahi = 90% accurate).
- **Dataset:** Past emails uthaye — e.g. "My card was charged twice" → Billing; "App crashes on login" → Technical; "What are your hours?" → General. Manually labelled "Golden Dataset" banaya.
- **Eval Method:** Yahan **Automated** (simple Python script) — kyunki output ek fixed label hai (Billing/Technical/General), directly compare ho sakta hai. Human ya LLM judge ki zaroorat nahi (unnecessary cost).
- **Run + Evaluate:** Dataset system me run kiya → accuracy score mila = **80%** (80/100 sahi classify hue).
- **Analyze:** 20% errors kahan ho rahe hain? Possible reasons: (a) **System prompt** confusing hai — Billing aur Technical me confuse ho raha; (b) **Model** weak/small hai, task samajh nahi pa raha.
- **Improve + Iterate:** Pehle prompt tweak kiya → re-run → accuracy **90%**. Manager bola aur improve karo → model change (bada/better LLM) kiya → re-run → accuracy **95%**. Manager satisfied.
- **Deploy + Monitor:** System deploy kiya. Production me continuously monitor karna hai — test dataset pe 95% accurate tha, par naya real customer data aane par galtiyan ho sakti hain.
- **Feedback Loop:** Jo email galat classify hua (e.g. Billing email ko Technical bola gaya), us instance ko utha ke wapas Golden Dataset me add karo → poor eval-improve loop dobara chalao. Isse dataset gradually **richer** hota jaata hai.

3. Key Takeaways (Interview-Important)

Choosing the Right Evaluation Method:

- **Automated (code)** — jab output structured/fixed ho (labels, numbers) — fast & free, e.g. classification accuracy.
- **Human** — jab nuanced judgement chahiye (long text answers compare karna, semantic meaning check karna) — accurate but **costly & slow**.
- **LLM-as-judge** — beech ka option — long/unstructured text (e.g. chatbot answers) ko automatically judge karne ke liye, human se sasta & scalable.

Continuous Loop = Reliable Systems

- Eval sirf pre-deployment step nahi — **define** → **run** → **evaluate** → **analyze** → **improve** → **deploy** → **monitor** → **feed failures back** — ye loop system ke deployment me rehne tak chalta rehta hai.
- Yehi wo **repeatability** hai jo pehle discuss ki thi — same golden dataset pe baar-baar changes (prompt/model) test karke versions compare karte ho.

One Application → Multiple Evals

- Ek hi LLM application (e.g. RAG system) pe usually **multiple separate evals** chalte hain, jaise:
 - Retriever ka performance test karne ke liye ek eval
 - Embedding model ka performance check karne ke liye ek eval
 - Poore RAG workflow (end-to-end) ko test karne ke liye ek eval
 - System ki latency check karne ke liye ek eval

Interview Tip: Ye ek bahut important point hai — "single app = single eval" nahi sochna; real systems me component-level aur system-level, dono tarah ke multiple evals ek saath chalte hain.

Why Multiple LLM Eval Pipelines? (Concept-Clear)

1. Key Idea: One LLM App → Several Eval Pipelines

Ek single LLM application banate waqt sirf ek eval pipeline kaafi nahi hoti — usually **multiple independent eval pipelines** chalti hain. Iske peeche **2 main reasons** hain: (1) Multiple Failure Points, (2) Multiple Risk Categories.

2. Reason 1 — Multiple Failure Points (RAG Example)

RAG setup: Query → **Retriever** (vector DB se relevant docs fetch karta hai) → **Generator** (LLM jo query + retrieved context ke basis pe answer banata hai).

Failure Points 3 Levels Pe Exist Karte Hain:

- **(a) Component Level** — Retriever galat docs fetch kar sakta hai; Generator sahi context milne ke baad bhi hallucinate kar sakta hai. Dono pe alag-alag eval lagta hai:
 - Retriever eval → checks **context relevance** (query ke hisaab se sahi docs mile ya nahi)
 - Generator eval → checks **faithfulness/groundedness** (answer sirf given context se bana, khud se facts add nahi kiye)
- **(b) Workflow Level** — Retriever aur Generator dono **individually** sahi kaam kar rahe ho, phir bhi **combination** fail kar sakta hai.

Example: K=5 (top-5 docs fetch) me sahi answer wala doc D5 (lowest priority) me mila. Retriever ne apna kaam kar diya (5 attempts me se ek me sahi doc laaya). Generator ko "higher-priority docs ko zyada weight do" bola gaya tha, to usne D1–D4 (galat/unrelated) ke basis pe answer bana diya — **galat answer, par dono component individually theek** kaam kar rahe the. Isliye alag se ek **workflow-level eval** chahiye jo retriever+generator ke combination ko test kare. Fix: ek **reranker** add karo jo D5 ko top pe le aaye.
- **(c) Application Level** — Component aur workflow dono sahi hone ke baad bhi poora system fail ho sakta hai — e.g. answer **correct** hai par pipeline ko **10 seconds** lag rahe hain (latency too high) — production ke liye **not deployable**. Isliye latency/cost check karne ke liye application-level eval bhi chahiye.

Core Insight: Component-level evals pass hona **workflow** sahi hone ki guarantee nahi deta, aur workflow sahi hona **poore application** ke sahi hone ki guarantee nahi deta. Har level pe alag eval chahiye.

3. Reason 2 — Multiple Risk Categories

Har failure point (component/workflow/application) ke andar bhi **multiple aspects/risks** hote hain jo alag-alag matter karte hain. Inhe 3 broad categories me organize karte hain:

- **Application Quality** — Kya app apna actual job achhe se kar raha hai? (correct, relevant, complete answers)
- **Safety** — Kya output harmful nahi hai? (toxic/dangerous/biased content, private data leak, jailbreak nahi hona chahiye)
- **Operations** — Kya system deploy hone par fast, cheap aur reliable tarike se chal sakta hai?

4. Risk Categories — Detailed Breakdown by App Type

(A) Application Quality — App-Type Wise

- **General LLM App** (e.g. text summarizer):
 - Correctness & Accuracy, Relevance, Completeness, Instruction Following (format/length specify kiya to follow hua ya nahi)
- **RAG-specific:**
 - Context Relevance, Retriever Recall, Groundedness/Faithfulness, Citation Accuracy (source se link kar pa raha hai jaisa ChatGPT karta hai)
- **Agent-specific:**

- Tool Selection (sahi tool choose kiya?), Parameter Correctness (tool ko sahi params pass hue?), Task Completion, Error Recovery (galti hone par recover kar paya?)

- **Multi-turn Chatbot-specific:**

- Context Retention (purani conversation yaad rakhna), Clarification Behavior (ambiguous query par sahi se clarify karna)

(B) Safety

- Toxicity, Harmful Content (self-harm, weapons, illegal acts), Bias (sabko equally treat kar raha ya profile-based alag behave kar raha), Private Data Leak (kisi ka card/contact info leak na ho), Prompt Injection & Jailbreak Resistance.

(C) Operations

- Latency, Cost per Request, Token Efficiency, Error/Failure Rate, Latency under Load.

5. Key Takeaway

Same application ke liye **alag-alag risk category ke liye alag eval pipeline** banate hain — e.g. ek pipeline latency ke liye, ek safety ke liye, ek correctness ke liye. **~99.99% cases me** ek LLM application me ek se zyada evaluation pipelines lagti hain, kyunki (1) multiple failure points hote hain (component/workflow/application) aur (2) har jagah multiple risk categories (quality/safety/operations) cover karni padti hain.

LLM Eval Methods — Programmatic, Human & LLM-as-Judge

1. LLM Eval Method — Definition & 3 Types

Eval Method = the mechanism/procedure jo decide karta hai ki LLM ka output **good hai ya nahi**. Simple question: eval pipeline ko execute kaun kar raha hai?

- **Programmatic / Deterministic** — ek program/code execute karta hai.
- **Human** — ek insaan judge karke score deta hai.
- **Model-Graded / LLM-Graded** — ek LLM khud judge ban ke evaluate karta hai.

Note: Golden dataset banana ek separate activity hai (usually human hi banata hai) — yahan hum discuss kar rahe hain ki final scoring/evaluation kaun execute kar raha hai.

2. Method 1: Programmatic — Example: Retriever Eval (Recall@K)

Setup: CampusX RAG chatbot ke retriever component ko evaluate karna hai.

- **Task/Target:** Retriever sahi kaam kar raha hai ya nahi (single component).
- **Success Criteria/Metric — Recall@K:** "Out of all correct/relevant docs that exist, kitne system ne apne top-K results me retrieve kiye?" Example: 2 correct docs the (D1001, D1003); K=5 results me sirf D1001 mila → **Recall = $\frac{1}{2} = 50\%$** .
- **Dataset (Golden Dataset):** 50–100 questions liye, ek human expert ne har question ke liye vector DB me sahi/correct document IDs manually identify kiye.
- **Eval Method → Programmatic:** Python script har question retriever ko bhejta hai, retrieved docs ko correct docs se compare karta hai, per-question recall nikalta hai, phir sabka **average** leta hai → overall Recall@K.
- **Result & Improve:** e.g. Recall = 67% aaya → low-recall questions analyze karo → fix options: better **embedding model**, **query expansion**, K badhana, ya **reranker** add karna.
- **Why Programmatic?** Task objective/unambiguous hai (doc match hua ya nahi) — code se easily check ho sakta hai, human ki zaroorat nahi (unnecessary cost).

3. Method 2: Human-Based — Example: Chatbot Helpfulness Eval

Setup: Poore RAG chatbot ki **helpfulness** evaluate karni hai (Application-level, Application Quality risk). Helpfulness = answer accurate + complete + right tone.

- **Why Human?** Helpfulness bahut **nuanced/subjective** hai — koi program likh ke automatically judge nahi kar sakte.
- **Success Criteria — 1–5 Rubric:** 5 = accurate + complete + correct tone; 3 = partially helpful; 1 = not helpful at all.
- **Dataset:** 50–100 diverse questions (normal, difficult, edge cases) — **sirf questions**, koi "correct answer" column nahi.
- **Eval Method → Human:** Har question chatbot ko diya → answer mila → ek human grader Q+A dekh ke rubric ke basis pe 1–5 score deta hai → sab scores ka average = helpfulness score.
- **Multiple Graders ka fayda:** Agar 2+ graders ke scores bahut disagree karte hain, iska matlab hai **rubric me ambiguity hai** — rubric ko refine karna padega.
- **Advantage:** High reliability/trust (human judgment). **Disadvantage: Cost** — millions of users scale pe humans afford nahi ho sakte.

5 Types of Human Evaluation (Important List):

1. **Direct Grading & Rating** — jo abhi upar dekha (Q+A dekh ke score dena).
2. **Red Teaming** — humans jaanbujh kar system ko attack/break karne ki koshish karte hain, breaking points find karte hain.
3. **A/B Testing** — 2 versions production me daal ke real users se rating lete hain; better-rated version pura rollout hota hai.
4. **Golden Dataset & Rubric Creation** — jab human correct answers/criteria define karta hai (khud ek tarah ka evaluation hi hai).
5. **Human-in-the-Loop** — bahut complex/ambiguous cases jaha automated/LLM checks trust nahi kiye ja sakte, wahan judgment human ko pass kiya jaata hai.

4. Method 3: LLM-as-a-Judge — Example: UPSC Mains Auto-Grading

Setup: Platform jo lakhs students ke UPSC Mains (subjective) mock-answers grade karta hai. Human graders **too costly** at scale; programmatic check **not possible** (subjective long-text). Solution: **LLM-as-Judge** — programmatic (cheap) aur human (reliable) ke beech ka option.

- **Task/Target:** Check karna ki platform answers ko **human expert jaisa hi** grade kar pa raha hai ya nahi.
- **Success Criteria:** Platform ki grading, human expert ki grading ke **jitna close** ho.

Building the Golden Dataset (2 steps):

- **Step 1 — Rubric:** Ek subject expert har question ke liye 4–5 dimensions define karta hai (e.g. concept mention hua? link explain hua? mechanism diya? examples cite kiye? balanced conclusion?).
- **Step 2 — Human Grading:** 50–100 real student answers uthaye, ek human expert rubric ke basis pe unhe grade karta hai (marks + per-dimension check) → yehi **golden dataset** banta hai.
- **Eval Method** → **LLM:** LLM ko diya jaata hai — question, max marks, exact rubric, aur student ka answer. Instruction: har dimension ke liye check karo genuinely address hua ya nahi, marks allocate karo (verbosity/keyword-stuffing reward mat karo), total marks + reasoning do.
- **Metric — MAE (Mean Absolute Error):** Har answer ke liye $|\text{LLM ke marks} - \text{Human ke marks}|$ nikalo, sabka average lo. e.g. $\text{MAE} = 2.3$ → LLM on average human se ± 2.3 marks deviate kar raha hai. **Goal: MAE ko 0 ke jitna paas lao** → LLM bilkul human jaisa grade kar raha hai.
- **Iterate:** Better LLM use karo, system prompt tweak karo, ya rubric refine karo → re-run → MAE kam karte jao.

5. Reference-Based vs Reference-Free Evaluation

- **Reference-Based:** Golden dataset me har test case ke liye ek **pre-known/correct answer** diya hota hai; output ko us reference se compare karke grade karte hain.
 - Examples: Retriever eval (correct doc IDs pehle se pata), UPSC LLM-judge eval (human ke diye marks hi reference hain).
- **Reference-Free:** Koi pre-defined "correct answer" nahi hota; output ki quality directly ek **rubric/criteria/standard** ke against judge karte hain (per-item correct answer nahi, sirf ek scale/standard).
 - Example: Chatbot helpfulness eval — dataset me sirf questions the, koi correct answer nahi; human ne 1–5 rubric ke basis pe apne judgement se grade kiya.

Quick Test: Kya golden dataset me aapko "correct answer" diya ja raha hai? **Haan** → Reference-Based. **Nahi** → Reference-Free.

6. Coming Up Next

Abhi tak jo bhi discuss kiya wo mostly **Offline Evaluation** hai (pre-deployment). Deploy hone ke baad bhi evaluation chalta rehta hai — usse **Online Evaluation** bolte hain. Ye next lecture ka topic hoga.

Offline vs Online LLM Evaluation (Concept-Clear)

1. Offline Eval — Definition & 3 Key Benefits

Definition: Koi bhi eval pipeline jo aap apne LLM app par **deploy karne se pehle** chalte ho — wahi Offline Eval hai. Ab tak jo bhi is course me discuss kiya (retriever eval, UPSC grader, chatbot helpfulness) — sab **Offline Eval** ke examples the.

- 1. Pre-Release Testing / Release Gate** — Deploy karne se pehle proper testing milti hai. CI/CD ke saath automate bhi kar sakte ho: agar eval score threshold (e.g. 95%) se upar → auto-deploy; neeche → notification/rollback.
- 2. Version Comparison** — Do options hain (e.g. Claude vs OpenAI model, ya alag prompts/rerankers/vector DBs) → dono versions par **same golden dataset + same eval** chalao, jo better score kare wahi choose karo.
- 3. Regression Testing** — "Test of change": ek cheez improve karte waqt kahin doosri cheez kharab to nahi ho gayi, ye check karna. Golden dataset me diverse categories rakho (refund, pricing, curriculum) — ek category ka score sudden drop dikhaye to regression hai.

2. Offline Kaafi Nahi Hai — 3 Production Risks

- 1. Unanticipated Inputs** — Golden dataset me sirf 200–500 anticipated questions the. Real users kuch bhi pooch sakte hain — Hindi-English mix, ambiguous half-questions, angry rants, adversarial prompt injections — jinke liye system test hi nahi hua tha.
- 2. Emergent/Systematic Failures** — Kuch problems sirf **scale** par dikhti hain: sudden traffic spike se latency badh jaana, ya thousands of conversations ke baad hi visible hone wala subtle bias (e.g. non-technical users ke saath alag behavior).
- 3. Drift** — Golden dataset ek fixed point-in-time pe banaya gaya tha, par real world (prices, curriculum, policies) time ke saath change hota rehta hai. Golden dataset update na ho to eval pipeline gradually **obsolete/stale** ho jaata hai — offline results ache aate rehte hain, par production me negative feedback badh jaata hai.

3. Online Eval — Definition

Definition: System ko **live production traffic** par, deployment ke baad, real users ke interactions ke through evaluate karna.

Biggest Defining Feature: Online eval **bina answer key/golden dataset** ke kaam karta hai — kyunki production me user kuch bhi naya pooch sakta hai jiska pehle se correct answer maujood nahi hota.

4. Offline vs Online — Quick Comparison

Aspect	Offline Eval	Online Eval
Timing	Before deployment	After deployment, continuously
Data	Fixed golden dataset (aap create karte ho)	Live production traffic
Answer Key	Haan — golden dataset me hoti hai	Nahi — on-the-go estimate karna padta hai
Input	Sirf anticipated cases	Kuch bhi aa sakta hai
Catches	Regressions	Drift, surprises, emergent bugs
Best For	Release gating (CI/CD)	Drift detection, real-world health
Cost/Speed	Fast, cheap, repeatable (small dataset)	Costly at scale → sampling use hota hai

Important: Offline aur Online eval **rivals nahi, complementary** hain — dono hamesha saath chalte hain, ek doosre ka replacement nahi.

5. Core Purpose: Correctness vs Normality

Offline Eval batata hai — kya aapka application **correctly** kaam kar raha hai? **Online Eval** batata hai — kya aapka application **normally** (production me abhi) run kar raha hai?

Example (UPSC Grader): Offline me humne correctness check ki thi — LLM ke diye marks, human ke marks ke **kitne close** hain. Deploy hone ke baad, naye student answers ka koi human reference hi nahi hota — so **correctness measure nahi kar sakte**. Instead, hum **normality** check karte hain: is week ke scores ka **distribution** pichhle weeks ke baseline distribution se

compare karo. Agar suddenly distribution shift ho jaye (e.g. average marks bahut high ho jayein), that's a **trigger** to investigate — kuch abnormal ho raha hai.

Correctness ka alternative signal: Agar direct correctness measure nahi ho sakta, proxy signals use karo — e.g.

thumbs-up/down rate me sudden spike, ya **faithfulness** jaisa metric jisko correct-answer jaane bina bhi check kiya ja sakta hai (sirf context+answer compare karke).

6. Online Eval — Two Types of Signals

- **Captured Signals** — pehle se present hote hain, bas store karna hai (koi computation nahi chahiye). Examples: thumbs up/down, latency (ms), token usage/cost.
- **Computed Signals** — evaluator se calculate karne padte hain. Examples: faithfulness, answer relevance, correctness, hallucination rate, toxicity, bias/fairness.

7. Online Eval Pipeline — Full Flow

Step 1 — Logging (Universal First Step)

Har conversation turn ko structured tarike se record karo (tool: e.g. LangSmith) — conversation ID, turn ID, user ID, timestamp, question, retrieved context, output, latency, tokens, cost, error codes, behavioral signals (thumbs up/down, escalation, rephrasing).

Logging Properties: (a) **Non-blocking** — logging se chatbot ki latency nahi badhni chahiye; (b) **Durable & Queryable** — future me fetch kar sako; (c) **Late-signal attachment** — delayed signals bhi conversation ID se link karo (e.g. escalation ek din baad hui email se); (d) **PII Handling** — sensitive info (phone, card, address) store karne se pehle mask/redact karo.

Path A — Captured Quantity (e.g. Latency)

Log → **Dashboard** (time-based graphs: last 1hr/24hr/1week) → **Alerting** (threshold cross hone par Slack/email/WhatsApp alert) → engineer action leta hai (e.g. more compute allocate karna).

Path B — Computed Quantity (e.g. Hallucination Rate)

Log → **Sampling** (poore traffic pe evaluator chalana costly hai, isliye sample lete hain) → **Evaluator** (usually reference-free LLM-as-judge) → metric compute hota hai → Dashboard → Alerting.

Sampling Strategy — Stratified, Not Random: Random sampling se better hai **stratified sampling** — conversations ko categories me divide karo (thumbs-down mila, abruptly ended, escalated, money/refund-related) aur **problematic categories se zyada samples** lo — isse issues detect hone ka chance badh jaata hai.

8. The Self-Improving Loop

Production me jo bhi failures identify hote hain (galat classify, hallucination, escalation), unhe wapas **Offline Golden Dataset** me add kar dete hain → dobara Offline Eval run karo → improve/redeploy karo → naye failures aayenge → cycle continues.

Tool Note: Platforms jaise LangSmith ek hi jagah dono handle karte hain — "Tracing" pe evaluator chalana = **Online Eval**; "Dataset" pe evaluator chalana = **Offline Eval**. Production ka koi bura conversation "Add to Dataset" se offline dataset me seedha add ho sakta hai — isi se ye loop close hota hai.

Model Evals — Why & What (Concept-Clear)

1. Why AI Engineers Need Model Evals (4 Reasons)

Model Evals ka basic purpose: LLM ki **capabilities measure** karna ("if you can't measure, you can't improve"). Par AI Engineer ke perspective se — jo apna kaam mostly **Application Evals** me lagata hai — Model Evals kyun zaroori hain? 4 concrete reasons:

- 1. Model Comparison & Selection** — Naya app banate waqt sabse pehla decision: OpenAI ka model use karein ya Claude ka? Professional setting me "koi bhi chalega" nahi bol sakte — concrete evidence chahiye. Model Evals hi ye concrete comparison dete hain.
- 2. Tracking Real Improvement** — Naya model version aaya (e.g. Opus 4.8 se Fable). Manager poochta hai "upgrade karein?" — ye decide karne ka sirf ek tarika hai: Model Evals ke numbers dekh kar justify karna ki naya model sach me better hai ya nahi.
- 3. Safety Checking** — Model Evals batate hain deployed/deploy-hone-wala LLM kitna hallucinate karta hai, kitna safe hai, jailbreak kiya ja sakta hai ya nahi.
- 4. Build vs Buy Decision** — Proprietary API (e.g. Claude) use karein ya khud ka open-source model (e.g. DeepSeek) host karein? Dono ke apne trade-offs hain (cost vs capability) — is comparison ko bhi Model Evals hi facilitate karte hain.

Bottom line: Model Evals ke bina aap "blind" ho — do models compare karne ka koi aur tareeka nahi hai.

2. Model Eval — Formal Definition & 4-Step Process

Definition: Model Eval ek **systematic process** hai jisse aap kisi LLM ki capabilities, behavior, reliability aur operational characteristics ko **controlled conditions** ke under measure karte ho.

Key Insight: Humans ke IQ jaisa ek single number LLMs ke liye exist nahi karta — LLM ki **har capability ke liye alag eval** hota hai (koi single test sab kuch cover nahi karta).

Har Model Eval Follows 4 Steps:

- 1. Decide Capability** — Kaunsi capability test karni hai? (reasoning, coding, safety, instruction following, etc.)
- 2. Bring a Test** — Us capability ko test karne ke liye ek mechanism/exam laate ho.
- 3. Run Under Fixed Protocol** — Model us test pe "exam deta hai" — prompt, conditions sab fix rakhte ho taaki repeatable ho aur multiple models ko same condition mile.
- 4. Score & Interpret** — Test complete hone ke baad result score karke publish/interpret karte ho.

3. Two Types of Tests: Benchmarks vs Custom Evals

Benchmarks — **Standardized, shared tests** (e.g. MMLU, SWE-Bench). Sabhi models same test dete hain — models ko "common ground" pe compare karne ke liye best. Universally recognized aur used.

Custom Evals — Aap apne **actual task** se data assemble karte ho, jo aapke specific use-case ke liye matter karta hai — generic usefulness ki jagah **aapke application ke liye kya relevant hai** wo measure karta hai.

4. Why Benchmarks Alone Aren't Enough — Zomato Example

Task: Zomato ke incoming emails ko classify karna hai (Billing / Technical / Refund). Do model choices hain:

- **Model A** — Top of leaderboard, bahut powerful (e.g. Claude Opus-type), par expensive: **\$15 / 1M tokens**.
- **Model B** — Chhota model, public benchmarks pe mid-table, par bahut sasta: **\$0.50 / 1M tokens**.

Public benchmarks ke hisaab se Model A **har** capability me Model B ko beat karega — maths, coding, language generation, sab me. Sirf benchmarks dekh kar obvious answer lagega "Model A use karo".

Custom Eval Approach: 200–500 past emails ka golden dataset banaya (labelled: Billing/Technical/Refund) aur dono models ko run kiya:

Metric	Model A (Big/Expensive)	Model B (Small/Cheap)
Classification Accuracy	94%	91%
Urgency Accuracy	88%	87%
Cost per 1000 emails	~\$6	~\$0.21
Latency	4.1 seconds	~0.9 seconds

Conclusion: Accuracy me difference bahut chhota hai (94% vs 91%), par cost aur latency me Model B **bahut better value** deta hai. Is specific task ke liye **Model B better choice** hai — despite losing on every public benchmark.

Key Takeaway: Agar sirf public benchmarks pe depend karte, to ye conclusion kabhi nahi mil pata — Model A hamesha jeetta dikhta. Custom eval (apne actual data/task pe run karna) hi ye reveal karta hai ki kaunsa model **aapke specific use-case** ke liye actually better hai.

LLM Core Capabilities — Interview Notes (Concept-Clear)

Ye 8 core capabilities hain jo har LLM me evaluate ki jaati hain. Har benchmark in me se kisi ek (ya zyada) capability ko target karta hai — isliye benchmarks padhne se pehle in 8 capabilities ka clear hona zaroori hai.

1. Knowledge & Reasoning

Ye domain kya measure karta hai

Do cheezein:

- Model ne training ke dauraan **kitna factual knowledge** seekha hai.
- Kya wo **multiple pieces of information ko connect** karke, unke through reason karke sahi conclusion tak pahunch sakta hai.

Ismein kya aata hai

- **Factual recall across subjects** — biology, physics, chemistry, history, law, medicine, economics, philosophy jaise areas ke facts test karna. Ye model ne kya seekha hai uska sabse basic test hai.
- **Multi-step logical reasoning** — kya model kai facts ko sahi sequence me combine kar sakta hai. Sirf ek fact se answer nahi, balki reasoning ki poori chain follow karke final answer tak pahunchna.

Frontier labs isko alag se kyun evaluate karte hain

- Ye capability “model kitna intelligent hai” is question ka **proxy** maana jaata hai.
- Media aur general public me sabse zyada discuss hone wali capability bhi yahi hai — isliye labs ko dikhana padta hai ki unka model widely-recognized knowledge/reasoning benchmarks pe competitors ko match/beat kar raha hai.

Real-world relevance

- Research papers analyze karna, complex customer questions answer karna, technically accurate documents likhna, law/medicine/science/finance professionals ko assist karna.

2. Coding & Software Engineering

Ye domain kya measure karta hai

- Kya model **actually working code** likh sakta hai.
- Kya wo real software engineering environments (large codebases, multiple files, dev tools, repositories, complex workflows) ke andar kaam kar sakta hai.

Ismein kya aata hai

- **Function-level code generation** — natural-language instructions se individual functions likhna, edge cases handle karna, inputs validate karna, function interface follow karna.
- **Real-world bug fixing** — ek real GitHub issue aur existing repository diya jaata hai, model ko root cause identify karke correct patch banana hota hai, baki software ko tode bina.

- **Multi-file, long-horizon engineering tasks** — kai files me changes karna, system ke different parts ki dependencies samajhna, refactoring ya naye features add karna.
- **Command-line & system administration** — terminal ke through multi-step tasks (packages install karna, servers configure karna, diagnostic commands chalana, tools combine karna).
- **API & function calling** — sahi parameters ke saath API calls karna, errors handle karna, returned results ko sahi se process karna.

Frontier labs isko alag se kyun evaluate karte hain

- Better coding performance **developer productivity** directly improve karta hai aur bada business value create karta hai.
- Developers ek important customer group aur revenue ka major source hain, isliye labs coding benchmarks pe bahut compete karte hain.

Real-world relevance

- Code generation, debugging, code review, CI/CD automation, software maintenance, general software engineering work.
 - **Coding evaluations me leading models aur weaker models ke beech sabse bada performance difference dikhta hai.**
-

3. Mathematics

Ye domain kya measure karta hai

- Kya model accurate **symbolic aur numerical reasoning** perform kar sakta hai.
- Maths evaluation ke liye especially useful hai kyunki final answer generally ya **correct hota hai ya incorrect** — sirf convincing sound karke response successful nahi ho sakta (jaise open-ended text me ho sakta hai).

Ismein kya aata hai

- **Grade-school maths & basic arithmetic reasoning** — multi-step word problems solve karna, natural language ko mathematical expressions me convert karna, sahi sequence me calculations karna.
- **Competition-level problem solving** — AIME jaise competitions ke problems, jisme creative reasoning, combinatorics, geometry, number theory, pattern recognition chahiye.
- **Undergraduate & graduate-level maths** — real analysis, abstract algebra, topology, probability theory, differential equations jaise advanced areas — long reasoning chains chahiye ho sakti hain.
- **Research-level mathematical reasoning** — mathematical research ke frontier ke qareeb ke problems, jahan expert mathematicians ko bhi solution tak pahunchne me kai din lag sakte hain.

Frontier labs isko alag se kyun evaluate karte hain

- Maths **genuine reasoning ka sabse cleanest test** deta hai.
- Ek plausible-sounding lekin galat answer ke liye yahan bahut kam gunjaish hai. Isse ye check karna aasan hota hai ki model **formal aur verifiable reasoning** kar raha hai ya sirf pattern-matching pe reliant hai.

Real-world relevance

- Scientific computing, financial modelling, engineering simulations, data analysis, koi bhi application jisko precise numerical results chahiye.
-

4. Long Context

Ye domain kya measure karta hai

- Kya model **bahut lambe inputs** (kabhi-kabhi lakhon tokens) se information ko effectively use kar sakta hai.

Ismein kya aata hai

- **Single-fact retrieval from long contexts** — ek bahut lambe document ke andar kahin chhupi hui ek specific piece of information dhoondhna.
- **Multi-fact co-reference across long texts** — same person, entity, concept, ya constraint ko track karna jab wo document/conversation me kai jagah, ek doosre se door mention hui ho.
- **Aggregation over long content** — bade text me spread hui information ko summarize, count, compare, ya pattern-identify karna.
- **Code repository understanding** — large codebase navigate karna, files ke beech relationships samajhna, cross-file dependencies pe reason karna.

Frontier labs isko alag se kyun evaluate karte hain

- Context-window size ek important **marketing number** ban gaya hai: 128K → 200K → 1M tokens.
- Lekin bada advertised context window ye guarantee nahi karta ki model us poori information ko **effectively use** kar sakta hai.
- Labs ko dikhana padta hai ki unke claimed context capacity ko model genuinely use kar sakta hai.

Real-world relevance

- Legal contracts review karna, academic papers analyze karna, large codebases maintain karna, lambi customer-support conversations handle karna, documents ke bade collections ke saath kaam karna.
-

5. Vision & Multimodal

Ye domain kya measure karta hai

- Kya model text ke saath **visual information ko samajh aur reason** kar sakta hai.

Ismein kya aata hai

- **Scientific chart & graph understanding** — axes, legends, data points, trends padhna. Model ko exact values extract karni pad sakti hain aur chart ka broader meaning bhi explain karna pad sakta hai.
- **Document & form understanding** — scanned documents, invoices, tables, forms process karna. Words ke saath-saath page pe information kaise arrange hui hai wo bhi samajhna zaroori hai.
- **GUI & screen understanding** — computer/mobile screens samajhna — buttons, menus, dialog boxes, input fields, aur application ka current state identify karna.

Frontier labs isko alag se kyun evaluate karte hain

- Vision se models un tasks ko handle kar paate hain jo text-only systems nahi kar sakte.
- Ye naye use-cases enable karta hai aur ek important area ban gaya hai jahan frontier models differentiate kar sakte hain — especially jab bahut saare text benchmarks saturate ho chuke hain.

Real-world relevance

- Invoices process karna, charts aur research figures analyze karna, software interfaces navigate karna, medical images padhna, engineering diagrams samajhna, visually complex documents ke saath kaam karna.
-

6. Agentic & Tool Use

Ye domain kya measure karta hai

- Kya model sirf text generate karne ke bajaye **actions le sakta hai**.
- Ek agentic model internet browse kar sakta hai, APIs call kar sakta hai, files manipulate kar sakta hai, code execute kar sakta hai, software operate kar sakta hai, aur complex multi-step tasks complete kar sakta hai.
- Success ke liye model ko: apne actions **plan** karne padte hain, sahi tools **choose** karne padte hain, progress **track** karni padti hai, aur jab kuch galat ho jaaye to **recover** karna padta hai.
- Ye AI evaluation ke **next major frontiers** me se ek maana jaata hai.

Ismein kya aata hai

- **Web browsing & persistent research** — multiple websites navigate karna, links follow karna, information search karna, kai online sources se findings combine karna. Kuch tasks me dozens ya hundreds pages tak visit karni pad sakti hain.
- **Structured tool calling & API interaction** — external tools/APIs ko sahi se use karna — sahi function select karna, valid parameters pass karna, responses handle karna, calls ko chain karna, errors se recover karna.
- **Desktop & computer use** — real applications ko visually operate karna (clicking, typing) — word processors, spreadsheets, email clients, file managers ke across navigate karna.
- **Multi-turn task execution with planning** — bade goal ko chhote steps me todna, unhe sahi order me complete karna, intermediate results check karna, jab kuch fail ho to plan adjust karna.

Frontier labs isko alag se kyun evaluate karte hain

- Ek model ke paas bahut saara knowledge ho sakta hai lekin fir bhi wo us knowledge ko ek practical task complete karne ke liye use nahi kar paata.
- Agentic evaluation ye measure karta hai: **“Model kya jaanta hai”** vs **“Model actually kya kar sakta hai”** — is difference ko.
- Ye gap frontier models me bhi abhi bahut bada hai.

Real-world relevance

- Coding agents, research assistants, customer-service bots, data-entry automation, personal assistants — koi bhi workflow jahan AI ko sirf answer dene ke bajaye actions perform karne hain.

7. Safety & Alignment

Ye domain kya measure karta hai

- Kya model **responsibly behave** karne ke liye trust kiya ja sakta hai.
- Ye check karta hai: model harmful requests refuse karta hai ya nahi, deception/sycophancy avoid karta hai ya nahi, adversarial attacks resist karta hai ya nahi, aur user pressure daale tab bhi safety principles follow karta rehta hai ya nahi.

Ismein kya aata hai

- **Refusal of harmful content generation** — cybercrime, violence, harassment, misinformation, illegal activities involving requests ko model appropriately refuse karta hai ya nahi. Saath hi model itna over-cautious bhi na ho jaaye ki harmless/legitimate requests bhi refuse karne lage.
- **Resistance to adversarial attacks & jailbreaks** — jab users prompt injection, role-playing, encoded instructions, ya doosre adversarial methods se safety rules bypass karne ki koshish karein, tab bhi model apne safety rules maintain karta hai ya nahi.

- **Truthfulness vs sycophancy** — jab model user ke beliefs/expectations se disagree karta ho, tab bhi sahi answer deta hai ya nahi. Ek safe/reliable model sirf wahi nahi bolna chahiye jo user sunna chahta hai.
- **Cybersecurity capabilities** — cryptography, reverse engineering, digital forensics, exploitation jaise advanced cybersecurity tasks model perform kar sakta hai ya nahi. Ye potentially dangerous autonomous capabilities samajhne ke liye especially important hai.

Frontier labs isko alag se kyun evaluate karte hain

- Safety ek **regulatory requirement** bhi hai aur ek major **reputational concern** bhi.

Real-world relevance

- Koi bhi organization jo AI model public ko deploy karti hai, use ye test karna zaroori hai ki uske safety controls sahi se kaam kar rahe hain.
 - Ye domain directly affect karta hai: legal liability, regulatory compliance, brand reputation, user trust, responsible deployment.
-

8. Instruction Following

Ye domain kya measure karta hai

- Kya model **exactly wahi karta hai jo user ne bola** — na zyada, na kam.
- Instructions me format, length, tone, style, structure, ya content se related requirements ho sakti hain.
- Ek model ka knowledge bahut achha ho sakta hai, lekin agar wo baar-baar simple instructions ignore karta hai to use karna frustrating ho jaata hai.

Ismein kya aata hai

- **Verifiable constraint satisfaction** — aise instructions follow karna jo automatically check ho sakein, jaise:
 - Word limit ke andar rehna.
 - Bullet points use karna.
 - Valid JSON produce karna.
 - Markdown structure follow karna.
 - Specific section headings use karna.
- **Handling ambiguous/incomplete instructions** — jab request underspecified ho to user ko probably kya chahiye ye samajhna, zaroorat pade to clarifying question poochna, aur jahan appropriate ho wahan reasonable defaults choose karna.

Frontier labs isko alag se kyun evaluate karte hain

- Instruction following strongly affect karta hai ki everyday use me model helpful lagta hai ya frustrating.
- Real production environments me **user satisfaction se sabse directly connected** capabilities me se ek yahi hai.

Real-world relevance

- Almost har practical LLM use instruction following pe depend karta hai. Examples:
 - Required format me reports generate karna.
 - Specific style rules ke hisaab se code likhna.
 - Structured JSON outputs produce karna.
 - Brand-tone requirements follow karna.
 - Coherent multi-turn conversations maintain karna.
-

Quick Recap Table (Interview Cheat-Sheet)

#	Capability	Kya Test Karta Hai (One-Liner)	Real-World Use
1	Knowledge & Reasoning	Factual recall + multi-step logical reasoning	Research analysis, professional Q&A (law/medicine/finance)
2	Coding & Software Engineering	Working code + real repo/dev-environment tasks	Code gen, debugging, CI/CD, maintenance
3	Mathematics	Symbolic/numerical reasoning, verifiable answers	Scientific computing, financial modelling
4	Long Context	Effectively use very long inputs (100K+ tokens)	Legal contracts, large codebases, long conversations
5	Vision & Multimodal	Understand + reason over visual + text info	Invoices, charts, medical images, GUI screens
6	Agentic & Tool Use	Take actions (browse, call APIs, use tools), not just generate text	Coding agents, research assistants, automation
7	Safety & Alignment	Trustworthy, refuses harm, resists jailbreaks, truthful not sycophantic	Legal liability, compliance, brand trust
8	Instruction Following	Does exactly what's asked — format, length, tone, structure	Structured outputs, brand tone, multi-turn coherence

One-liner for interviews: *“LLM evaluation is organized around 8 core capabilities — Knowledge & Reasoning, Coding, Mathematics, Long Context, Vision & Multimodal, Agentic & Tool Use, Safety & Alignment, and Instruction Following — and every benchmark you’ll encounter is designed to target one or more of these, so knowing this map first makes learning any individual benchmark much easier.”*

LLM Benchmarks — Interview Notes (Concept-Clear)

1. Benchmark kya hota hai?

Benchmark = ek **standardized test** jo kisi LLM ki ek **specific capability** ko measure karta hai.

Model evals do type ke hote hain:

- **Standardized benchmarks** (fixed test, sab models pe same tareeke se apply hota hai)
- **Custom evals** (apni application ke hisaab se banaye jaate hain)

Har benchmark ek **research paper** se aata hai — jaise GSM8K, MMLU, SWE-Bench sab research papers hain jisme dataset, run config, scoring aur aggregation sab define hota hai.

2. Har Benchmark ke 4 Core Components

Example: **GSM8K** (Grade School Math, ~8K rows, 2021 me aaya)

(1) Dataset / Task

- Dataset = **Questions + Golden Answers** dono hote hain.
- GSM8K: grade-school level math word problems.
 - *Q: Natalia sold clips to 48 friends in April, half as many in May. Total?*
 - *A: 72*
- **Task** = model ko diya jaane wala instruction — “solve this math problem and give the answer.”

(2) Run Configuration

Ye batata hai ki evaluation ke waqt **settings kya rahengi** (dono models pe SAME hona chahiye, warna comparison invalid ho jaata hai).

(a) Prompt Construction

- **Zero-shot vs Few-shot** — zero-shot me sirf question diya jaata hai; few-shot me pehle kuch solved examples dikhaye jaate hain. GSM8K classically **8-shot** hota hai.
- **Chain-of-Thought (CoT) vs Direct** — CoT allow karne se model step-by-step soch ke answer deta hai, accuracy badh jaati hai. Direct me galti ka chance zyada.

(b) Decoding & Sampling Config

- **Temperature** — zero ke qareeb rakha jaata hai (deterministic output ke liye). Zyada temperature = creative/random answers.
- **Max tokens** — agar kam rakha to CoT beech me cut ho sakta hai (model answer complete nahi kar payega). Zyada bhi excess reasoning allow karta hai. Dono models ko same max tokens milna chahiye.

(c) Scoring Strategy & Tools

- **pass@1** — ek hi attempt, right ya wrong.
- **pass@k** — k attempts diye, agar ek bhi sahi aaya to correct maana jaata hai (lenient).
- **maj@k (majority@k)** — k baar same question puchha, jo answer sabse zyada baar aaya wahi final answer.
- **Tools allowed or not** — jaise Python interpreter ya web search allow karna score ko artificially boost kar sakta hai. GSM8K me tools NOT allowed. SWE-Bench jaise benchmarks me tools (GitHub access) allowed hote hain.

(3) Scoring Method

Do steps:

1. **Extraction** — model ka raw output (jo kisi bhi format me ho sakta hai — “The answer is 72” ya sirf “72”) se sahi answer nikalna. Usually **regex** ya structured output se hota hai.
2. **Comparison**:
 - **Closed-ended** (exact match, jaise $72 == 72$) — programmatic comparison, straightforward.
 - **Open-ended** (paragraph-type answers, no fixed answer) — **LLM-as-a-judge** ya human evaluation use hota hai.

(4) Aggregation

- Har question ka score (0 ya 1) hota hai; sabko combine karke final % nikala jaata hai.
- **Simple aggregation**: 920/1000 correct → 92% score.
- **Complex aggregation**: jaise MMLU (57 subjects) — har subject ka alag score hota hai, phir unhe combine karna hota hai — **simple average nahi**, kyunki subjects ke questions unequal hote hain, isliye **weighted mean** use hota hai.

3. Evaluation Process (Step-by-Step Loop)

Model evaluation basically ek **loop over the dataset** hai. Har item (question) ke liye 7 steps hote hain:

1. **Load the item** — dataset se ek question + uska golden answer uthao.
2. **Build the prompt** — few-shot examples inject karo, chat template apply karo, instructions add karo. Ab tumhare paas wo exact string (ya message list) ready hai jo model ko bheji jaayegi.
3. **Call the model** — pinned decoding config (temperature, max_tokens, etc.) ke saath. API model ke liye ye ek HTTP call hai; local model ke liye ek forward pass.
4. **Capture the raw output** — aur usse **verbatim log** karo. Ye important hai: agar score galat lage, to raw output hi batayega ki fail model ne kiya ya tumhare parser ne.
5. **Extract the answer** — regex ya parser use karke prose se sahi answer nikalo (jaise “The answer is 72” se sirf 72 nikalna).
6. **Grade** — extracted answer ko golden key se compare karo → result 1 (correct) ya 0 (incorrect).
7. **Store** — per-item result save karo, saath me wo prompt aur raw output bhi jisse ye result aaya (debugging/auditing ke liye zaroori).

Loop khatam hone ke baad — sab per-item scores ko **aggregate** karke final headline number (jaise “92% on GSM8K”) milta hai.

Pseudocode (isko samajhna interview ke liye kaafi hai):

```
results = []
for item in dataset:
    prompt = build_prompt(item, few_shot=8, template=chat_template)
    output = model.generate(prompt, temperature=0, max_tokens=512, stop=["\n\n"])
    pred = extract_answer(output)          # regex: last number
    score = (pred == item.gold)            # 1 or 0
    results.append({prompt, output, pred, item.gold, score})

accuracy = mean(r.score for r in results)
```

Problem: Ye loop dikhne me simple hai, lekin production-level implement karne me complex hai — error handling, retries, rate limits, batching, structured extraction, etc.

Eval Harness

Shuru me evaluation simple lagta hai: question lo, model ko bhejo, answer check karo, repeat karo. Lekin real me aur bhi cheezein handle karni padti hain:

- Final answer sahi se extract karna jab model **“The answer is C”**, **“C.”**, ya **“Option C”** likh de — sab ko same

treat karna.

- Benchmark ke **exact scoring method** se grade karna (har benchmark ka apna method hota hai).
- Hazaaron questions ko efficiently **batches** me bhejna.
- **Failed API requests ko retry** karna.
- **Rate limits** handle karna.
- **Eval harness** = ek ready-made system jo ye saari details already handle karta hai. Harness ko pehle se pata hota hai:
 - Benchmark (GSM8K, MMLU, etc.) ke questions kahan se milenge
 - Har question ko kaise format karna hai
 - Model ko kaise bhejna hai
 - Model ke answer ko kaise extract karna hai
 - Sahi hai ya galat kaise judge karna hai
 - Final benchmark score kaise calculate karna hai
- **Analogy**: Benchmark = exam paper, Eval Harness = **complete exam administration system** (instructions, answer-sheet format, checking rules, aur score calculation — sab kuch).
- Popular tools: **lm-evaluation-harness** (EleutherAI, industry standard), **Inspect**, **HELM**, **DeepEval** (zyada application-eval-focused, thoda zyada code likhna padta hai).
- lm-eval-harness se sirf ek **command** se pura benchmark run ho jaata hai — no manual loop coding needed.

4. Kaun Evaluation Karta Hai? (3 Stakeholders)

1. Frontier Labs (model builders)

Kaun: OpenAI, Anthropic, Google DeepMind, Meta, Mistral, DeepSeek, Qwen — jo bhi model ship kar raha hai.

Kyun evaluate karte hain:

- **Internally, during development** — ye invisible majority hai eval work ki. Evals guide karte hain pretraining decisions, post-training, aur ye decide karne me ki kaunsa checkpoint ship karna hai. Iske bina model development **blind** hai — jo measure nahi kar sakte wo improve nahi kar sakte.
- **Release gating** — decide karna ki model itna achha (aur safe) hai ki ship kiya jaaye.
- **Marketing** — launch post me jo benchmark table dikhti hai, wahi part hai jo public dekhta hai.

Numbers kaise read karein: Lab ke numbers ek **ceiling** hote hain — favorable conditions me measure kiye gaye, selection aur configuration me biased.

2. Third-Party Evaluators

Kaun: Independent organizations jinka product hi evaluation hai — **LMarena (Chatbot Arena)**, **Artificial Analysis**, **Epoch AI**, **Vals AI**, aur similar.

Kyun evaluate karte hain: Ye unka reason for existing hai. Ye models ko independently run karte hain aur comparisons publish karte hain. Inka business model hi neutral dikhne pe depend karta hai.

Numbers kaise read karein: Ye group AI engineer ke liye **sabse useful** hai — ek reason se: ye **har model ko khud, same conditions me** run karte hain. Ye wo bhi measure karte hain jo labs omit karte hain: **cost aur latency**.

3. Companies & AI Engineering Teams

- Labs **general capability** measure karte hain, third parties **cross-model comparability** measure karte hain.
- Lekin koi bhi ye measure nahi karta ki model tumhare **support emails ko sahi price pe sahi classify karta hai ya nahi** — apni specific use-case ke liye ye kaam khud AI engineers/companies ko karna padta hai.

Interview tip: Kabhi bhi sirf frontier lab ke published numbers pe blindly trust mat karo — pucho “evaluation kisne kiya, kis condition me?”

5. Benchmarks ki Limitations (IMPORTANT for Interviews)

(a) Benchmark Contamination

- Public benchmarks ka dataset internet pe available hota hai.
- Naye models jab pre-training me internet data scrape karte hain, to benchmark ke questions+answers bhi training data ka part ban sakte hain.
- Result: model "reasoning" nahi kar raha, balki **memorize** kar chuka hai answer.
- Solution:** private benchmarks ya dynamic benchmarks (jo regularly update hote hain) use karo.

(b) Benchmark Saturation

- Naya benchmark aata hai → shuru me sab models poorly perform karte hain (25-35%).
- Time ke saath models improve hote hain → scores 90%+ pe cluster ho jaate hain.
- Jab sab models ka score similar ho jaaye, to benchmark models ko **differentiate** nahi kar pata → benchmark **saturated** ho jaata hai aur retire kar diya jaata hai.
- Example: GSM8K, MMLU, SWE-Bench (old version) — sab saturated maane jaate hain.

(c) Configuration Gaming

- Labs apne model ko **favorable conditions** dete hain (jaise extra tools allow karna) aur competitor ko default conditions.
- Isse score me 5-10% ka artificial difference aa sakta hai.
- Isliye "config" ka pura transparency zaroori hai — agar nahi diya gaya, to number trust mat karo.

(d) Aggregation Gaming

- Multi-subject benchmarks (jaise MMLU) me sirf **average score** dikhaya jaata hai, individual subject-wise weaknesses chhupa di jaati hain.
- Example: Model Physics me achha, Economics me kharab — lekin sirf overall 85% dikhaya jaayega.
- Agar aapko specific domain application banani hai, to sirf aggregate score pe bharosa mat karo — subject-wise breakdown maango.

7. Knowledge Capability — 7 Famous Benchmarks ki Evolution Story

Knowledge capability ka matlab hai: LLM ne training ke dauraan kitna **world knowledge retain** kiya (uske weights/biases me kitna parametric knowledge chhupa hai). Ye sabse **fundamental capability** maani jaati hai — reasoning, coding jaise capabilities baad me "emergent behavior" ki tarah aayi.

Neeche di gayi **evolution story** yaad rakhna interview ke liye bahut useful hai — kyunki isse pata chalta hai ki har naya benchmark **kis problem ko solve karne ke liye** aaya.

Roadmap (Evolution Tree)

```
MMLU (2020) — "Mother of all benchmarks" — Breadth of Knowledge
|
|→ TruthfulQA (2021) → saturate → SimpleQA (2024) — Reliability/Truthfulness
|→ AGIEval (2023) — Human exams reuse karo
|→ GPQA (2023) — Depth of Knowledge (breadth ki jagah)
|→ MMLU-Pro (2024) — MMLU ki flaws fix karo
```

Sab saturate/near-saturate ho gaye → HLE (2025) — Breadth × Depth combined

(1) MMLU — Massive Multitask Language Understanding

“Mother of all benchmarks” (Sept 2020)

- **Kya test karta hai:** Sirf **Breadth of Knowledge** (depth nahi).
 - **Dataset:** 57 subjects, ~14,000 MCQs (4 options each). Questions GRE, USMLE, AP jaise real exams se aur textbooks se liye gaye.
 - **Task:** Question + 4 options do, model ko sahi option (A/B/C/D) batana hai.
 - **Scoring:** 2 tareeke — (a) model directly answer generate kare, ya (b) **log-probability** nikalo har option ki, jiski probability sabse zyada wahi answer. Dono me 2-3% ka difference aa sakta hai.
 - **Run config:** 5-shot, direct (no CoT), temperature 0, tools not allowed. **Prompt format bahut sensitive hai** — thoda sa wording change karne se score kaafi wave ho sakta hai (config gaming ka risk).
 - **History:** GPT-3 (2020) → 43.9% vs Human expert baseline 90%. GPT-4 (2023) → 86%. 2024 tak frontier models 86-92% range me cluster ho gaye → **saturated**.
 - **Kyun saturate hua:** Study me pata chala ~6.5% questions hi **galat/ambiguous** hain — isliye koi bhi 100% score nahi kar sakta, aur sab ek jagah cluster ho gaye.
 - **Kya measure NAHI karta:** Reasoning depth, calibration (model ko pata hai ki usse nahi aata?), open-ended generation, multilingual (English-only, Western curriculum-centric).
 - **Known issues:** Label errors (6.5%), high contamination (2020 se public hai), prompt format gaming.
-

(2) TruthfulQA — Reliability Branch

(Sept 2021)

- **Problem jo solve karna tha:** MMLU breadth test karta hai, lekin ye guarantee nahi karta ki model **truthful** hai. Internet par misconceptions bhi hain (jaise “knuckle cracking se arthritis hota hai”) — bade models zyada data absorb karte hain to zyada misconceptions bhi absorb kar sakte hain.
 - **Famous finding:** Bade/zyada capable models is benchmark pe **zyada fail** karte the — “capability ≠ alignment” wala idea yahin se popular hua.
 - **Dataset:** 817 adversarial questions, 38 categories — common human misconceptions ke around.
 - **Scoring 3 tareeke:** Generation (model khud A/B/C/D print kare), **MC1** (sirf ek sahi answer maan ke log-prob compare), **MC2** (multiple sahi answers ho sakte hain, unki probabilities add karke score — ye default/standard method hai).
 - **Run config quirk:** “Zero-shot” bola jaata hai lekin actually 6 fixed unrelated example questions har baar prompt me bhejte hain (static hone ki wajah se zero-shot maana jaata hai).
 - **Kya measure nahi karta:** Factual recall (kyunki MCQ hai), “honesty under pressure” (kya model pressure daalne par jaan-boojh kar galat boleگا — ye MASK benchmark check karta hai).
 - **Saturation:** 2024 tak saturate ho gaya kyunki RLHF/instruction-tuning jaisi alignment techniques aane se models zyada truthful ho gaye. Interesting: iska **contamination training stage pe nahi, alignment stage pe hota hai** — ye dataset alignment fine-tuning data ka part ban jaata hai.
 - **Replaced by:** SimpleQA aur MASK (alag-alag directions me).
-

(3) AGIEval — Human Exams Reuse Branch

(April 2023)

- **Philosophy:** Naya benchmark banane ke bajaye, **existing human exams** (SAT, LSAT, Gaokao — China ka exam) hi le lo aur unhe LLM ko do. Isse do fayde: (a) naya dataset banane ki mehnat nahi, (b) **real human baseline** milta hai (estimated nahi, actually measured).
- **Dataset:** 20 exam sections, 8000+ questions. Bilingual — English + Chinese (pehla bilingual benchmark).
- **Format:** 18/20 exams MCQ, 2 exams short-answer.
- **Results:** GPT-4 launch pe 58%, Human average 67%, Human toppers 91%. Isse pata chala LLMs abhi bhi top humans se kaafi peeche hain.
- **Important interview point:** Jab news aati hai “LLM ne IIT-JEE me humans ko beat kar diya” — iska matlab ye NAHI hota ki model human intelligence ko surpass kar gaya. Ek exam crack karna ≠ multi-step reasoning, tool use, long-horizon tasks handle karna. Ye sirf **exam-specific knowledge** test karta hai.

- **Saturation:** 2024-25 tak frontier models human topper baseline (91%) ke qareeb pahunch gaye → saturated, ab use nahi hota.

(4) GPQA — Google-Proof Q&A (Depth Branch)

(Nov 2023)

- **Philosophy:** MMLU breadth test karta hai lekin questions **easy** the. Rather than breadth, chalo **depth** test karte hain — sirf 3 subjects (Physics, Chemistry, Biology) lekin **PhD-level, research-grade** questions.
- **“Google-proof” naam ka matlab:** Questions itne difficult hain ki agar ek non-expert ko Google access bhi diya jaaye aur 30 minutes diye jaayein, tab bhi wo solve nahi kar payega.
- **Quality control:** Har question **2 domain experts** ne validate kiya hai.
- **3 subsets:** Main (448), Extended (546), **Diamond (198)** — sabse difficult, aur jab log “GPQA score X%” bolte hain to generally Diamond subset ka hi hota hai.
- **History:** GPT-4 (2023) → 39%. GPT-4o (2024) → 56% (Diamond). OpenAI’s o1 (reasoning model) → 78%. OpenAI PhD experts hire karke test kiya → 69.7% (paper me claim tha 81.3% — discrepancy hai, isliye “PhD ko beat kar diya” wale marketing claims pe pura trust mat karo). Grok 4 (2025) → ~87%.
- **Kya measure nahi karta:** General graduate knowledge (sirf 3 science subjects), open-ended problem solving, **reasoning trace correctness** (agar model sahi answer tuk-ke se bhi pahunch jaaye to bhi marks milte hain — reasoning process valid hai ya nahi wo check nahi hota).
- **Known issues:** Bahut kam questions (198 diamond) → statistical confidence kam. PhD baseline number disputed. Contamination badh raha hai (2-3 saal purana). Sirf 3 domains cover karta hai.
- **Status (2026):** Near saturation — reasoning models 80%+ score karne lage hain.

(5) MMLU-Pro — “MMLU Rebuilt to Fix Its Flaws”

24.

- **Philosophy:** MMLU ek achha product tha, sirf uski flaws fix karo — naya banane ke bajaye upgrade karo.
- **3 major changes MMLU se:**
 1. **4 options → 10 options** — elimination-based guessing mushkil ho jaata hai, difficulty badh jaati hai.
 2. **Trivia/noisy questions hataye, reasoning-based questions add kiye** — ab sirf fact-recall se kaam nahi chalega.
 3. **57 subjects → 14 broad categories** — better balanced representation, kisi subject ka under/over-representation nahi.
- **Proof ye changes kaam kar rahe the:** Reasoning-capable models non-reasoning models se **20 points aage** the — matlab ab sochne ki zaroorat pad rahi hai.
- **Fun fact:** “MMLU-Redux” ek benchmark **nahi** hai — ye ek paper hai jisne prove kiya ki MMLU me 6-8% questions incorrect hain — isi finding ne MMLU-Pro banane ko trigger kiya.
- **Dataset:** 12,000 questions, 14 disciplines.
- **Kya measure nahi karta:** Open-ended generation, reasoning trace correctness, calibration. Reasoning models ko **unfair advantage** milta hai (explicitly reasoning-heavy questions design kiye gaye).
- **Status (2026):** Nearing saturation — models 80-90% range me.

(6) SimpleQA — Reliability Branch v2 (Replaces TruthfulQA)

(2024, by OpenAI)

- **Kyun special hai:** Ye **abhi bhi active** hai (saturate nahi hua) — poore session me discuss kiye gaye 7 benchmarks me se sabse “alive” wala.
- **Format innovation:** Ye MCQ NAHI hai — **short-answer, fact-seeking questions**. Options nahi diye jaate, khud answer likhna/generate karna hota hai. Isse difficulty automatically badh jaati hai (subjective exams MCQ se harder hote hain).
- **Dataset:** 4326 short factual questions — aise questions jin par us waqt ka state-of-the-art model (GPT-4) **fail** hua

tha.

- **Unique innovation — Calibration bhi measure karta hai:** 3 categories me grade karta hai — **Correct, Incorrect, Not Attempted** (“I don’t know”). Agar model “pass/I don’t know” bolta hai to ye batata hai ki model apne knowledge ke limits ke baare me aware hai (hallucinate nahi kar raha).
- **3 metrics:** (a) **Correct** — overall kitne sahi, (b) **Correct given attempted** — sirf attempt kiye gaye questions me accuracy, (c) **F-score** — dono ka harmonic mean.
- **Result:** Same model jo MMLU pe 88% score karta tha, SimpleQA pe sirf ~40% score kar paaya — kaafi bada difference.
- **Kya measure nahi karta:** Long-form factuality (sirf short answers), everyday factual reliability (dataset hi rare/hard questions se bana hai — GPT-4 ki failures se).
- **Known issues:** LLM-as-judge use hota hai grading ke liye (judge khud improve hota rehta hai, isliye 2-saal-purane results se compare nahi kar sakte). **Answer staleness** (jaise “current world #1 rugby player” — answer time ke saath change ho jaata hai). GPT-4 ke against **adversarial/selection bias** (dataset hi GPT-4 ki failures pe based hai — dusre models ke liye fair nahi ho sakta).

(7) HLE — Humanities’ Last Exam (Breadth × Depth Combined)

(Jan 2025, still ACTIVE)

- **Core idea:** MMLU se **breadth** liya, GPQA se **depth** liya — dono ko combine karke ek massive benchmark banaya.
- **Naam ka matlab:** “Thesis, not marketing” — idea ye hai ki agar models is benchmark ko crack kar lein (closed-ended, broad, expert-level, unambiguous), to closed-ended Q&A test karne ko aur kuch bacha nahi — evaluation ko phir **open-ended, agentic tasks** ki taraf move karna padega.
- **Scale:** 2500 questions, **100+ subjects** (classics se rocket engineering tak), 1000+ experts, 500+ institutions, 50 countries ne contribute kiya — massive collaborative effort (typically benchmark ek chhoti research team se aata hai, ye alag hai).
- **Format:** 80% short-answer (generate karna hai), 20% MCQ. **10% questions multimodal hain** (image-based) — pehli baar kisi benchmark me visual data bhi hai.
- **Anti-contamination measure:** Ek **public set** (2500 questions, released) aur ek **private set** (institute ke paas, kabhi public nahi hoga) — naye models ko private set pe bhi test kiya ja sakta hai, taaki contamination check ho sake.
- **Calibration bhi measure karta hai:** Model se answer ke saath confidence % bhi maangte hain (e.g. “80% confident”), phir RMS calibration error nikalte hain.
- **History:** Jan 2025 launch, initially models single-digit % pe the. Grok 4 → 24%, GPT-5 → 25%, Gemini → 38% (2025). Gemini 3 Pro → 38%+ (2026 tak).
- **Kya measure nahi karta:** Open-ended/agentic problem solving, everyday usefulness, vision (sirf 10% hai) aur multilingual (English-only) capabilities poori tarah nahi.
- **Known issues:** Kuch questions disputed the → original 3000 se 2500 reh gaye. LLM-as-judge grading errors. **Failure-filter selection bias** — dataset hi 2024 ke frontier models ki failures se bana hai, isliye general/daily knowledge ko represent nahi karta.
- **Status:** Currently **the** state-of-the-art benchmark for testing knowledge + reasoning + math combined.

Quick Recap Table (Interview Cheat-Sheet)

Benchmark	Year	Tests	Format	Status (2026)
MMLU	2020	Breadth of knowledge	4-option MCQ, 57 subj	Saturated
TruthfulQA	2021	Reliability/Truthfulness	MC1/MC2/Gen	Saturated
AGIEval	2023	Human-exam comparison	MCQ + short-answer	Saturated
GPQA	2023	Depth (Physics/Chem/Bio)	4-option MCQ	Near saturation
MMLU-Pro	2024	Breadth + light reasoning	10-option MCQ, 14 disc	Near saturation
SimpleQA	2024	Factuality + Calibration	Short-answer (no options)	Active
HLE	2025	Breadth × Depth + Calibration	80% short-answer, 20% MCQ, 10% multimodal	Active

One-liner for interviews: “Knowledge benchmarks evolved from MMLU (breadth) → branched into reliability

(TruthfulQA→SimpleQA), human-exam comparison (AGIEval), depth (GPQA), and MMLU repair (MMLU-Pro) → all converged into HLE, which combines breadth and depth and also tests calibration, explicitly designed so that once models saturate it, evaluation must move to open-ended agentic tasks.”

“A benchmark is a standardized test with 4 parts — dataset, run config, scoring method, and aggregation — used to measure a specific LLM capability. But benchmarks aren’t perfect: contamination, saturation, config gaming, and aggregation gaming mean you should never blindly trust published numbers — always check who evaluated, under what conditions, and whether subject-level breakdowns exist.”

LLM Leaderboards — Interview Notes (Concept-Clear)

1. LLM Leaderboards Kya Hain?

Definition: Ek LLM leaderboard ek **public ranking/comparison table** hai jo dikhata hai ki different LLMs ek **common set of evaluations** pe kaisa perform karte hain. Simply bolen to — benchmark ek exam hai, aur uska result kahin publish/display karna hi **leaderboard** hai.

Leaderboards Kyun Exist Karte Hain? (4 Reasons)

1. **Common reference se models compare karna** — sabko same exam diya, kaun top, kaun last, turant pata chal jaata hai. Isse **trust** bhi milta hai — kyunki agar OpenAI khud bole “hum sabse achhe hain” to utna trust nahi hoga (self-interest hai), lekin agar ek **neutral third-party** yehi result de to zyada credible lagta hai.
2. **Model selection guide karna** jab khud evals run karne ke resources na ho — 100s models khud test karna bahut costly hai (time + money), isliye leaderboard se seedha top candidates uthaye ja sakte hain.
3. **Saturation detect karna** — agar top models ek jagah cluster (jaise 92-94%) karne lage, to samajh jaate hain ki benchmark saturate ho raha hai.
4. **Naye models discover karna** — top 3-4 to hamesha same rehte hain (OpenAI, Google, Anthropic), lekin thoda neeche scroll karne se naye/cheaper models mil jaate hain jo apne use-case ke liye kaam aa sakte hain.

Kaun Use Karta Hai Leaderboards? (5 Stakeholders)

1. **AI Engineers** — 100 candidates ko 5 tak shortlist karne ke liye. Task-specific leaderboard use karte hain (jaise math app banana hai to math leaderboard dekhenge).
2. **Frontier Labs** — competitive positioning aur **release gating** ke liye. Agar naya model competitor ke pichhle model ko bhi beat nahi kar pa raha, to release nahi karte — internal go/no-go signal hai. (*Interesting example: Google ka “Nano Banana” model stealth mode me leaderboard pe aaya tha bina naam reveal kiye — jab benchmarks pe achha perform kiya tabhi officially launch kiya gaya.*)
3. **Researchers** — ye check karne ke liye ki ek nayi technique genuinely help kar rahi hai ya nahi, ek shared baseline ke against.
4. **Policymakers & Safety Institutes** — capability trends track karte hain forecasting/regulation ke liye.
5. **Open-weight ecosystem** — **discovery** ke liye sabse underrated function hai. Ek 3-log ki chhoti lab bhi ek frontier model ke saath same table pe dikh sakti hai agar unka model kisi benchmark pe achha kare.

4 Types of LLM Leaderboards

(1) Benchmark-Specific Leaderboards

- Sirf **ek single benchmark** ke results pe ranking banate hain (jaise sirf MMLU, ya sirf HumanEval, ya sirf GSM8K, ya sirf GPQA).
- **Answer karta hai:** “Is benchmark pe kaunsa model sabse best hai?”
- **Limitation:** Bahut **narrow view** deta hai — overall model kaisa hai wo pata nahi chalta. Zyadatar famous benchmarks ka apna dedicated leaderboard hota hai (example: HLE ka apna leaderboard).

(2) Multi-Benchmark Leaderboards — *Sabse useful category*

- Multiple benchmarks aur evaluation dimensions ko combine karke ek **cumulative/overall score** dete hain — Knowledge, Reasoning, Math, Coding, Instruction-following, Data analysis, sab ek saath.
- Practical deployment factors bhi dikhaate hain: **cost per token, latency, output speed, context-window size, price-to-performance ratio**.
- **Answer karta hai:** “Kaunsa model capability + cost + performance ka strongest overall combination deta hai?”

- Examples: **LiveBench** (23 objective tasks across 7 categories, contamination-free), **Artificial Analysis** (dedicated leaderboards for intelligence, speed, cost — bahut exhaustive list, coding agents, speech, image, audio, hardware sab cover karte hain).

(3) Human-Preference-Based Leaderboards

- Yaha benchmark score nahi, **human votes** se ranking banti hai. User ko 2 anonymous models (A vs B) ke response dikhaye jaate hain, user decide karta hai kaunsa better hai. Bahut saare votes collect karne ke baad ranking banti hai.
- **Answer karta hai:** “Users kaunse model ke responses zyada prefer karte hain?”
- Measure karta hai: helpfulness, clarity, writing quality, creativity, tone, conversational ability.
- Example: **LMarena (Chatbot Arena)** — “battle mode” me anonymous models compare karte hain.
- **Limitation:** Human bias hota hai — log lambe, zyada confident, better-formatted, “agreeable” answers ko prefer karte hain, jo zaroori nahi ki **sabse accurate/safe/best-reasoned** ho. Voting population bhi shayad tumhare actual users ko represent na kare.

(4) Application-Specific Leaderboards

- Ek particular real-world domain/task ke around banaye jaate hain — jaise software bugs fix karna, API/tool calling, SQL query generation, medical questions, agentic tasks, long documents.
- **Answer karta hai:** “Is specific application ke liye kaunsa model/system best hai?”
- Example: **Berkeley Function-Calling Leaderboard (BFCL)** — sirf tool-calling capability ko rank karta hai. Multiple benchmarks use karta hai, lekin single domain me.

Leaderboards Ko Blindly Trust Kyun Nahi Kar Sakte (7 Reasons)

1. **Benchmark performance real applications me transfer nahi hoti** — benchmark questions clean, well-defined, self-contained hote hain. Real world me ambiguous requests, missing info, company-specific data, multi-turn conversations, tool failures, edge cases hote hain. (Analogy: Kaggle competitions jeetna $\neq$ achha real-world data scientist hona — Kaggle data bahut clean hota hai.)
2. **Benchmark contamination score inflate kar sakti hai** — public questions training data ka part ban sakte hain, isliye high score genuine capability, memorization, ya similar-questions-se-familiarity — teeno me se kuch bhi ho sakta hai.
3. **Models leaderboard ke liye over-optimize ho sakte hain (leaderboard overfitting)** — companies apne model ko specifically leaderboard pe achha dikhane ke liye tune karti hain (jaise LMarena pe jeetne ke liye “human-pleasing” responses generate karna sikhaya jaata hai — lambe, confident, chapalusi-wale answers). Isse related hai **Goodhart’s Law**: *“Jab ek measure target ban jaata hai, wo measure ke roop me kam useful ho jaata hai.”* (Analogy: agar ek car company sirf mileage improve karne pe focus kare, to overall car quality gir jaayegi.)
4. **Overall ranking subjective design choices pe depend karti hai** — multi-benchmark leaderboards ko decide karna padta hai: kaunse benchmarks include/exclude karne hain, scores kaise normalize karne hain, har capability ko kitna weight dena hai. Ye choices change karne se final ranking bhi change ho sakti hai — aur ye choices leaderboard ke ander **transparent nahi hoti** hamesha.
5. **Chhote score differences meaningful nahi hote** — Model A: 84.3, Model B: 84.1 — sirf 0.2 ka difference third rank aur fifth rank bana sakta hai. Ye difference prompt variation, small test set, ya evaluation noise se aa sakta hai. Agar **confidence intervals nahi diye gaye**, to adjacent ranks ko effectively **tied** samjho.
6. **Human-preference leaderboards me human bias hota hai** — jaisa upar discuss kiya, “better lagne wala” answer zaroori nahi “sabse sahi” answer ho.
7. **Scores stale, incomplete, ya self-reported ho sakte hain** — leaderboards me purane model versions ke results ho sakte hain, naye release-hue models missing ho sakte hain, discontinued models ke results reh sakte hain, ya company khud apne results submit kar sakti hai (unverified self-report).

AI Engineer Ki Tarah Leaderboards Kaise Read Karein (5-Step Framework)

Sabse important line: *“Leaderboards are a filtering tool, not a decision tool.”* — Leaderboard se seedha ek model select nahi karte, sirf top candidates **shortlist** karte hain.

Step 1 — Apni requirements pehle likho Leaderboard kholne se pehle clear karo: task type, latency budget, cost ceiling, context needs, deployment constraints (on-prem? open weights? data retention?). Ye karne se tum **rank #1 ki taraf blindly bias** nahi hoge.

Step 2 — Sahi board choose karo (famous wala nahi, apne task se match karne wala)

- Agents → SWE-bench Pro / Terminal-Bench / OSWorld
- Tool use → BFCL
- Chat → matching LMArena sub-arena
- RAG → MTEB (embedding models ki ranking)
- Budget-constrained → price aur throughput dikhaane wala board (jaise Artificial Analysis)

Step 3 — Board ko sahi se padho

- Exactly kya score ho raha hai? (Answer key, test suite, human vote, ya LLM judge?)
- Model kisne run kiya? (Independent evaluator ya vendor self-report?)
- Kis inference compute pe? (Reasoning-effort settings score aur bill dono change karti hain)
- Eval set kitna purana hai? (Static + public = contamination ka risk maano)
- Kya saturated hai? (Top 10 agar 2-3 points ke andar hain, to samajh lo ho chuka hai)
- Confidence intervals kahan hain? (Na ho to adjacent ranks ko tied maano)
- Agar composite hai — weights kya hain? (Unpublished weights = unpublished opinion)

Step 4 — 3-5 candidates tak cut karo Hard filters pehle apply karo: license, price ceiling, latency, apne region me availability.

Step 5 — Apna khud ka eval chalao (yahi asli decision hota hai) Apne real traffic se 50-few hundred examples lo — ye kisi bhi public benchmark se better hota hai. Quality, cost, latency, aur failure behaviour pe score karo.

One-liner for interviews: “Leaderboards are a filtering tool, not a decision tool — use them to narrow 100 models down to 3-5 candidates that match your task’s leaderboard type, then always run your own evaluation on your real traffic before making the final call.”

Custom Model Evals — Case Study (Text-to-SQL) — Interview Notes (Concept-Clear)

1. Recap — Ye Session Kahan Fit Hota Hai

LLM Evals do types ke hote hain:

- **Model Evals** — khud LLM ki capability test karna (benchmarks se ya custom data se)
- **Application Evals** — poore application (RAG, agent, etc.) ka evaluation

Model Evals ke andar bhi do types hain:

- **Benchmarks** — generic, standardized datasets jo specific capabilities (knowledge, reasoning, math) test karte hain. Sab models ke liye same test.
- **Custom Model Evals** — apne khud ke data/task pe LLM ko chalake dekhna ki *aapke specific application* ke liye kaunsa model best hai.

Yahi is session ka core topic hai: Custom Model Evals kaise run karte hain. Benchmarks se sirf top candidates milte hain (“filtering tool”), lekin final decision ke liye apna khud ka eval chalana padta hai — yehi is poore session ka practical demonstration hai, ek real case study ke through.

2. Case Study Setup — ESPNCricinfo “Ask Cricinfo”

Real-world problem: ESPNCricinfo (top cricket score website) pe live match ke dauraan users bahut saare questions poochte hain (jaise “*Virat Kohli ka Pakistan ke against batting average kya hai?*”). Pehle ek human analyst team query likh ke commentary team ko deti thi — lekin bade matches (India vs Pakistan) me ye scale nahi kar pa raha tha (5 min me 500+ questions).

Solution jo unhone banaya: “Ask Cricinfo” feature — user natural language me question poochta hai, system use SQL query me convert karta hai, database pe run karta hai, aur result seedha user ko dikhata hai. Isse human analyst ki zaroorat khatam ho gayi aur system infinitely scalable ho gaya.

Humara role (case study ke liye): Hum AI engineers hain jo Cricinfo ke liye kaam kar rahe hain. Humein ye feature banana hai — ek **Text-to-SQL application**. Sabse pehla aur sabse bada question: **kaunsa LLM use karein iske liye?** Ye random guess se decide nahi hota (“Fable use kar lete hain, abhi ka top model hai” — aisa nahi bolte companies me) — proper evaluation process follow karna padta hai.

Note: Ye RAG nahi hai — model ko external documents nahi diye ja rahe. Sirf database ka **schema** diya ja raha hai (jo hamesha same rahega), aur usi ke basis pe SQL generate karna hai.

3. The 3-Stage Flow (Model Selection Process)

Puri model-selection journey **3 stages** me hoti hai:

1. **Requirements gather karo** — task, cost, latency, context, deployment, correctness sab clearly likh lo.
2. **Leaderboards pe jao aur 5-10 candidate models shortlist karo** — requirements ke basis pe filter karo. (Yaad rakho: *leaderboards are a filtering tool, not a selection tool.*)
3. **In 5-10 models pe apna custom eval chalao** — jo model sabse best score kare, wahi final selection hai.

4. Stage 1 — Requirements Gathering (Detailed)

(a) Task

Technical level pe: **Text-to-SQL application** banani hai. Model ka kaam sirf itna hai — question + database schema dekh ke ek SQL query generate karna.

System prompt structure (jo LLM ko bheja jaata hai):

```
"You are a text-to-SQL generator. Given a database schema and a question, return a single SQL query that answers it using SQLite syntax. Return ONLY the SQL query, no explanation."
```

```
[Database Schema — matches table + deliveries table]
```

```
[User's Question]
```

- **matches** table: match details (teams, ground, date, umpire, etc.)
- **deliveries** table: ball-by-ball data (batsman, bowler, runs, wicket, etc.)
- Domain narrowed sirf **IPL** tak (na international, na women's cricket) — scope simplify karne ke liye.

(b) Cost Ceiling

Business manager se pucha gaya budget — is case study me **₹3 lakh/month** decide hua (illustrative number, actual class exercise me ₹5 lakh use hua tha).

Kyun ye feature banaya gaya (business angle): Pure retention play hai — jitna zyada time user website pe rukega, utne zyada ads dikhenge, utna revenue badhega. Isliye feature ka ROI justify karna padta hai — cost ceiling isi trade-off se aata hai.

Cost calculation kaise karein — step by step:

1. Per-query token count nikalo:

- Input tokens (system prompt + schema + question) $\approx$ **400 tokens**
- Output tokens (generated SQL query) $\approx$ **100 tokens**
- Ratio: **4:1** (input:output)

2. Daily volume estimate karo: Business judgment se — is case study me **~5000 questions/day** assume kiya gaya (traffic bahut spiky hoga — bade matches jaise CSK vs MI pe spike aayega, lekin average nikalna hai).

3. Formula for monthly cost (USD):

```
Per-query cost = (Input_price × Input_tokens / 1,000,000)
                + (Output_price × Output_tokens / 1,000,000)

Daily cost = Per-query cost × Daily_questions
Monthly cost (USD) = Daily cost × 30
Monthly cost (INR) = Monthly cost (USD) × 95    [USD→INR conversion]
```

4. Worked example — Claude Fable 5:

- Input: \$10/million tokens, Output: \$50/million tokens
- Per-query cost = $(10 \times 400/1M) + (50 \times 100/1M) = \$0.004 + \$0.005 = \mathbf{\$0.009/query}$
- Daily cost = $0.009 \times 5000 = \mathbf{\$45/day}$ → wait, actual calculation in transcript: $(400 \times 10 + 100 \times 50)/1,000,000 = 9000/1,000,000 = \0.009 , then $\times 5000 = \$45/day$
- *(Transcript's own arithmetic arrived at ~\$450/day → \$13,500/month → ₹12.82 lakh/month at the ₹95 conversion rate)*
- **Result:** ₹12.82 lakh/month — **4x from the ₹3 lakh budget**. Fable is rejected — “aukat ke bahar ka model hai.”
- Conclusion: Need to look at cheaper tier — something like **Claude Sonnet** (~1/4th the price) fits better.

Key lesson: Cost calculation ek core AI engineering skill hai — sirf model capability nahi, budget-fit bhi utna hi

important hai.

(c) Prompt Caching — Cost Optimization Concept

Concept: Agar prompt ka ek bada portion (system prompt + schema) **saari queries me same** rehta hai (sirf question part change hota hai), to model providers “**prompt caching**” offer karte hain — cache kiye gaye portion ka baar-baar poora price nahi dena padta.

- **First query** (cache miss): thoda **zyada** charge lagta hai (jaise normal \$10/M ki jagah \$12.5/M) — kyunki cache *processing* ka extra cost hai.
- **Subsequent queries** (cache hit, within refresh window): **bahut kam** charge (jaise \$10 ki jagah sirf \$1/M) — kyunki cached content ko re-process nahi karna padta.
- **Cache refresh cycles:** Providers alag options dete hain — jaise **5-minute cache** ya **1-hour cache**. Agar next query cache-window ke andar aaye, to cheap rate milta hai; window ke bahar aaye to cache expire ho jaata hai aur phir se full price lagta hai.
- **Kab kaunsa refresh cycle choose karein:** High-traffic application (jaise Cricinfo — questions har 5 min me aa rahe hain) → **5-minute cache** achha hai. Low-traffic application (jaise sirf 1000 questions/day) → **1-hour cache** better hai (kam frequent hits, lekin jab bhi hit ho to zyada dair tak cached rahega).
- **Underlying mechanism:** KV Caching (Key-Value caching) — attention mechanism ke K aur V vectors ko cache kiya jaata hai.
- **Impact:** Is optimization se cost me **~90% tak reduction** ho sakta hai (₹12 lakh se ₹6-7 lakh tak potentially).
- **Limitation:** RAG chatbots jaise applications me prompt caching utna kaam nahi karta, kyunki wahan context (retrieved documents) har query ke saath change hota rehta hai — koi repetitive fixed portion nahi hota.

(d) Latency

- Application ka nature dekhna zaroori hai — agar ye ek research assistant hai jo deep research karta hai, latency matter nahi karti. Lekin yahan users **live match ke dauraan curiosity se** question poochte hain — turant answer chahiye.
- **Target decided: 2-3 seconds** (5 seconds bhi bahut lagta hai users ko).
- **Implication:** Koi bhi high-latency model ya heavy reasoning model (jo lamba soch ke answer deta hai) is use-case ke liye suitable nahi hai.

(e) Context Window

- Is application ke liye **context window bada matter nahi karta**.
- **Reason:** Har question ek disjoint/independent query hai — koi multi-turn conversation nahi ho raha, koi session memory nahi chahiye. Isliye ek chhote context-window wala model bhi chalega.
- *(Contrast: agar ye ek chatbot ya coding assistant hota, to context window bahut matter karta.)*

(f) Deployment

- Question: Open-source/on-prem model chahiye ya public API (OpenAI, Anthropic) chalega?
- **Decision:** Koi sensitive/private data involved nahi hai, isliye **public APIs open hain** — Anthropic, OpenAI, koi bhi use kar sakte hain. Public APIs ka infrastructure/reliability generally better hota hai on-prem setup se.

(g) Correctness

- **Bahut critical requirement hai.** Cricket fans records ke baare me bahut **finicky** (particular) hote hain — agar galti se galat answer (jaise “sabse zyada runs Bumrah ne banaye”) aa gaya, to turant screenshot viral ho jaayega, brand embarrassment hoga.
- **Model hallucinate nahi kar sakta, incorrect nahi ho sakta** — is requirement ko coding leaderboards (kyunki SQL generation ek coding task hai) aur custom eval dono se verify karna hai.

Requirements Summary

Requirement	Decision
Task	Text-to-SQL application
Cost ceiling	≤ ₹3 lakh/month
Latency	2-3 seconds max
Context window	Not a major factor
Deployment	Public API preferred (better reliability)
Correctness	Extremely important — no hallucination allowed

5. Stage 2 — Leaderboard Shortlisting (5-10 Candidates)

Text-to-SQL Specific Leaderboards — Kyun Reject Kiye Gaye

Ideal to ye hota ki ek dedicated Text-to-SQL leaderboard use kiya jaaye. Kuch exist karte hain (jaise **BIRD-SQL**, **Spider**, **LiveSQLBench**), lekin ye sab reject kiye gaye:

- **BIRD-SQL**: Outdated (purane models, kuch fine-tuned models bhi listed the jo directly use nahi kiye ja sakte). Naming convention bhi arbitrary lagi (BIRD = “BIG” + “Large” se letters uthaye).
- **LiveSQLBench**: Bhi outdated tha (Claude 4.6 jaise purane models hi listed the), aur unka evaluation methodology clear nahi tha.

Fallback Strategy — Coding Leaderboard Use Karo

Key insight: SQL query generate karna ek **coding activity** hai. Isliye Text-to-SQL specific leaderboard ki jagah, ek **general coding leaderboard** use karo, aur coding capability ko SQL-generation-capability ka **alias/proxy** treat karo.

Chosen leaderboard: **LLMStats.com** — ye ek third-party website hai jo alag-alag leaderboards se data aggregate karke ek overall coding rating deti hai. Isme 146 models the, sabse recent models bhi update the, aur cost + speed + context jaisi practical info bhi thi.

Shortlisting Process (Step-by-Step)

Step 1 — Cost filter apply karo Har model ka blended per-month cost calculate kiya gaya (leaderboard ek **blended price** deta hai jo input:output ratio pe based hota hai, jaise **4:1**):

$$\text{Blended price} = (4 \times \text{Input_price} + 1 \times \text{Output_price}) / 5$$

- Important insight: Cricinfo application ka apna input:output token ratio bhi exactly **4:1** tha (400:100) — isliye ye blended number **directly** use ho sakta tha, alag se calculation ki zaroorat nahi thi.
- Formula: $\text{Monthly cost} = (\text{Total_tokens_per_query} \times \text{Blended_price} / 1,000,000) \times \text{Daily_questions} \times 30 \times 95$
- Isse **146 models** me se har ek ka monthly cost nikala gaya, aur jo bhi **₹3 lakh se zyada** the unhe **reject** kar diya gaya. ~50-60 models bache.

Step 2 — Remaining models ko rank karo (2 criteria pe)

1. **Coding capability** (leaderboard rating)
2. **Latency/speed** (characters per second — throughput)

Normalization: Dono metrics ko **0 to 1 range** me normalize kiya gaya (Min-Max normalization, jaisa machine learning me seekha hota hai):

$$\text{Normalized_value} = (\text{Value} - \text{Min}) / (\text{Max} - \text{Min})$$

Weighted scoring formula:

$$\text{Final_Score} = 0.9 \times \text{Normalized_Rating} + 0.1 \times \text{Normalized_Latency}$$

- **Coding capability ko 90% weight, latency ko sirf 10% weight** diya gaya.
- **Kyun itna kam weight latency ko?** Kyunki output sirf ek **chhoti SQL query hai (~100 tokens)** — chahe model

thoda slow ho, itni chhoti output print karne me zyada time nahi lagta. Agar output bahut bada (jaise ek pura essay) hota, tab latency/speed bahut zyada matter karta.

Result: In sab steps ke baad top-10 candidate models mile, jinme se cost-effective options bhi the (jaise open-source Chinese models — MiniMax M3, Qwen, GLM) jo expensive proprietary models (GPT, Claude) se bahut peeche nahi the score me, lekin fraction of the cost pe available the.

Final 5 Models Chosen for Custom Eval (live class demo ke liye):

1. **GPT 5.6 Tera** — top candidate on leaderboard (expensive but strong)
2. **Kimi K3** — Moonshot AI ka naya model, 2.7 trillion parameters, open-weights, bahut hype tha ki Fable ke qareeb hai 1/3rd cost pe
3. **Grok 4.5** — xAI ka model
4. **Claude Sonnet 5** — sole Anthropic-family representative (kyunki cheaper hai Tera se)
5. **MiniMax M3** — cheap Chinese open-source model

(Note: GLM aur Qwen jaise doosre Chinese models bhi consider kiye ja sakte the, lekin time-constraint ki wajah se sirf ek Chinese model represent kiya gaya. Meta ka Llama Spark model region-restriction ki wajah se exclude ho gaya — sirf US me available tha.)

6. Stage 3 — Running the Custom Eval (12-Step Pipeline)

Step 1 — Dataset Collect Karo

- IPL ka complete historical dataset (2008-2024) **Kaggle** se download kiya gaya.
- Simplification ke liye sirf **2020-2024** ka data rakha gaya (evaluation fast karne ke liye).
- Do CSV files: `matches.csv` aur `deliveries.csv`.

Step 2 — Data Ko Database Me Load Karo

- CSV data ko **SQLite database** me load kiya gaya (`db.py` script se) — sirf 2 tables (matches, deliveries), sirf 2020-2024 ka data.

Step 3 — Database Schema Extract Karo

- **Kyun zaroori hai?** Schema (table names, column names, types) system prompt me hamesha include karna hota hai, taaki LLM ko pata ho database structure kya hai. Isliye schema ko ek baar extract karke `schema.sql` file me store kar liya gaya (`schema_extractor.py` se).

Step 4 — Golden Dataset Banao

Golden dataset = 20-50 (yahan 20) manually-curated **question + verified-correct-SQL-query pairs**, jo evaluation ka **“answer key”** ban jaata hai.

- **Ideally** ye human data-analyst manually banata hai (question sochta hai, SQL query likhta hai, database pe run karke validate karta hai ki sahi answer aa raha hai).
- Is demo me, LLM se generate karwaya gaya (instruction diya gaya: **“bahut difficult questions ka set banao”**), phir sabko database pe run karke validate kiya gaya ki koi syntax error to nahi hai.
- **Golden dataset banate waqt important considerations:**
 - **Representative honi chahiye** actual user-question distribution ka — jaise mix of easy/medium/hard questions (e.g., 10 easy, 20 medium, 20 hard agar 50 ka set ho).
 - **Variety honi chahiye** query-types me bhi — joins, sub-queries, window functions, sab types cover karne chahiye.
- Har golden entry me: question, golden SQL query, **expected row count**, aur **order-sensitivity flag** (kya query me `ORDER BY` clause chahiye ya nahi) — ye validation ke liye extra metadata hai.
- Dataset size trade-off: **koi hard limit nahi hai**, lekin do practical constraints hain:
 1. Human effort/cost lagta hai bade golden dataset banane me.

2. Zyada questions = zyada LLM API calls during evaluation = zyada eval cost bhi.
 - Typical range: **50 se 500 questions** theek hai. Production me deploy hone ke baad, jahan model fail hota hai wahan se naye questions golden dataset me add karte rehte hain (continuous improvement).

Step 5 — Flow Ko Ek Baar Test Karo (Sanity Check)

- Poore pipeline ko ek single question + ek test model (jaise GPT-4o, jo candidate list me nahi hai) pe chalake dekha gaya ki system correctly ek SQL query wapas de raha hai ya nahi — koi bug to nahi hai setup me.

Step 6 — Multi-Provider API Access — OpenRouter

- **Problem:** 5 candidate models alag-alag providers (OpenAI, Kimi/Moonshot, xAI, Anthropic, MiniMax) se hain — sabki API alag hoti hai.
- **Solution: OpenRouter** platform — ek single API ke through kisi bhi provider ke kisi bhi model ko access kar sakte ho. LangChain ke saath direct integration bhi hai (`ChatOpenRouter` jaisa `ChatOpenAI` ki tarah). Free credits bhi milte hain testing ke liye.

Step 7 — Candidate Models List Banao

- Ek file me 5 tuples ki list — har tuple me model ka naam aur uska **OpenRouter slug** (jo API call ke liye zaroori hai).

Step 8 — Evaluator Build Karo (Sabse Important Step)

Core logic: Do result-tables (golden query ka output vs LLM-generated query ka output) ko compare karna, ye determine karne ke liye ki generated SQL sahi hai ya nahi.

Kyun string-matching (character-by-character comparison) sahi approach NAHI hai: Ek hi result achieve karne ke liye **multiple different SQL queries** likhi ja sakti hain (textually alag, lekin functionally same). Isliye **queries ka text compare** nahi karte — **unke results (output tables) ko compare** karte hain.

Comparison algorithm (step-by-step):

1. **Row count check** — pehle check karo dono tables me rows ki sankhya same hai ya nahi. Agar different hai, to seedha **mismatch** — aage check karne ki zaroorat nahi.
2. **Value normalization** — values ko normalize karo taaki formatting differences ignore ho jaayen:
 - `2.0` aur `2` — same treat karna chahiye.
 - `2.999` aur `2.99` jaise floating-point near-equal values — same treat karna chahiye.
3. **Row-by-row structuring** — dono tables ki values ko list-of-tuples format me organize karo.
4. **Sorting-based comparison** — agar query me `ORDER BY` **nahi hai** (order-sensitivity flag se pata chalega), to dono tables ko sort karke compare karo (kyunki row-order vary ho sakta hai even for correct results). Agar order matter karta hai, to **bina sort kiye** directly compare karo — taaki verify ho ki LLM ne sahi `ORDER BY` clause use kiya hai ya nahi.
5. Agar sab match ho jaaye → **query correct hai**. Nahi to → **incorrect**.

(Iske alawa aur bhi kuch smaller edge-case checks the jo evaluator me implement kiye gaye.)

Step 9 — Orchestration (Main Pipeline)

Sab kuch ek `main.py` file me orchestrate hota hai:

1. Schema load karo
2. Golden dataset load karo
3. Database se connect karo
4. **Har candidate model ke liye loop chalao:**
 - Golden dataset ke har question ko model ko bhejo
 - Model se generated SQL query lo
 - Golden query aur generated query — **donon ko database pe run karo**
 - Evaluator function call karo (dono results compare karke correct/incorrect batao)
 - Result log karo

5. Sab models ke liye repeat karo → final scores mil jaate hain

Depends on files: `schema.sql` (system prompt banane ke liye), golden dataset CSV, model-OpenRouter-slug list, `evaluator.py`.

7. Live Results — Kya Nikla (Case Study Outcome)

Model	Approx. Accuracy	Notes
GPT 5.6 Tera	~80% (4 errors/20)	Strong but expensive
Kimi K3	~50-55%	Bahut hype tha, lekin poor real-task performance + slow (reasoning model hone ki wajah se latency issue) — SQL syntax errors bhi aaye
Grok 4.5	~90%	Sabse fast, sabse accurate, sabse sasta bhi (~₹2.5 lakh/month, Tera se lagbhag aadha)
Claude Sonnet 5	~85%	Bahut fast, reliable, cost ~₹2.84 lakh/month
MiniMax M3	~65%	Sabse cheap, lekin SQL syntax errors aaye (Chinese open-source models me syntax errors zyada dekhe gaye — ek pattern jo class me observe hua)

Key Practical Lessons Jo Is Live Demo Se Nikle

- Leaderboard hype real-task performance guarantee nahi karta** — Kimi K3 bahut news me chhaya hua tha (“Fable ke qareeb, 1/3rd cost”), lekin actual custom eval me poorly perform kiya (slow + inaccurate).
“Leaderboard pe tehalka macha hua model, aapke specific task pe utna hi achha karega — is baat ki koi guarantee nahi hai.”
- Normalization vs Denormalization insight:** Analytical workloads (jaise ye reporting/analysis use-case) ke liye **denormalized tables** prefer kiye jaate hain (fewer joins needed → less error-prone SQL generation), jabki transactional databases (OLTP) ke liye normalization better hota hai. Cricinfo jaisi company apna production data likely already denormalized rakhti hogi analytics ke liye.
- Chinese open-source models me relatively zyada SQL syntax errors dekhe gaye** is particular test me (MiniMax, Kimi) — jabki American models (GPT, Grok, Claude) consistently valid syntax generate karte rahe.
- Statistical confidence badhane ka tarika:** Chhote golden dataset (20 questions) se ek galti bhi 5% score move kar deti hai. Isliye production-grade eval ke liye bada dataset chahiye, aur/ya **same eval ko multiple times run karke average nikalna chahiye** (kyunki har API call independent hai, results thoda vary kar sakte hain) — isse zyada statistical evidence milta hai.

Final Decision

Do finalists bache: **Grok 4.5** (fastest, cheapest, ~90% accuracy) vs **Claude Sonnet 5** (~85% accuracy, thoda costlier, lekin Anthropic ki API reliability/infrastructure zyada trusted maana gaya).

Trade-off discussion: Pure numbers ke hisaab se Grok aage hai, lekin vendor reliability/trust bhi ek valid factor hai model-selection me — isliye final call purely accuracy-driven nahi, balki **business judgment + engineering confidence** ka combination hota hai. Jab team consensus na bane, **voting** bhi ek valid mechanism hai companies me.

Real-world nuance: High-stakes/sensitive application (jaise medical) ke liye extra price pay karke zyada correctness wala model choose karte hain. Consumer-facing, low-stakes application ke liye thoda kam accurate lekin sasta model bhi chalega. **Decision hamesha application-context-dependent hota hai.**

8. Overall Process — Quick Recap (Interview Cheat-Sheet)

```
STAGE 1: Requirements Gathering
↓ (Task, Cost ceiling, Latency, Context, Deployment, Correctness)
STAGE 2: Leaderboard Shortlisting (5-10 candidates)
↓ (Pick relevant leaderboard → apply cost filter → weighted score on remaining criteria)
STAGE 3: Custom Eval Execution
↓ (Dataset → DB → Schema → Golden Dataset → Test flow → Multi-provider API →
    Evaluator logic → Orchestration → Run all models → Compare accuracy)
FINAL DECISION: Best-fit model for YOUR specific application
```

One-liner for interviews: *“Model selection for a real application follows a three-stage funnel — first write down concrete requirements (task, cost ceiling calculated from token economics, latency, context needs, deployment constraints, and correctness bar), then use leaderboards purely as a filtering tool to shortlist 5-10 candidates that meet those hard constraints, and finally run a custom evaluation — with a golden dataset and a result-set comparison (not string-matching) evaluator — on your own data to make the actual decision, because leaderboard hype does not guarantee task-specific performance.”*

RAG Application Evaluation — Strategy Overview — Interview Notes (Concept-Clear)

1. Recap — Playlist Me Ab Kahan Hain

Is poori LLM Eval playlist ka safar ab tak:

- **Fundamentals** — LLM eval kyun zaroori hai, reference-based vs reference-free, online vs offline eval.
- **Model Evals** — do types: **Benchmarks** (standardized, jaise Knowledge capability wale) aur **Custom Model Evals** (apna dataset banake chalana — jo pichhle session me case study se dikhaya gaya).

Ab se shuru ho raha hai playlist ka **sabse important part: Application Evals**.

LLM-based applications ke kai types ho sakte hain: simple chatbot, RAG-based chatbot, agent, multimodal app (jaise image generator), fixed-schema-output app (jaise Zomato wali case study jahan email ko classify kiya jaata tha support/refund/technical me).

Sab kuch cover karna possible nahi hai, isliye instructor ne sirf **2 topics** choose kiye:

1. **RAG** — kyunki professional journey me zyadatar chatbots RAG-functionality ke saath hi banenge.
2. **Agents** — ye bhi bahut important hain.

(Simple chatbot bina RAG/agent ke, aur multimodal apps — ye cover nahi kiye ja rahe, kyunki agar difficult topics samajh aa gaye to easy topics khud samajh aa jaate hain. Production me bhi zyadatar RAG aur agent design patterns hi dikhte hain.)

Agle 3-4 sessions RAG Eval pe honge — ek RAG chatbot banayenge aur usko properly evaluate karenge. Iske baad ek **famous GenAI interview question** ka confident jawab de paoge:

“How do you evaluate your RAG chatbot?”

Ye question ~10 me se 8 interviews me poocha jaata hai, aur zyadatar candidates iska structured jawab nahi de paate — ya to eval hi nahi padha, ya padha bhi hai to sirf 3-4 metric names bol dete hain bina framework ke.

2. Case Study Setup — CampusX Doubt Solver

Problem statement (jaan-boojh kar simple rakha gaya): Is LLM course ke har lecture ki **recording + transcript** milta hai. Idea hai — sab transcripts ko documents ki tarah use karke ek LLM ko feed karna, aur phir us LLM se poore playlist ke baare me koi bhi doubt poocha ja sake.

Banayenge: “CampusX Doubt Solver” — is specific LLM course (sirf isi course, kisi aur course ka nahi) ke transcripts pe based ek RAG chatbot.

(Basic RAG architecture recap: Retriever + Generator + Vector Database + Embedding model — query aati hai, retriever vector DB se top-k relevant documents fetch karta hai, generator un documents + query ke basis pe answer generate karta hai. Ye already pata hona chahiye, isliye deep explanation nahi ki gayi.)

Approach: Pehle chatbot banayenge, **saath-saath usko evaluate bhi karte jaayenge** — banane aur evaluate karne ka process ek saath chalta hai, alag-alag nahi.

3. Core Framework — Eval Suite: 3 Levels + Regression + Online

Jaisa software engineering me hota hai — aap poora application pehle bana ke fir test nahi karte, balki **feature-level aur function-level pe bhi test karte rehte hain** — waisa hi yahan hota hai. Iska matlab **components ko test karo, phir pipeline ko, phir poore application ko**.

Koi bhi LLM-based application **3 levels** pe evaluate hota hai:

Level 1 — Component Level

Jab RAG chatbot bana rahe ho, usi waqt uske individual **components ko isolation me evaluate** karte rehte ho (poore application ka wait nahi karte).

(a) Retriever (pehle banega)

- Kaam: Documents load karna → chunking karna → embedding model se vectors banana → naye query ke against relevant documents fetch karna.
- **Kaise evaluate karein:** Retriever ko isolation me test karo — matlab, ek query di gayi to kya sahi documents vector database se fetch ho rahe hain?
- **2 metrics:**
 - **Recall** — total sahi (relevant) documents me se kitne actually retrieve kiye gaye.
 - **Precision** — jitne documents retrieve kiye, unme se kitne genuinely useful/relevant the.
- Recall/precision satisfy hone ke baad hi aage badhte hain.

(b) Generator (uske baad banega)

- Kaam: Ek LLM hai jisko question + relevant context diya jaata hai, wo answer generate karta hai.
- **Important:** Generator ko bhi **isolation me** evaluate karte hain — is stage pe generator retriever se connected NAHI hota. Manually kuch questions + kuch context diya jaata hai (ek golden-dataset-jaisa setup), retriever se information nahi mil raha — kyunki abhi pipeline banaya hi nahi hai. Ye pure **component-level** evaluation hai.
- **3 metrics:**
 - **Faithfulness** — generated answer diye gaye context se hi aaya hai, ya model ne khud se galat info (**hallucination**) bana diya.
 - **Answer relevance** — generated answer, poochhe gaye question ke liye relevant hai ya nahi.
 - **Citation accuracy** — jaise ChatGPT kabhi-kabhi batata hai ki “ye information is document/link se li gayi hai” — waise hi humara Doubt Solver batayega ki ye particular question Nitish sir ne **kaunse session me** discuss kiya tha, aur us transcript ka link dega. Isi citation ki correctness ko verify karte hain.

Jab retriever aur generator dono isolation me achhe se kaam kar rahe hote hain, tab **Level 1 (Component Level) complete** hota hai.

Level 2 — Pipeline Level (“RAG Triad”)

Step: Ab retriever aur generator ko **connect** karte hain — ye connection hi RAG pipeline banata hai (code-wise bahut simple hai, bas dono ko jodna hai).

Ab check karte hain: Jab dono connect ho gaye, to poori pipeline sahi se kaam kar rahi hai ya nahi. Iske liye ek famous evaluation term hai — **RAG Triad**.

Kyun “Triad” (teen) bolte hain? Yahan 3 cheezein exist karti hain: (1) **User’s question**, (2) vector database se aayi hui **context**, (3) generator se bana **answer**. In teen ke beech **3 pairs** banate hain, aur har pair ke liye ek metric hota hai:

Pair	Metric	Kya batata hai
Question ↔ Context	Context Relevance	Retriever se aaya context, question ke liye relevant hai ya nahi
Context ↔ Answer	Faithfulness	Generated answer context se hi aaya, ya hallucination hai
Question ↔ Answer	Answer Relevance	Generated answer, poochhe gaye question ke liye relevant hai ya nahi

Ye teeno metrics ek saath test karte hain poore pipeline ke liye. Agar teeno sahi kaam kar rahe hain, matlab **RAG pipeline sahi se kaam kar raha hai** — aur Level 2 bhi complete ho gaya.

(Note: Faithfulness aur Answer Relevance yahan bhi wahi concepts hain jo Component Level ke Generator me the — lekin ab wo real retrieved context pe, end-to-end pipeline me test ho rahe hain, isolated hand-fed context pe nahi.)

Level 3 — Application/System Level

Ab poora application (Doubt Solver) test hota hai — matlab poori application sahi se kaam kar rahi hai ya nahi.

Quality-related metrics:

- **Correctness** — jo answer aaya hai, wo sahi hai ya nahi. Ye find out karna padta hai (golden answer se compare karke).
- **Completeness** — diya gaya answer, question ko **poori tarah** answer kar raha hai ya nahi. Example: agar question ke 2 parts hain aur answer sirf ek part ka jawab de raha hai, dusra part ignore ho gaya — to ye completeness me fail hai, chahe ek part ka answer sahi hi kyun na ho.
- **Style** — CampusX Doubt Solver ka explanation style, CampusX instructors jaisa hona chahiye (tone rubric ke through evaluate hota hai).

Safety-related metrics (bhi is level pe aate hain):

- Response **toxic** to nahi hai.
- Response me koi **PII (personally identifiable information)** leak to nahi ho raha.
- CampusX Doubt Solver ko **jailbreak** kiya ja sakta hai ya nahi.

Operations-related metrics:

- **Latency** — kitna time lag raha hai response dene me.
- **Cost per query** — per query kitna kharcha ho raha hai.
- **Tokens** — kitne tokens consume ho rahe hain.

In sab teen levels (Component + Pipeline + Application) ko milakar hum “Eval Suite” bolte hain — ye poore testing suite ki tarah hai. Application ko test karna ho, to individual tests run karne ke bajaye ye sara suite ek saath run karte hain, taaki har angle se pata chale ki application sahi kaam kar raha hai ya nahi.

Golden dataset ki zaroorat: Kuch evaluations ke liye golden dataset chahiye hota hai, kuch ke liye nahi (context depends karta hai kaunsi metric hai).

4. Tooling — DeepEval Library (Not Ragas)

Pichhle custom-model-eval session me pura evaluation code khud likha gaya tha. Lekin is poore RAG eval suite ko khud se code karna bahut bada project ban jaayega. Isliye ek **library** use karenge: **DeepEval**.

DeepEval me already available hain (koi custom code likhne ki zaroorat nahi):

- RAG ke liye: **Answer Relevancy, Faithfulness, Contextual Precision/Recall, Contextual Relevancy** — sab pehle se implemented.
- Safety ke liye: **Toxicity, PII leakage** — ye bhi already available.

DeepEval kyun choose kiya (Ragas ke bajaye) — 2 reasons:

1. **Ragas** already Advanced RAG course me padhaya ja chuka hai.
2. **DeepEval zyada broad-scope library hai** — sirf RAG tak limited nahi, balki **agents, multi-turn chatbots, non-LLM applications, image-based applications** sab ke liye use ho sakta hai. Isliye adoption bhi zyada hai, aur agle 1-2 saal me ye LLM evaluation ka **de-facto standard/benchmark library** ban sakta hai.

Best part: DeepEval ka poora syntax **PyTest** (Python software testing ki main library) pe based hai. Agar pehle kabhi PyTest use kiya ho software testing ke liye, to ye library bahut familiar lagegi.

5. Regression Testing — Eval Suite Ka Use

Ek baar poora Eval Suite ban jaaye, uske baad aata hai **Regression Testing** — Eval Suite ko poore application pe run karne ka process.

Simple definition: Agar software ka **Version 1** chal raha tha, aur ab aapne **Version 2** banaya, to kaise pata chalega ki Version 2 objectively Version 1 se **kharab to nahi** ho gaya? Yehi answer Regression Testing deta hai.

Kaise kaam karta hai: Poora Eval Suite ek baar run karo, ek **full report** milega — kaunse metric me value badhi, kaunse me ghati. Isse clear picture milta hai ki naya version purane se objectively behtar hai ya nahi — aur isi basis pe decide hota hai ki naya version deploy karna hai ya nahi.

Project Folder Structure (Practical Setup)

```
project/
├── src/                                # RAG chatbot ka source code
│   ├── retriever.py
│   ├── generator.py
│   ├── rag_pipeline.py
│   └── (FastAPI for API, Streamlit for UI)
├── evals/                             # Eval Suite — saari evaluation files
│   ├── eval_retriever.py
│   ├── eval_generator.py
│   ├── eval_pipeline.py              # (RAG Triad)
│   ├── eval_application.py
│   ├── eval_safety.py
│   └── eval_ops.py
└── run_evals.py                      # Sab evals ko trigger karke report banane wali file
```

`run_evals.py` trigger hote hi ye saari test files ko ek-ek karke application pe run karta hai aur ek report generate karta hai — us report se pata chalta hai naya version purane se behtar hai ya nahi.

Regression Testing Ke 3 Levels (Maturity Ladder)

Level 1 — Simple/Basic Regression Testing

- Software ek baar run karo, baseline numbers milte hain (jaise Recall = 82, Precision = 68).
- Har naye run ke baad numbers ko **manually** baseline se compare karo.

Level 2 — Experiment Tracking

- MLOps se familiar ho to **MLflow** jaisa tool yaad aayega, jo multiple experiments track karne ke liye use hota hai. Yahan bhi wahi concept hai.
- Configuration settings (jaise chunk size, chunk overlap, temperature, embedding model settings) ek jagah rakho, pehli baar Eval Suite run karo — ye **baseline** ban jaata hai.
- Fir improvement try karo (jaise chunk size badhao, overlap kam karo), phir se run karo — naya reading bhi log hota hai.
- Sab kuch (config + metric values) ek tool me log hota hai (jaise **MLflow**), aur ek **dashboard** milta hai jahan pichhle 10 runs ke metrics ko visually compare kar sakte ho.
- **Isi ka part hai:** Experiment tracking + dashboarding.

Level 3 — CI/CD Integration (Continuous Integration)

- Ek CI tool lo (jaise **GitHub Actions**). Condition set karo: jab bhi code me change karke git push karo (jaise chunk size badal ke), automatically poora **Eval Suite trigger** ho.
- Naya reading current baseline se compare hota hai. Ek **threshold** set kar sakte ho (jaise “3 points se zyada kam nahi hona chahiye”).
- Agar naya reading baseline se **kam** hai → deployment **rok do** (“software regress kar gaya, kharab ho gaya”).
- Agar naya reading **behtar** hai → deployment **allow** karo, aur baseline update ho jaata hai.
- Isse ek **gating mechanism** ban jaata hai — engineers bina chinta ke changes push kar sakte hain, kyunki deploy tabhi hoga jab metrics baseline se behtar honge.

*(Ye topic thoda LLMops/engineering domain me jaata hai, pure evaluation se thoda hatke — lekin related hone ki wajah se cover kiya gaya. Alternative tools: DeepEval ki apni company ka **Confident AI**, aur **Weights & Biases** bhi ye kaam karte hain. Abhi tak LLM evaluation ke liye koi single “de-facto standard” tool nahi bana hai — jaisa ML me MLflow ban gaya hai. Isliye tool pe fix mat ho jaana, **concept** samajhna zaroori hai — concept clear ho to koi bhi naya tool 1 hafte me seekha ja sakta hai.)*

6. Deployment Ke Baad — Online Evaluation

Regression testing pass hone ke baad, software **deploy** hota hai. Lekin **deployment ke baad bhi evaluation nahi rukta** — isi ko **Online Evaluation** bolte hain.

Live software pe 3-4 cheezein measure karte hain:

(a) Captured Signals (jaisa aata hai waisa hi store karte hain)

- Har student query ke response ki **latency**.
- **Cost** per query.
- User ne **thumbs up ya thumbs down** diya.

*(Tool suggestions jo class me diye gaye: **LangSmith**, **LangFuse**, aur **Confident AI** bhi ye kaam karta hai. LLM world me isko **Tracing/Observability** bolte hain — code me thoda tracing-code add karte hain, jisse live interactions ka data automatically capture hota rehta hai aur ek dashboard — jaise LangSmith ka — pe dikhta hai.)*

(b) Computed Values (kuch cheezein calculate bhi karni padti hain)

- Deployed hone ke baad bhi **faithfulness, answer relevance/correctness** jaisi wahi cheezein jo offline test ki thi, unhe **online** (live traffic pe) bhi check karte hain — usually LLM-judge se, sampled traffic pe.

(c) Drift

- **Drift ka matlab:** time ke saath application ki performance kharab to nahi ho rahi.
- Example: pichhle **24 ghante** ka ek graph rakhte hain (jaise Faithfulness ka). Agar pichhle 8 ghante me faithfulness graph achanak **neeche** gir jaaye — ye ek type ka **drift** hai jo detect hota hai, aur uske baad **alert** deke usko improve karne ka process shuru karte hain.

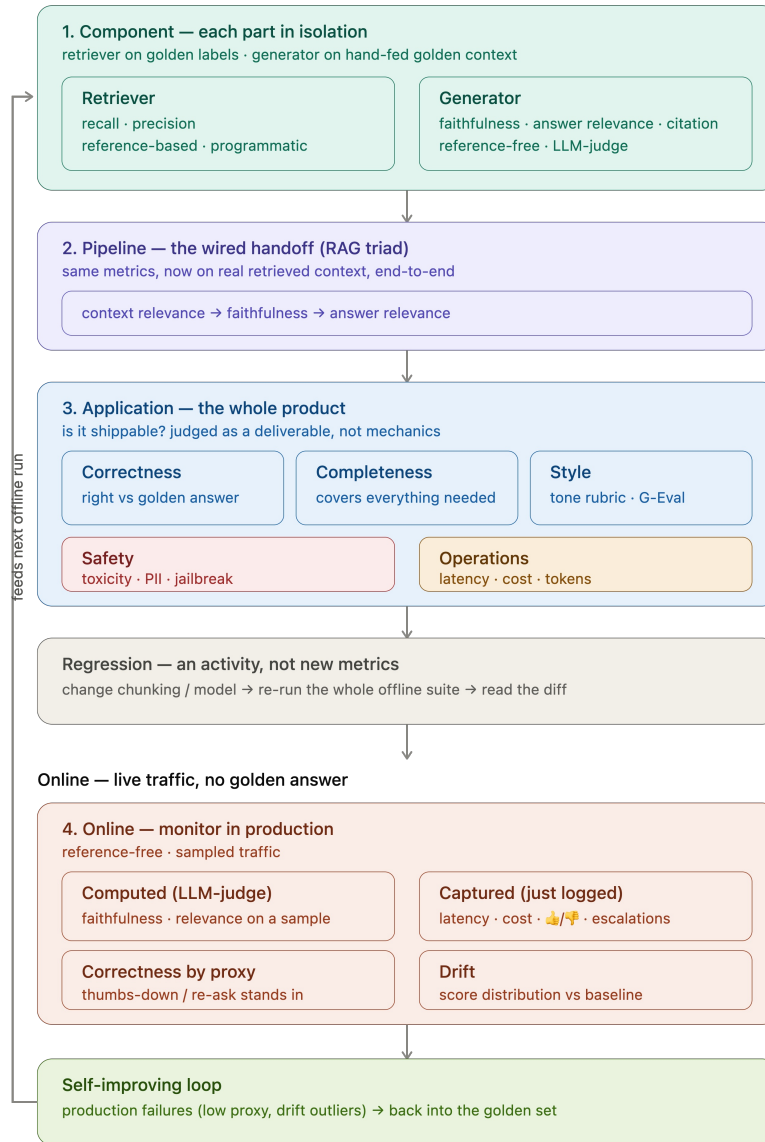
(d) Self-Improving Loop

- Kabhi-kabhi kisi student ke saath chat me application **galat behave** kar sakta hai. Aise specific examples ko utha ke **offline evaluation dataset (golden dataset) ka part** bana lete hain.
- Isse offline/golden datasets **aur zyada rich** hote jaate hain, taaki future me jab naya version banega aur offline evaluate hoga, to wo evaluation aur behtar ho.
- Ye ek **loop** ban jaata hai — production failures wapas golden set me feed hote hain.

7. Reference Flowchart — Full RAG Eval Pipeline

Neeche di gayi diagram poori strategy ko ek nazar me summarize karti hai — Offline (component → pipeline → application → regression) se lekar Online (production monitoring → self-improving loop) tak:

Offline — built bottom-up, before & at deploy



The spine: faithfulness & relevance recur at generator → pipeline → online.
What changes is the context they run on and whether you have an answer key.

RAG Evaluation Full Pipeline

Diagram ka core spine (jo isme highlight kiya gaya hai): **Faithfulness aur Relevance** baar-baar recur karte hain — pehle Generator (component level) me, phir Pipeline level (RAG Triad me), phir Online level me bhi. **Jo change hota hai wo hai: context kis type ka hai (hand-fed vs real-retrieved vs live-production), aur kya aapke paas answer-key (golden answer) available hai ya nahi.**

- **Offline** (built bottom-up, before & at deploy): Component → Pipeline → Application → Regression — sab kuch golden datasets/answer-keys ke against measure hota hai.
- **Online** (live traffic, no golden answer): Reference-free evaluation — sampled traffic pe LLM-judge se compute karte hain, kuch signals sirf capture/log karte hain, correctness ko proxy signals (thumbs-down, re-ask) se estimate karte hain, aur drift ko baseline distribution se track karte hain.
- **Self-improving loop**: Production failures wapas golden set me jaate hain, jo agla offline run behtar banata hai — cycle continue hota hai.

8. Roadmap — Agle 4 Sessions

Poori strategy explain karne ke baad, instructor ne agle 4 sessions ka plan bataya (practical implementation):

1. **Session 1** — Poori **Component Level Evaluation**. Retriever banana + test karna, phir Generator banana + test karna — isi process me pehli baar **DeepEval** use hoga.
2. **Session 2** — **RAG Pipeline banana** (retriever + generator connect karna) aur **RAG Triad** (3 metrics) run karna.
3. **Session 3** — **Application Level** pe poora evaluation run karna.
4. **Session 4** — **Regression Testing** aur **Online Evaluation** cover karna.

9. Interview-Ready Answer Template

Agar koi poochhe: **“How do you evaluate your RAG chatbot?”** — is tarah structured jawab do:

1. **“Main ek Evaluation Suite banaunga.”** → *“Eval Suite kya hai?”* → **“Main apna application 3 levels pe test karunga:”**
 - **Component Level**: Retriever (Recall, Precision) aur Generator (Faithfulness, Answer Relevance, Citation Accuracy) ko isolation me test karunga.
 - **Pipeline Level**: Retriever+Generator connect karke **RAG Triad** run karunga — Context Relevance, Faithfulness, Answer Relevance.
 - **Application Level**: Correctness, Completeness, Style (quality) + Toxicity, PII, Jailbreak (safety) + Latency, Cost, Tokens (operations).
2. **“Ye sab hone ke baad, main Regression Testing karunga”** — jiske 3 levels ho sakte hain: Basic, Experiment Tracking, ya CI/CD-based gating — company ki need ke hisaab se koi ek level choose karunga.
3. **“Regression testing pass hone ke baad naya baseline milega, main deploy karunga.”**
4. **“Deploy karne ke baad bhi main rukunga nahi — Online Evaluation run karunga”** — latency/cost/feedback capture, faithfulness/relevance compute (sampled), drift monitor.
5. **“Jahan bhi mistakes pakdi jaayengi, unhe collect karke apne offline evaluation dataset ka part bana lunga”** — continuous improvement loop.

Interview tip: Sirf 3-4 metric names bol dena (Recall, Precision, Answer Relevance) kaafi nahi hai — jo candidates poore **framework/structure** ke saath jawab dete hain (3 levels + regression + online + self-improving loop), unhi ko interviewer genuinely “ye deeply jaanta hai” maan ke respect karta hai.

One-liner for interviews: *“RAG chatbot evaluation follows a layered strategy — component-level testing of retriever and generator in isolation, pipeline-level testing via the RAG Triad (context relevance, faithfulness, answer relevance) once they’re wired together, and application-level testing of correctness, completeness, style, safety, and operations — all bundled into an eval suite that powers regression testing (basic → experiment-tracking → CI/CD-gated) before every deploy, followed by continuous online evaluation and a self-improving loop that feeds production failures back into the golden dataset.”*

RAG Application Evaluation — Strategy Overview — Interview Notes (Concept-Clear)

1. Recap — Playlist Me Ab Kahan Hain

Is poori LLM Eval playlist ka safar ab tak:

- **Fundamentals** — LLM eval kyun zaroori hai, reference-based vs reference-free, online vs offline eval.
- **Model Evals** — do types: **Benchmarks** (standardized, jaise Knowledge capability wale) aur **Custom Model Evals** (apna dataset banake chalana — jo pichhle session me case study se dikhaya gaya).

Ab se shuru ho raha hai playlist ka **sabse important part: Application Evals**.

LLM-based applications ke kai types ho sakte hain: simple chatbot, RAG-based chatbot, agent, multimodal app (jaise image generator), fixed-schema-output app (jaise Zomato wali case study jahan email ko classify kiya jaata tha support/refund/technical me).

Sab kuch cover karna possible nahi hai, isliye instructor ne sirf **2 topics** choose kiye:

1. **RAG** — kyunki professional journey me zyadatar chatbots RAG-functionality ke saath hi banenge.
2. **Agents** — ye bhi bahut important hain.

(Simple chatbot bina RAG/agent ke, aur multimodal apps — ye cover nahi kiye ja rahe, kyunki agar difficult topics samajh aa gaye to easy topics khud samajh aa jaate hain. Production me bhi zyadatar RAG aur agent design patterns hi dikhte hain.)

Agle 3-4 sessions RAG Eval pe honge — ek RAG chatbot banayenge aur usko properly evaluate karenge. Iske baad ek **famous GenAI interview question** ka confident jawab de paoge:

“How do you evaluate your RAG chatbot?”

Ye question ~10 me se 8 interviews me poocha jaata hai, aur zyadatar candidates इसका structured jawab nahi de paate — ya to eval hi nahi padha, ya padha bhi hai to sirf 3-4 metric names bol dete hain bina framework ke.

2. Case Study Setup — CampusX Doubt Solver

Problem statement (jaan-boojh kar simple rakha gaya): Is LLM course ke har lecture ki **recording + transcript** milta hai. Idea hai — sab transcripts ko documents ki tarah use karke ek LLM ko feed karna, aur phir us LLM se poore playlist ke baare me koi bhi doubt poocha ja sake.

Banayenge: “CampusX Doubt Solver” — is specific LLM course (sirf isi course, kisi aur course ka nahi) ke transcripts pe based ek RAG chatbot.

(Basic RAG architecture recap: Retriever + Generator + Vector Database + Embedding model — query aati hai, retriever vector DB se top-k relevant documents fetch karta hai, generator un documents + query ke basis pe answer generate karta hai. Ye already pata hona chahiye, isliye deep explanation nahi ki gayi.)

Approach: Pehle chatbot banayenge, **saath-saath usko evaluate bhi karte jaayenge** — banane aur evaluate karne ka process ek saath chalta hai, alag-alag nahi.

3. Core Framework — Eval Suite: 3 Levels + Regression + Online

Jaisa software engineering me hota hai — aap poora application pehle bana ke fir test nahi karte, balki **feature-level aur function-level pe bhi test karte rehte hain** — waisa hi yahan hota hai. Iska matlab **components ko test karo, phir pipeline ko, phir poore application ko**.

Koi bhi LLM-based application **3 levels** pe evaluate hota hai:

Level 1 — Component Level

Jab RAG chatbot bana rahe ho, usi waqt uske individual **components ko isolation me evaluate** karte rehte ho (poore application ka wait nahi karte).

(a) Retriever (pehle banega)

- Kaam: Documents load karna → chunking karna → embedding model se vectors banana → naye query ke against relevant documents fetch karna.
- **Kaise evaluate karein:** Retriever ko isolation me test karo — matlab, ek query di gayi to kya sahi documents vector database se fetch ho rahe hain?
- **2 metrics:**
 - **Recall** — total sahi (relevant) documents me se kitne actually retrieve kiye gaye.
 - **Precision** — jitne documents retrieve kiye, unme se kitne genuinely useful/relevant the.
- Recall/precision satisfy hone ke baad hi aage badhte hain.

(b) Generator (uske baad banega)

- Kaam: Ek LLM hai jisko question + relevant context diya jaata hai, wo answer generate karta hai.
- **Important:** Generator ko bhi **isolation me** evaluate karte hain — is stage pe generator retriever se connected NAHI hota. Manually kuch questions + kuch context diya jaata hai (ek golden-dataset-jaisa setup), retriever se information nahi mil raha — kyunki abhi pipeline banaya hi nahi hai. Ye pure **component-level** evaluation hai.
- **3 metrics:**
 - **Faithfulness** — generated answer diye gaye context se hi aaya hai, ya model ne khud se galat info (**hallucination**) bana diya.
 - **Answer relevance** — generated answer, poochhe gaye question ke liye relevant hai ya nahi.
 - **Citation accuracy** — jaise ChatGPT kabhi-kabhi batata hai ki “ye information is document/link se li gayi hai” — waise hi humara Doubt Solver batayega ki ye particular question Nitish sir ne **kaunse session me** discuss kiya tha, aur us transcript ka link dega. Isi citation ki correctness ko verify karte hain.

Jab retriever aur generator dono isolation me achhe se kaam kar rahe hote hain, tab **Level 1 (Component Level) complete** hota hai.

Level 2 — Pipeline Level (“RAG Triad”)

Step: Ab retriever aur generator ko **connect** karte hain — ye connection hi RAG pipeline banata hai (code-wise bahut simple hai, bas dono ko jodna hai).

Ab check karte hain: Jab dono connect ho gaye, to poori pipeline sahi se kaam kar rahi hai ya nahi. Iske liye ek famous evaluation term hai — **RAG Triad**.

Kyun “Triad” (teen) bolte hain? Yahan 3 cheezein exist karti hain: (1) **User’s question**, (2) vector database se aayi hui **context**, (3) generator se bana **answer**. In teen ke beech **3 pairs** banate hain, aur har pair ke liye ek metric hota hai:

Pair	Metric	Kya batata hai
Question ↔ Context	Context Relevance	Retriever se aaya context, question ke liye relevant hai ya nahi
Context ↔ Answer	Faithfulness	Generated answer context se hi aaya, ya hallucination hai
Question ↔ Answer	Answer Relevance	Generated answer, poochhe gaye question ke liye relevant hai ya nahi

Ye teeno metrics ek saath test karte hain poore pipeline ke liye. Agar teeno sahi kaam kar rahe hain, matlab **RAG pipeline sahi se kaam kar raha hai** — aur Level 2 bhi complete ho gaya.

(Note: Faithfulness aur Answer Relevance yahan bhi wahi concepts hain jo Component Level ke Generator me the —

lekin ab wo real retrieved context pe, end-to-end pipeline me test ho rahe hain, isolated hand-fed context pe nahi.)

Level 3 — Application/System Level

Ab poora application (Doubt Solver) test hota hai — matlab poori application sahi se kaam kar rahi hai ya nahi.

Quality-related metrics:

- **Correctness** — jo answer aaya hai, wo sahi hai ya nahi. Ye find out karna padta hai (golden answer se compare karke).
- **Completeness** — diya gaya answer, question ko **poori tarah** answer kar raha hai ya nahi. Example: agar question ke 2 parts hain aur answer sirf ek part ka jawab de raha hai, dusra part ignore ho gaya — to ye completeness me fail hai, chahe ek part ka answer sahi hi kyun na ho.
- **Style** — CampusX Doubt Solver ka explanation style, CampusX instructors jaisa hona chahiye (tone rubric ke through evaluate hota hai).

Safety-related metrics (bhi is level pe aate hain):

- Response **toxic** to nahi hai.
- Response me koi **PII (personally identifiable information)** leak to nahi ho raha.
- CampusX Doubt Solver ko **jailbreak** kiya ja sakta hai ya nahi.

Operations-related metrics:

- **Latency** — kitna time lag raha hai response dene me.
- **Cost per query** — per query kitna kharcha ho raha hai.
- **Tokens** — kitne tokens consume ho rahe hain.

In sab teen levels (Component + Pipeline + Application) ko milakar hum “Eval Suite” bolte hain — ye poore testing suite ki tarah hai. Application ko test karna ho, to individual tests run karne ke bajaye ye sara suite ek saath run karte hain, taaki har angle se pata chale ki application sahi kaam kar raha hai ya nahi.

Golden dataset ki zaroorat: Kuch evaluations ke liye golden dataset chahiye hota hai, kuch ke liye nahi (context depends karta hai kaunsi metric hai).

4. Tooling — DeepEval Library (Not Ragas)

Pichhle custom-model-eval session me pura evaluation code khud likha gaya tha. Lekin is poore RAG eval suite ko khud se code karna bahut bada project ban jaayega. Isliye ek **library** use karenge: **DeepEval**.

DeepEval me already available hain (koi custom code likhne ki zaroorat nahi):

- RAG ke liye: **Answer Relevancy, Faithfulness, Contextual Precision/Recall, Contextual Relevancy** — sab pehle se implemented.
- Safety ke liye: **Toxicity, PII leakage** — ye bhi already available.

DeepEval kyun choose kiya (Ragas ke bajaye) — 2 reasons:

1. **Ragas** already Advanced RAG course me padhaya ja chuka hai.
2. **DeepEval zyada broad-scope library hai** — sirf RAG tak limited nahi, balki **agents, multi-turn chatbots, non-LLM applications, image-based applications** sab ke liye use ho sakta hai. Isliye adoption bhi zyada hai, aur agle 1-2 saal me ye LLM evaluation ka **de-facto standard/benchmark library** ban sakta hai.

Best part: DeepEval ka poora syntax **PyTest** (Python software testing ki main library) pe based hai. Agar pehle kabhi PyTest use kiya ho software testing ke liye, to ye library bahut familiar lagegi.

5. Regression Testing — Eval Suite Ka Use

Ek baar poora Eval Suite ban jaaye, uske baad aata hai **Regression Testing** — Eval Suite ko poore application pe run karne ka process.

Simple definition: Agar software ka **Version 1** chal raha tha, aur ab aapne **Version 2** banaya, to kaise pata chalega ki Version 2 objectively Version 1 se **kharab to nahi** ho gaya? Yehi answer Regression Testing deta hai.

Kaise kaam karta hai: Poora Eval Suite ek baar run karo, ek **full report** milega — kaunse metric me value badhi, kaunse me ghati. Isse clear picture milta hai ki naya version purane se objectively behtar hai ya nahi — aur isi basis pe decide hota hai ki naya version deploy karna hai ya nahi.

Project Folder Structure (Practical Setup)

```
project/
├── src/                                # RAG chatbot ka source code
│   ├── retriever.py
│   ├── generator.py
│   ├── rag_pipeline.py
│   └── (FastAPI for API, Streamlit for UI)
├── evals/                              # Eval Suite — saari evaluation files
│   ├── eval_retriever.py
│   ├── eval_generator.py
│   ├── eval_pipeline.py                # (RAG Triad)
│   ├── eval_application.py
│   ├── eval_safety.py
│   └── eval_ops.py
└── run_evals.py                        # Sab evals ko trigger karke report banane wali file
```

`run_evals.py` trigger hote hi ye saari test files ko ek-ek karke application pe run karta hai aur ek report generate karta hai — us report se pata chalta hai naya version purane se behtar hai ya nahi.

Regression Testing Ke 3 Levels (Maturity Ladder)

Level 1 — Simple/Basic Regression Testing

- Software ek baar run karo, baseline numbers milte hain (jaise Recall = 82, Precision = 68).
- Har naye run ke baad numbers ko **manually** baseline se compare karo.

Level 2 — Experiment Tracking

- MLOps se familiar ho to **MLflow** jaisa tool yaad aayega, jo multiple experiments track karne ke liye use hota hai. Yahan bhi wahi concept hai.
- Configuration settings (jaise chunk size, chunk overlap, temperature, embedding model settings) ek jagah rakho, pehli baar Eval Suite run karo — ye **baseline** ban jaata hai.
- Fir improvement try karo (jaise chunk size badhao, overlap kam karo), phir se run karo — naya reading bhi log hota hai.
- Sab kuch (config + metric values) ek tool me log hota hai (jaise **MLflow**), aur ek **dashboard** milta hai jahan pichhle 10 runs ke metrics ko visually compare kar sakte ho.
- **Isi ka part hai:** Experiment tracking + dashboarding.

Level 3 — CI/CD Integration (Continuous Integration)

- Ek CI tool lo (jaise **GitHub Actions**). Condition set karo: jab bhi code me change karke git push karo (jaise chunk size badal ke), automatically poora **Eval Suite trigger** ho.
- Naya reading current baseline se compare hota hai. Ek **threshold** set kar sakte ho (jaise “3 points se zyada kam nahi hona chahiye”).
- Agar naya reading baseline se **kam** hai → deployment **rok do** (“software regress kar gaya, kharab ho gaya”).
- Agar naya reading **behtar** hai → deployment **allow** karo, aur baseline update ho jaata hai.
- Isse ek **gating mechanism** ban jaata hai — engineers bina chinta ke changes push kar sakte hain, kyunki deploy tabhi hoga jab metrics baseline se behtar honge.

(Ye topic thoda LLMops/engineering domain me jaata hai, pure evaluation se thoda hatke — lekin related hone ki wajah se cover kiya gaya. Alternative tools: DeepEval ki apni company ka **Confident AI**, aur **Weights & Biases** bhi ye kaam karte hain. Abhi tak LLM evaluation ke liye koi single “de-facto standard” tool nahi bana hai — jaisa ML me MLflow ban gaya hai. Isliye tool pe fix mat ho jaana, **concept** samajhna zaroori hai — concept clear ho to koi bhi naya

tool 1 hafte me seekha ja sakta hai.)

6. Deployment Ke Baad — Online Evaluation

Regression testing pass hone ke baad, software **deploy** hota hai. Lekin **deployment ke baad bhi evaluation nahi rukta** — isi ko **Online Evaluation** bolte hain.

Live software pe 3-4 cheezein measure karte hain:

(a) Captured Signals (jaisa aata hai waisa hi store karte hain)

- Har student query ke response ki **latency**.
- **Cost** per query.
- User ne **thumbs up ya thumbs down** diya.

(Tool suggestions jo class me diye gaye: **LangSmith**, **LangFuse**, aur **Confident AI** bhi ye kaam karta hai. LLM world me isko **Tracing/Observability** bolte hain — code me thoda tracing-code add karte hain, jisse live interactions ka data automatically capture hota rehta hai aur ek dashboard — jaise LangSmith ka — pe dikhta hai.)

(b) Computed Values (kuch cheezein calculate bhi karni padti hain)

- Deployed hone ke baad bhi **faithfulness**, **answer relevance/correctness** jaisi wahi cheezein jo offline test ki thi, unhe **online** (live traffic pe) bhi check karte hain — usually LLM-judge se, sampled traffic pe.

(c) Drift

- **Drift ka matlab:** time ke saath application ki performance kharab to nahi ho rahi.
- Example: pichhle **24 ghante** ka ek graph rakhte hain (jaise Faithfulness ka). Agar pichhle 8 ghante me faithfulness graph achanak **neeche** gir jaaye — ye ek type ka **drift** hai jo detect hota hai, aur uske baad **alert** deke usko improve karne ka process shuru karte hain.

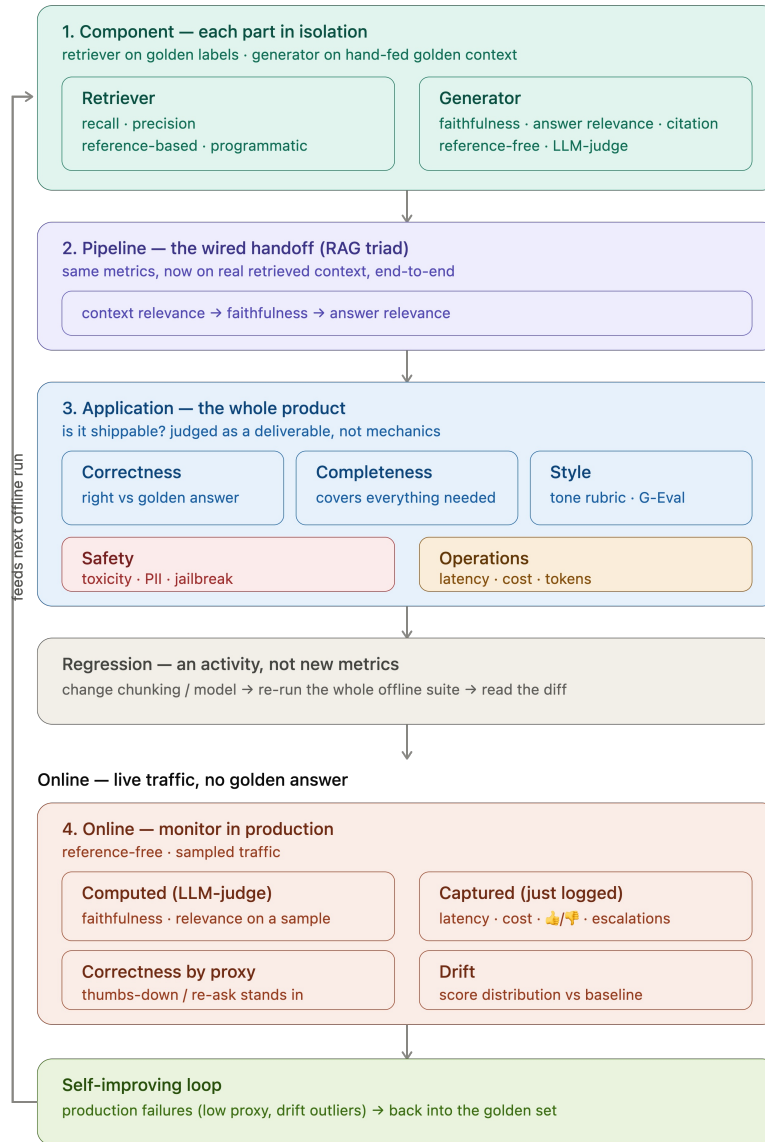
(d) Self-Improving Loop

- Kabhi-kabhi kisi student ke saath chat me application **galat behave** kar sakta hai. Aise specific examples ko utha ke **offline evaluation dataset (golden dataset) ka part** bana lete hain.
- Isse offline/golden datasets **aur zyada rich** hote jaate hain, taaki future me jab naya version banega aur offline evaluate hoga, to wo evaluation aur behtar ho.
- Ye ek **loop** ban jaata hai — production failures wapas golden set me feed hote hain.

7. Reference Flowchart — Full RAG Eval Pipeline

Neeche di gayi diagram poori strategy ko ek nazar me summarize karti hai — Offline (component → pipeline → application → regression) se lekar Online (production monitoring → self-improving loop) tak:

Offline — built bottom-up, before & at deploy



The spine: faithfulness & relevance recur at generator → pipeline → online.
What changes is the context they run on and whether you have an answer key.

RAG Evaluation Full Pipeline

Diagram ka core spine (jo isme highlight kiya gaya hai): **Faithfulness aur Relevance** baar-baar recur karte hain — pehle Generator (component level) me, phir Pipeline level (RAG Triad me), phir Online level me bhi. **Jo change hota hai wo hai: context kis type ka hai (hand-fed vs real-retrieved vs live-production), aur kya aapke paas answer-key (golden answer) available hai ya nahi.**

- **Offline** (built bottom-up, before & at deploy): Component → Pipeline → Application → Regression — sab kuch golden datasets/answer-keys ke against measure hota hai.
- **Online** (live traffic, no golden answer): Reference-free evaluation — sampled traffic pe LLM-judge se compute karte hain, kuch signals sirf capture/log karte hain, correctness ko proxy signals (thumbs-down, re-ask) se estimate karte hain, aur drift ko baseline distribution se track karte hain.
- **Self-improving loop**: Production failures wapas golden set me jaate hain, jo agla offline run behtar banata hai — cycle continue hota hai.

8. Roadmap — Agle 4 Sessions

Poori strategy explain karne ke baad, instructor ne agle 4 sessions ka plan bataya (practical implementation):

1. **Session 1** — Poori **Component Level Evaluation**. Retriever banana + test karna, phir Generator banana + test karna — isi process me pehli baar **DeepEval** use hoga.
2. **Session 2** — **RAG Pipeline banana** (retriever + generator connect karna) aur **RAG Triad** (3 metrics) run karna.
3. **Session 3** — **Application Level** pe poora evaluation run karna.
4. **Session 4** — **Regression Testing** aur **Online Evaluation** cover karna.

9. Interview-Ready Answer Template

Agar koi poochhe: **“How do you evaluate your RAG chatbot?”** — is tarah structured jawab do:

1. **“Main ek Evaluation Suite banaunga.”** → *“Eval Suite kya hai?”* → **“Main apna application 3 levels pe test karunga:”**
 - **Component Level**: Retriever (Recall, Precision) aur Generator (Faithfulness, Answer Relevance, Citation Accuracy) ko isolation me test karunga.
 - **Pipeline Level**: Retriever+Generator connect karke **RAG Triad** run karunga — Context Relevance, Faithfulness, Answer Relevance.
 - **Application Level**: Correctness, Completeness, Style (quality) + Toxicity, PII, Jailbreak (safety) + Latency, Cost, Tokens (operations).
2. **“Ye sab hone ke baad, main Regression Testing karunga”** — jiske 3 levels ho sakte hain: Basic, Experiment Tracking, ya CI/CD-based gating — company ki need ke hisaab se koi ek level choose karunga.
3. **“Regression testing pass hone ke baad naya baseline milega, main deploy karunga.”**
4. **“Deploy karne ke baad bhi main rukunga nahi — Online Evaluation run karunga”** — latency/cost/feedback capture, faithfulness/relevance compute (sampled), drift monitor.
5. **“Jahan bhi mistakes pakdi jaayengi, unhe collect karke apne offline evaluation dataset ka part bana lunga”** — continuous improvement loop.

Interview tip: Sirf 3-4 metric names bol dena (Recall, Precision, Answer Relevance) kaafi nahi hai — jo candidates poore **framework/structure** ke saath jawab dete hain (3 levels + regression + online + self-improving loop), unhi ko interviewer genuinely “ye deeply jaanta hai” maan ke respect karta hai.

One-liner for interviews: *“RAG chatbot evaluation follows a layered strategy — component-level testing of retriever and generator in isolation, pipeline-level testing via the RAG Triad (context relevance, faithfulness, answer relevance) once they’re wired together, and application-level testing of correctness, completeness, style, safety, and operations — all bundled into an eval suite that powers regression testing (basic → experiment-tracking → CI/CD-gated) before every deploy, followed by continuous online evaluation and a self-improving loop that feeds production failures back into the golden dataset.”*

RAG Component-Level Evaluation — Retriever — Interview Notes (Concept-Clear)

Code note: Is session ka poora code repo yahan available hai — <https://github.com/campusx-official/rag-eval-deepeval>. Neeche diye gaye code snippets transcript me jaisa dikhaya/explain kiya gaya tha usi structure aur function names (`build_retriever` , `load_store` , `load_transcripts` , chunk size/overlap values, `LLMTestCase` , metrics, `evaluate()`) ke basis pe likhe gaye hain — exact repo se copy-paste nahi hain. Real, runnable, up-to-date code ke liye upar diya gaya GitHub repo hi source of truth hai.

1. Recap — Ye Session Kahan Fit Hota Hai

Pichhle session me humne poora **roadmap** set kiya tha — RAG application ko **3 levels** pe evaluate karna hai: **Component Level** → **Pipeline Level** → **Application Level**. Jab ye teeno eval pipelines ban jaayengi, to unhe collectively **Eval Suite** bolte hain, jise aage **Regression Testing** me use karenge.

Aaj ka goal: Eval Suite ka **pehla part** — **Component Level** complete karna. RAG ke do sabse important components hote hain — **Retriever** aur **Generator**. Aaj sirf **Retriever** banayenge aur evaluate karna seekhenge (Generator agle session me).

- **Retriever** ka kaam: Ek query ke liye vector database se **relevant context** fetch karke lana.
- **Generator** ka kaam: Query + relevant context lekar ek **relevant answer** produce karna.

2. Project Setup

Naya project banaya gaya: `rag-eval-project`. Ye agle 2-3 classes tak chalega, isliye achhi tarah structure karna zaroori hai.

Standard Folder Structure

```
rag-eval-project/
├── data/           # Saari lecture transcripts (.vtt files) — hamara knowledge base
├── src/           # Application source code (retriever, generator, rag pipeline)
├── evals/         # Saare evaluation pipelines (eval_retriever.py, eval_generator.py, ...)
├── goldens/       # Golden datasets store honge
├── .env           # OPENAI_API_KEY
└── pyproject.toml # UV se dependency management
```

Setup Steps

1. **Data collect karo:** Har lecture ka transcript (`.vtt` format — subtitle files jaise, timestamp + spoken text) `data/` folder me daalo.
2. **Virtual environment banao** — **UV** use karke (Python 3.11). Dependencies install kiye: `langchain` , `openai` , `deepeval` , `pytest` (DeepEval ki dependency), `python-dotenv` .
3. **.env file banao** — OpenAI API key store karne ke liye.

Design philosophy note: Instructor ne jaan-boojh kar code ko **bahut simple/basic** rakha hai (production-

grade nahi) — taaki explain karna aasan ho aur ek glance me samajh aa jaaye. Ye ek conscious choice hai kyunki class ka focus **evaluation** pe hai, application-development best-practices pe nahi (jaise data ingestion aur retrieval ko decouple karna — jo ideally hona chahiye, lekin abhi simplify kiya gaya hai).

3. Retriever Build Karna

Golden rule (pichhle session se): Kabhi bhi poora application pehle bana ke phir test nahi karte — **software ki tarah**, module-by-module banate ho aur test karte jaate ho. Isliye order: **Retriever banao → evaluate karo → Generator banao → evaluate karo → Pipeline banao → evaluate karo → Application-level evaluate karo**.

Retriever Kya Karta Hai (Recap)

Ek query leta hai → embedding model se usse **vector** me convert karta hai → vector database me us vector ke **nearest K vectors** search karta hai → un vectors se corresponding chunks fetch karke **context** ke roop me return karta hai.

Code Structure (`src/retriever.py`)

```
# --- Imports & setup ---
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from dotenv import load_dotenv
import os

load_dotenv() # .env se OPENAI_API_KEY load karna

DATA_DIR = "data"
DB_DIR = "chroma_store" # Vector database yahan store hoga
EMBEDDING_MODEL = "text-embedding-3-small" # OpenAI ka embedding model
CHUNK_SIZE = 750
CHUNK_OVERLAP = 100
TOP_K = 5

def load_transcripts():
    """
    data/ folder ke andar sab .vtt files padhta hai, line-by-line reads karta hai.
    Timestamp wali lines ko IGNORE karta hai (sirf raw text rakhte hain —
    warna semantic meaning capture karna mushkil ho jaata hai, kyunki
    beech-beech me timestamps aane se text ka flow toot jaata hai).
    Har extracted line ko ek LangChain Document object me daalta hai,
    saath hi metadata me 'session number' bhi store karta hai —
    taaki aage chalke citation de sakein (kis session me ye baat boli gayi thi).
    """
    ...
    return documents

def load_store():
    """
    Check karta hai ki vector database (chroma_store) already exist
    karta hai ya nahi.
    - Agar EXIST karta hai -> naya database nahi banate (existing use karte hain).
    - Agar NAHI karta hai -> load_transcripts() se documents lo,
      chunk karo (size=750, overlap=100), embed karo (text-embedding-3-small),
      aur Chroma vector database me store karo.
    """
    ...

def build_retriever():
    """
    load_store() ko call karke vector database ready karta hai,
    fir usse ek retriever object banata hai jisme K=5 set hai
    (matlab har query ke liye top-5 nearest chunks fetch honge).
    """
    vectorstore = load_store()
    retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})
    return retriever

if __name__ == "__main__":
    retriever = build_retriever()
    query = "What is regression testing?"
    results = retriever.invoke(query)
    for doc in results:
        print(doc.page_content)
```

Kaise chalata hai: `retriever.invoke(query)` call karo → retriever query ko vector me convert karta hai → vector database se nearest-5 vectors fetch karta hai → unke chunks return karta hai.

Test run kiya gaya: Query — *“What is regression testing?”* — 5 chunks retrieve hue. Pehli baar chalane pe vector database create hone me thoda time lagta hai (kyunki chunks embed ho rahe hote hain).

Document changes handle karna: Agar naya lecture transcript aaya (jaise Session 10), to naya `.vtt` file `data/` me daalo, phir **existing `chroma_store` folder delete karo** aur `retriever.py` دوبارا chalao — isse naya embedding process trigger hoga aur naya vector database banega. *(Ideally data ingestion aur retrieval decoupled hone chahiye, lekin abhi eval-focus hai isliye simplify kiya gaya hai.)*

Is point pe: **Retriever ban gaya aur kaam kar raha hai** — lekin **kitna achha** kaam kar raha hai, ye abhi evaluate nahi kiya. Yahi agla step hai.

4. Retriever Ke 2 Failure Modes → 2 Core Metrics

Ye samajhna zaroori hai: **Retriever fail kaise hota hai?**

Retriever ek function hai — query leta hai, K relevant chunks return karta hai. Do tarah se ye fail ho sakta hai:

Failure Mode 1 — Sahi context bilkul nahi la paya Example: Question A/B/C ko sahi answer karne ke liye chunks 1, 15, aur 13 chahiye the (unhi me sahi info chhupi thi). Lekin retriever laaya chunks 27, 28, 29 — ek bhi relevant chunk nahi laya. Ye ek **bad retriever** hai.

Failure Mode 2 — Sahi context laaya, lekin noisy bhi laaya Example: Sahi context tha 1, 15, 13. Retriever laaya 1, 15, 27, 28, 29 — 2 sahi la paaya, lekin 3 fazool (noise) bhi le aaya. Ab ye noisy context bhi generator ko jaayega, jisse final answer kharab ho sakta hai.

In 2 Failure Modes Se 2 Metrics Nikalte Hain

Failure Mode	Metric	Definition
Sahi context miss karna	Recall	Vector DB me jitne total sahi/correct contexts available the, unme se retriever kitne la paaya
Noisy context laana	Precision	Retriever jo bhi contexts laaya, unme se kitne genuinely useful the

Example: 5 sahi contexts exist karte the, retriever 3 laaya → **Recall = 3/5**. 5 contexts laaya, 2 hi useful the → **Precision = 2/5**.

Ideal retriever: Recall bhi high, Precision bhi high — dono 1 (100%) ke jitna close.

Recall vs Precision — The Trade-off (K ke through)

Recall improve karne ka **quick hack: K ko badha do** (jaise 5 se 10). Isse chances badh jaate hain ki bache hue sahi chunks bhi mil jaayein — **Recall hamesha badhega**.

Lekin problem: K badhane se **Precision hit leta hai** — kyunki ab zyada chunks aayenge, jinme se zyada noise bhi honge. Jaise-jaise Recall 100% ki taraf jaata hai, Precision 0 ki taraf move karta hai.

Key insight: Recall aur Precision ke beech **tension/trade-off** hota hai, especially jab aap K ki value badalte ho. Dono ko ek saath badhaana thoda tricky kaam hai.

5. Ye Evals Reference-Based Hain (Important Distinction)

Quick check: Kya Recall aur Precision reference-based evaluations hain ya reference-free? — **Answer: Reference-based.**

Kyun: Recall/Precision measure karne ke liye aapko pata hona chahiye ki har query ke liye **“correct context” kaisa dikhta hai**. Isliye aapko ek **golden dataset** aur **golden context** chahiye. Jahan bhi “sahi answer/context kya hai” batana padta hai, wo evaluation **reference-based** ban jaata hai.

6. Golden Dataset Banane Ka GALAT Tareeka (Aur Kyun)

Galat Tareeka — Chunk ID Approach

Ek naive approach ye lagti hai: Golden dataset me 2 columns rakho — **Question** aur **Document/Chunk ID(s)**.

Example: | Question | Correct Chunk IDs | |—|—| | What is regression testing? | 72, 89, 100 | | What is RAG Triad? | 120, 111 | | What are online evals? | 151, 121, 130 |

Isse Recall/Precision calculate karte: Retriever ko query bhejo, jo chunk IDs wo laata hai unhe golden chunk IDs se compare karo.

Ye Tareeka Kyun GALAT Hai (Deep Reasoning)

Ye approach **kuch applications ke liye chalta hai** — jaise agar aapke documents ekdum **cleanly separated** hain (Document 1, Document 2 completely independent, unrelated) aur aap apne **chunking parameters kabhi change nahi karte**.

Lekin humare case me ye kaam nahi karega, 2 reasons se:

1. **Manually banana बहुत hectic hai:** Jisko ye task diya jaayega, use **800+ chunks** ko manually padh ke, har ek 50 questions ke liye batana padega ki kaun-kaun se chunks relevant hain. 50 questions × 800 chunks — koi ye kaam khushi se nahi karega. **Not scalable**.
2. **Chunking parameters change karte hi puri golden dataset VOID ho jaati hai** (asli bada reason): Jaise hi aap **chunk size ya overlap change karte ho** (recall improve karne ke liye, jo ek common technique hai), aapke chunks bhi change ho jaate hain — same information ab **alag chunk numbers** me chala jaata hai. 800 chunks the, ab shayad 700 ho gaye. Purani chunk-ID-based golden dataset ab **meaningless** ho gayi — usse dobara banana padega. Aur agar aap baar-baar tuning karte ho (jo normal hai), to har baar dobara golden dataset banana padega — **too much effort, not sustainable**.

Interview-worthy insight: “We should look at the actual content/transcript, not chunk-ID numbers” — iska asli reason ye NAHI hai ki humne numbers use kiye. Asli reason ye hai ki **chunk boundaries khud unstable hain** — chunking parameters tune karte hi wo shift ho jaate hain, isliye unpe dependency banana fragile hai.

7. Golden Dataset Banane Ka SAHI Tareeka — “Ideal Answer” Approach

Naya Golden Dataset Format

Do columns: **Question** aur **Ideal Answer** (chunk IDs nahi).

Important clarification: “Ideal Answer” ka matlab Google se liya hua answer NAHI hai. Ye wahi answer hai jo hamare **vector database ke content (transcripts) me actually taught kiya gaya** — usi content ko combine karke ek answer banaya gaya hai. Sahi/galat hona zaroori nahi, bas ye **course content ka faithful representation** hona chahiye.

Example: | Question | Ideal Answer | |—|—| | What is regression testing? | (3 chunks — 72, 89, 100 — combine karke bana answer) | | What is RAG Triad? | (2 chunks — 120, 111 — combine karke bana answer) |

Fayda: Chunk-Independence

Chunking parameters change karne se chunks reshuffle/change ho sakte hain — **lekin Ideal Answer nahi badlega**, kyunki wo already-written text hai, chunk numbers pe dependent nahi hai. Isliye **ek baar ki mehnat se baar-baar chunking-parameter-tuning ke saath golden dataset dobara banane ki zaroorat khatam** ho jaati hai.

DeepEval me is technique ka naam: Contextual Recall (aur Contextual Precision). Pehle “Recall @ K” wala simple concept padhaya gaya tha — ye alag hai, ye LLM-as-a-judge flavor wala version hai.

8. Contextual Recall — Kaise Calculate Hota Hai (Step-by-Step)

1. Question retriever ko bhejo → retriever K=5 chunks return karta hai (jaise 72, 81, 89, 99, 100).
2. Ek **LLM-as-a-judge** lao. Usse bolo: *"Ye Ideal Answer hai — isko **atomic claims** me break karo."* Jaise agar answer 3 sentences ka hai, to LLM 3 claims banayega (Claim 1, Claim 2, Claim 3).
3. Ab isi LLM ko bolo: *"Ab har retrieved chunk me jaao aur batao ki in 3 claims me se kaun-kaun se claims is chunk me exist karte hain."*
4. Jaise: Chunk 72 → Claim 1 mila. Chunk 81 → koi claim nahi. Chunk 89 → Claim 2 mila. Chunk 99 → koi nahi. Chunk 100 → Claim 3 mila.
5. Result: Ideal Answer ke **3 claims me se 3 hi kahin-na-kahin retrieved chunks me mil gaye** → **Recall = 3/3 = 1 (100%)**.

Doosra example: 2 claims the, sirf 1 hi retrieved chunks me mila → **Recall = 1/2 = 0.5**.

Har question ke liye ye process repeat karo, phir sab ka average nikalo → wahi **average Contextual Recall** hai.

Isme 2 Design Choices Chhupi Hain (Reliability Ke Liye)

- Golden dataset banane wala insaan aise likhega ki answer **atomic, easily-separable claims** se bana ho (taaki LLM-judge asaani se claims me tod sake).
- Ek **achhe quality ka LLM judge** use karna, jiska system instruction achhe se likha ho aur jo usse achhe se follow bhi kare.

9. Contextual Precision — Kaise Calculate Hota Hai (Rank-Aware)

Basic Version

Har retrieved chunk ke liye LLM-judge ko system prompt diya jaata hai:

"Here is the question. Here is the ideal answer. Here is one retrieved chunk. Is this chunk relevant? Does it contain information that helps produce that expected answer? Answer yes/no and give a reason."

Har chunk ke liye yes/no milta hai — jitne "yes" hain, unhe count karo → jaise 3 sahi, 2 noise → **Precision = 3/5**.

Advanced Version — Rank-Aware Precision (DeepEval Ka Actual Tareeka)

Important insight jo transcript me discuss hua: Sirf "kitne sahi hain" kaafi nahi — **rank/order bhi matter karta hai**.

Do cases compare karo: Dono me 5 chunks hain, dono me 2 sahi + 3 noisy. Simple precision formula se dono ka precision **same (2/5) aayega**. Lekin:

- **Case A:** Sahi 2 chunks **upar (rank 1, 2)** hain, noise neeche.
- **Case B:** Noise **upar** hai, sahi 2 chunks sabse **neeche (rank 4, 5)** hain.

Case A obviously ek behtar retriever hai — kyunki relevant content ko upar rank kiya. Lekin plain precision formula ye differentiate nahi kar paata. Isiliye **rank-aware calculation** chahiye.

Rank-Aware Calculation Kaise Hota Hai

Har position tak "running precision" nikalo — matlab **abhi tak dekhe gaye chunks me se kitne sahi hain**:

Case A (sahi-sahi-noise-noise-noise):

- Position 1: 1 chunk dekha, sahi → $1/1 = 1.0$
- Position 2: 2 dekhe, dono sahi → $2/2 = 1.0$
- Position 3: 3 dekhe, 2 sahi → $2/3 = 0.67$
- Position 4: 4 dekhe, 2 sahi → $2/4 = 0.5$
- Position 5: 5 dekhe, 2 sahi → $2/5 = 0.4$

- **Average** = $(1.0+1.0+0.67+0.5+0.4)/5 \approx 0.71$

Case B (noise-noise-noise-sahi-sahi):

- Position 1: $0/1 = 0$
- Position 2: $0/2 = 0$
- Position 3: $0/3 = 0$
- Position 4: $1/4 = 0.25$
- Position 5: $2/5 = 0.4$
- **Average** = $(0+0+0+0.25+0.4)/5 = 0.13$

Result: Case A ka precision (0.71) Case B (0.13) se **kaafi zyada aata hai** — jo sahi reflect karta hai ki Case A ek behtar retriever hai. Yehi hai **Contextual Precision** ka rank-aware calculation, jo DeepEval use karta hai.

Core idea: Precision ka basic concept to same hai (kitne useful hain), lekin isme ek dusra concept bhi incorporate kiya gaya hai — **kya sahi wale chunks upar rank ho rahe hain ya nahi**.

10. Golden Dataset Banane Ke 4 Tareeke

1. Hand-authored (khud se banao) — *Best method* Apko poora data ka knowledge hona chahiye. Instructor khud is position me tha (usne saari classes padhaayi thi), isliye khud manually questions + ideal-answers bana saka — session number yaad tha to us session ke relevant chunks me jaake answer construct kiya.

- **Pros:** Human judgment use hota hai, galti ka chance kam.
- **Cons: Not scalable** — 50 questions ke baad thak jaoge, ya kisi aur ko paisa dena padega.

2. LLM-Assisted Drafting — *Instructor ne is session ke liye yahi use kiya* Apna data Claude/LLM ko upload karo, usse bolo golden dataset banane ko (question + ideal answer format me). Fir **carefully human review** karo.

- **Pros:** Cost/time/effort kam.
- **Cons:** Galti hone ke chances badh sakte hain (LLM apni internet-knowledge se kuch add kar sakta hai jo class me nahi padhaya gaya) — isliye human review zaroori hai.
- Instructor ne 15 questions ek-ek karke banaye (batch me nahi — taaki quality control ho sake).

3. DeepEval Synthesizer Module DeepEval library ke andar ek **Synthesizer** class hai jo random chunks uthaake, khud se golden dataset generate kar sakti hai.

- **Instructor ka experience:** Achha nahi tha is particular application ke liye. Kai generated questions **student ki language jaisi nahi lagti thi** (jaise overly-academic/research-style phrasing), ya **course ke scope se bahar ke topics** (jaise ek specific case-study ke internal mechanics) pe focus kar rahe the. DeepEval ko ye pata nahi hota ki kahan emphasis dena hai — bas transcript se kuch bhi utha leta hai.
- Isliye **is current project ke liye use nahi kiya gaya**, lekin ye ek valid technique hai jo option available hai.

4. Production Logs Se App deploy hone ke baad, jin interactions me **positive signals** mile (jaise thumbs-up), unhe uthaake golden dataset me add karte jao — kyunki wahan retriever + chatbot sahi kaam kar raha tha.

- **Limitation:** Ye pehla method nahi ho sakta — shuru me bhi kuch entries chahiye hoti hain.

Is session ke liye use kiya gaya tareeka: Method 2 (LLM-Assisted). 15 questions banaye gaye (`retriever_goldens.json`), manually review kiye — questions “student-language” jaise lage (jaise *“What is an online eval and how is it different from offline eval?”*, *“Faithfulness vs groundedness kya hota hai?”*) aur answers transcripts se hi liye gaye the.

11. DeepEval Library — Core Structure (3 Cheezein)

Kisi bhi DeepEval code me primarily **3 cheezein** hoti hain:

(1) LLMTestCase

Ye aapke **golden dataset ki ek row** ko represent karta hai. Agar dataset me 15 rows hain, to 15 `LLMTestCase` objects banenge. Har test case me fields hoti hain:

- `input` — question
- `expected_output` — ideal/golden answer
- `retrieval_context` — retriever se aaye chunks (list)
- `actual_output` — generator ka output (agar generator involved nahi hai, to placeholder text daal dete hain)

(2) Metric

Kis metric pe evaluate karna hai — jaise `AnswerRelevancyMetric`, `ContextualPrecisionMetric`, `ContextualRecallMetric`. Har metric ke andar parameters set hote hain:

- `model` — LLM-as-a-judge kaunsa use hoga (jaise `gpt-4.1`)
- `threshold` — is score se neeche score aane par test case **fail** maana jaata hai
- `include_reason=True` — pass/fail ka reason bhi diya jaata hai

(3) evaluate() Function

Sab test cases aur metrics ko is function me pass karo — ye har test case ko har metric pe run karta hai aur score deta hai.

```
# General DeepEval pattern (illustrative)
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric
from deepeval import evaluate

test_case_1 = LLMTestCase(
    input="What is the capital of France?",
    actual_output="The capital of France is Paris."
)
test_case_2 = LLMTestCase(
    input="...",
    actual_output="..."
)

answer_relevancy_metric = AnswerRelevancyMetric(
    threshold=0.7,
    model="gpt-4.1",
    include_reason=True
)

evaluate(
    test_cases=[test_case_1, test_case_2],
    metrics=[answer_relevancy_metric]
)
```

12. Retriever Eval Code — Full Structure (`evals/eval_retriever.py`)

```

import json
from deepeval.test_case import LLMTestCase
from deepeval.metrics import ContextualRecallMetric, ContextualPrecisionMetric
from deepeval import evaluate
from src.retriever import build_retriever

GOLDEN_DATASET_PATH = "goldens/retriever_goldens.json"
JUDGE_MODEL = "gpt-4.1"
THRESHOLD = 0.7

# Golden dataset load karo (JSON file — 15 questions)
with open(GOLDEN_DATASET_PATH) as f:
    goldens = json.load(f)

retriever = build_retriever()

test_cases = []
for golden in goldens:
    question = golden["question"]
    ideal_answer = golden["ideal_answer"]

    # Retriever ko call karo, retrieved chunks ka text nikalo
    retrieved_docs = retriever.invoke(question)
    retrieved_context = [doc.page_content for doc in retrieved_docs]

    test_case = LLMTestCase(
        input=question,
        expected_output=ideal_answer,
        retrieval_context=retrieved_context,
        actual_output="Generator not evaluated in this run." # placeholder
    )
    test_cases.append(test_case)

contextual_recall = ContextualRecallMetric(
    threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True
)
contextual_precision = ContextualPrecisionMetric(
    threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True
)

evaluate(
    test_cases=test_cases,
    metrics=[contextual_recall, contextual_precision]
)

# Config logging (info ke liye)
print({
    "embedding_model": "text-embedding-3-small",
    "chunk_size": 750,
    "chunk_overlap": 100,
    "top_k": 5,
    "judge_model": JUDGE_MODEL,
    "golden_dataset": GOLDEN_DATASET_PATH,
})

```

Structure summary: (1) 15 rows se 15 `LLMTestCase` banaye — har ek me question, ideal answer, aur retriever se live-fetched context; (2) 2 metrics select kiye — `ContextualRecallMetric` aur `ContextualPrecisionMetric`; (3) `evaluate()` call karke sab kuch run kiya.

Run karne ka tareeka: Module import errors se bachne ke liye (`src` aur `evals` folders ke beech), dono folders me `__init__.py` files add ki gayin taaki unhe Python modules ki tarah treat kiya ja sake, aur command yeh use hua:

```
python3 -m evals.eval_retriever
```

13. Live Results — Baseline Aur Iterative Improvements

Baseline Run

Settings: `chunk_size=750`, `chunk_overlap=100`, `k=5`, embedding model = `text-embedding-3-small`.

Result: Recall = 80%, Precision = 80% — 15 me se 10 test cases pass, 5 fail (threshold 0.7 ke against).

Improvement Round 1 — Chunk Size Badhaya

Change: `chunk_size` 750→1000, `chunk_overlap` 100→150.

- **Reasoning:** Bade chunks me zyada context/information hoti hai, isliye claims milne ke chances badh jaate hain.
- **Zaroori step:** Chunking parameters badalne ka matlab hai chunks khud change ho jaayenge — isliye **existing** `chroma_store` **delete karke dobara embed karna padta hai**.

Result: Recall = 97%, Precision = 83% — dono improve hue, ab sirf 3 fail ho rahe (pehle 5 the).

Improvement Round 2 — Reranker Add Kiya

Reranker kya karta hai: Retriever ke fetched chunks ko leta hai aur unhe **re-rank** karta hai — sabse **meaningful** chunks ko upar le aata hai, non-meaningful (noise) ko neechे.

Kyun help karta hai: Jaisa humne discuss kiya, Contextual Precision **rank-aware** hai — sahi chunks upar honge to precision automatically badhega.

Code change minimal tha — `build_retriever()` ki jagah `build_reranking_retriever()` use kiya gaya (implementation same repo me `reranker.py` ke through available hai, HuggingFace ka ek sentence-transformer model power karta hai).

Result: Recall = 92%, Precision = 85% — precision thoda aur badha (83→85), sirf 2 fail ho rahe (pehle 3 the). Recall thoda gira (97→92) — trade-off expected tha.

Improvement Round 3 — Bada Embedding Model

Change: `text-embedding-3-small` → `text-embedding-3-large`.

- **Trade-off:** Cost badhega, lekin quality potentially better.

Result: Recall = 99%, Precision = 85% — recall aur zyada badh gaya, precision same reh gaya. 3 fail ho rahe (thoda variance — chhote dataset (sirf 15 rows) ki wajah se run-to-run thoda fluctuation aata hai, jo normal hai).

Improvement Round 4 — K Ghataya (Try Kiya)

Change: `k=5` → `k=3`.

- **Reasoning:** Kam chunks fetch karne se noise kam aayega, precision improve ho sakta hai (recall thoda hit lega).

Result: Precision **84%** (koi significant improvement nahi) — is particular setup me fayda nahi hua.

Final Summary Table

Iteration	Chunk Size/Overlap	Reranker	Embedding Model	K	Recall	Precision	Fails (of 15)
Baseline	750/100	No	text-embedding-3-small	5	80%	80%	5
Round 1	1000/150	No	text-embedding-3-small	5	97%	83%	3
Round 2	1000/150	Yes	text-embedding-3-small	5	92%	85%	2
Round 3	1000/150	Yes	text-embedding-3-large	5	99%	85%	3
Round 4	1000/150	Yes	text-embedding-3-large	3	~lower	84%	—

Final achieved: ~95%+ Recall aur **~85% Precision** — dono “kaafi achhe” mane gaye.

Aage aur improve karne ke potential raaste: Behtar quality ka reranking model use karna, chunking parameters (jaise 1500/200) ke saath aur experiment karna.

14. Session Ka Outcome

Is session me humne:

1. Ek RAG project setup kiya (proper folder structure ke saath).
2. **Retriever** build kiya (load transcripts → chunk → embed → vector DB → retriever object).
3. Retriever ke **2 failure modes** samjhe — missing relevant context, aur noisy context — jinse **Recall** aur **Precision** metrics nikalte hain.
4. Golden dataset banane ka **galat tareeka** (chunk-ID based) dekha aur samjha ki wo kyun fail karta hai (chunking-parameter changes se dataset void ho jaata hai).
5. **Sahi tareeka** seekha — Ideal-Answer-based golden dataset, jisse **Contextual Recall** (claim-based) aur **Contextual Precision** (rank-aware) calculate hote hain, dono LLM-as-a-judge use karke.
6. Golden dataset banane ke **4 techniques** dekhe (hand-authored, LLM-assisted, DeepEval synthesizer, production logs).
7. DeepEval ka core structure seekha — `LLMTestCase`, `Metric`, `evaluate()`.
8. Poora eval code likha aur chalaya, aur **iteratively retriever ko improve kiya** (chunk size → reranker → embedding model → K) — baseline 80/80 se final ~95%+/85% tak.

Confidence statement: Ab humein pata hai ki hamare retriever ki retrieval quality **genuinely achhi hai** (Recall 95%+, Precision ~85%) — is confidence ke saath hum aage **Generator** banayenge (agla session).

Quick Recap (Interview Cheat-Sheet)

Concept	One-liner
Retriever failure modes	(1) Relevant context miss karna, (2) Noisy context la lena
Recall	Total correct contexts me se kitne retrieve hue
Precision	Retrieved contexts me se kitne genuinely useful the
Recall vs Precision trade-off	K badhane se Recall ↑ lekin Precision ↓
Reference-based eval	Golden dataset/answer chahiye — Recall/Precision dono reference-based hain
Golden dataset — galat tareeka	Chunk IDs store karna — chunking params change hote hi void ho jaata hai
Golden dataset — sahi tareeka	Question + Ideal Answer (chunk-independent, reusable)
Contextual Recall	Ideal answer ko claims me todo, dekhne ho retrieved chunks me kitne claims mile
Contextual Precision	Rank-aware — sirf “kitne sahi” nahi, “sahi wale upar hain ya nahi” bhi check karta hai
DeepEval core 3 things	<code>LLMTestCase</code> (ek row), Metric (kya measure karna), <code>evaluate()</code> (run karna)
Improvement levers	Chunk size/overlap, Reranker, Embedding model, K value

One-liner for interviews: *“To evaluate a retriever, I’d measure Recall and Precision using DeepEval’s Contextual Recall and Contextual Precision metrics — both LLM-judge-based and reference-based, requiring a golden dataset of questions paired with ideal answers (not chunk IDs, since those become invalid every time chunking parameters change). Contextual Recall checks how many atomic claims from the ideal answer were found across retrieved chunks, while Contextual Precision is rank-aware — it rewards retrievers that rank the most relevant chunks higher, not just retrievers that happen to include them somewhere in the top-K.”*

RAG Component & Pipeline-Level Evaluation — Retriever, Generator, RAG Triad — Interview Notes (Concept-Clear)

Code note: Poora code repo yahan available hai — <https://github.com/campusx-official/rag-eval-deepeval>. Is note me di gayi code (Retriever, Generator, aur RAG Pipeline eval — teeno) **real repo files se hai** (verbatim, fabricated nahi). Sirf **Application-Level eval** (`eval_application.py`) ka class-transcript abhi tak nahi mila hai — uska code Section 27 (Appendix) me diya gaya hai, sirf reference ke liye.

1. Recap — Ye Session Kahan Fit Hota Hai

Pichhle session me humne poora **roadmap** set kiya tha — RAG application ko **3 levels** pe evaluate karna hai: **Component Level** → **Pipeline Level** → **Application Level**. Jab ye teeno eval pipelines ban jaayengi, to unhe collectively **Eval Suite** bolte hain, jise aage **Regression Testing** me use karenge.

Aaj ka goal: Eval Suite ka **pehla part — Component Level** complete karna. RAG ke do sabse important components hote hain — **Retriever** aur **Generator**. Aaj sirf **Retriever** banayenge aur evaluate karna seekhenge (Generator agle session me).

- **Retriever** ka kaam: Ek query ke liye vector database se **relevant context** fetch karke lana.
- **Generator** ka kaam: Query + relevant context lekar ek **relevant answer** produce karna.

2. Project Setup

Naya project banaya gaya: `rag-eval-project`. Ye agle 2-3 classes tak chalega, isliye achhi tarah structure karna zaroori hai.

Standard Folder Structure

```
rag-eval-project/
├── data/           # Saari lecture transcripts (.vtt files) — hamara knowledge base
├── src/            # Application source code (retriever, generator, rag pipeline)
├── evals/          # Saare evaluation pipelines (eval_retriever.py, eval_generator.py, ...)
├── goldens/        # Golden datasets store honge
├── .env            # OPENAI_API_KEY
└── pyproject.toml  # UV se dependency management
```

Setup Steps

1. **Data collect karo:** Har lecture ka transcript (`.vtt` format — subtitle files jaise, timestamp + spoken text) `data/` folder me daalo.
2. **Virtual environment banao** — **UV** use karke (Python 3.11). Dependencies install kiye: `langchain`, `openai`, `deepeval`, `pytest` (DeepEval ki dependency), `python-dotenv`.
3. **.env file banao** — OpenAI API key store karne ke liye.

Design philosophy note: Instructor ne jaan-boojh kar code ko **bahut simple/basic** rakha hai (production-grade nahi) — taaki explain karna aasan ho aur ek glance me samajh aa jaaye. Ye ek conscious choice hai kyunki

class ka focus **evaluation** pe hai, application-development best-practices pe nahi (jaise data ingestion aur retrieval ko decouple karna — jo ideally hona chahiye, lekin abhi simplify kiya gaya hai).

3. Retriever Build Karna

Golden rule (pichhle session se): Kabhi bhi pooraa application pehle bana ke phir test nahi karte — **software ki tarah**, module-by-module banate ho aur test karte jaate ho. Isliye order: **Retriever banao → evaluate karo → Generator banao → evaluate karo → Pipeline banao → evaluate karo → Application-level evaluate karo**.

Retriever Kya Karta Hai (Recap)

Ek query leta hai → embedding model se usse **vector** me convert karta hai → vector database me us vector ke **nearest K vectors** search karta hai → un vectors se corresponding chunks fetch karke **context** ke roop me return karta hai.

Code Structure (`src/retriever.py`)

```
# --- Imports & setup ---
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from dotenv import load_dotenv
import os

load_dotenv() # .env se OPENAI_API_KEY load karna

DATA_DIR = "data"
DB_DIR = "chroma_store" # Vector database yahan store hoga
EMBEDDING_MODEL = "text-embedding-3-small" # OpenAI ka embedding model
CHUNK_SIZE = 750
CHUNK_OVERLAP = 100
TOP_K = 5

def load_transcripts():
    """
    data/ folder ke andar sab .vtt files padhta hai, line-by-line reads karta hai.
    Timestamp wali lines ko IGNORE karta hai (sirf raw text rakhte hain —
    warna semantic meaning capture karna mushkil ho jaata hai, kyunki
    beech-beech me timestamps aane se text ka flow toot jaata hai).
    Har extracted line ko ek LangChain Document object me daalta hai,
    saath hi metadata me 'session number' bhi store karta hai —
    taaki aage chalke citation de sakein (kis session me ye baat boli gayi thi).
    """
    ...
    return documents

def load_store():
    """
    Check karta hai ki vector database (chroma_store) already exist
    karta hai ya nahi.
    - Agar EXIST karta hai -> naya database nahi banate (existing use karte hain).
    - Agar NAHI karta hai -> load_transcripts() se documents lo,
      chunk karo (size=750, overlap=100), embed karo (text-embedding-3-small),
      aur Chroma vector database me store karo.
    """
    ...

def build_retriever():
    """
    load_store() ko call karke vector database ready karta hai,
    fir usse ek retriever object banata hai jisme K=5 set hai
    (matlab har query ke liye top-5 nearest chunks fetch honge).
    """
    vectorstore = load_store()
    retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})
    return retriever

if __name__ == "__main__":
    retriever = build_retriever()
    query = "What is regression testing?"
    results = retriever.invoke(query)
    for doc in results:
        print(doc.page_content)
```

Kaise chalata hai: `retriever.invoke(query)` call karo → retriever query ko vector me convert karta hai → vector database se nearest-5 vectors fetch karta hai → unke chunks return karta hai.

Test run kiya gaya: Query — *“What is regression testing?”* — 5 chunks retrieve hue. Pehli baar chalane pe vector database create hone me thoda time lagta hai (kyunki chunks embed ho rahe hote hain).

Document changes handle karna: Agar naya lecture transcript aaya (jaise Session 10), to naya `.vtt` file `data/` me daalo, phir **existing `chroma_store` folder delete karo** aur `retriever.py` دوبارا chalao — isse naya embedding process trigger hoga aur naya vector database banega. *(Ideally data ingestion aur retrieval decoupled hone chahiye, lekin abhi eval-focus hai isliye simplify kiya gaya hai.)*

Is point pe: **Retriever ban gaya aur kaam kar raha hai** — lekin **kitna achha** kaam kar raha hai, ye abhi evaluate nahi kiya. Yahi agla step hai.

4. Retriever Ke 2 Failure Modes → 2 Core Metrics

Ye samajhna zaroori hai: **Retriever fail kaise hota hai?**

Retriever ek function hai — query leta hai, K relevant chunks return karta hai. Do tarah se ye fail ho sakta hai:

Failure Mode 1 — Sahi context bilkul nahi la paya Example: Question A/B/C ko sahi answer karne ke liye chunks 1, 15, aur 13 chahiye the (unhi me sahi info chhupi thi). Lekin retriever laaya chunks 27, 28, 29 — ek bhi relevant chunk nahi laya. Ye ek **bad retriever** hai.

Failure Mode 2 — Sahi context laaya, lekin noisy bhi laaya Example: Sahi context tha 1, 15, 13. Retriever laaya 1, 15, 27, 28, 29 — 2 sahi la paaya, lekin 3 fazool (noise) bhi le aaya. Ab ye noisy context bhi generator ko jaayega, jisse final answer kharab ho sakta hai.

In 2 Failure Modes Se 2 Metrics Nikalte Hain

Failure Mode	Metric	Definition
Sahi context miss karna	Recall	Vector DB me jitne total sahi/correct contexts available the, unme se retriever kitne la paaya
Noisy context laana	Precision	Retriever jo bhi contexts laaya, unme se kitne genuinely useful the

Example: 5 sahi contexts exist karte the, retriever 3 laaya → **Recall = 3/5**. 5 contexts laaya, 2 hi useful the → **Precision = 2/5**.

Ideal retriever: Recall bhi high, Precision bhi high — dono 1 (100%) ke jitna close.

Recall vs Precision — The Trade-off (K ke through)

Recall improve karne ka **quick hack: K ko badha do** (jaise 5 se 10). Isse chances badh jaate hain ki bache hue sahi chunks bhi mil jaayein — **Recall hamesha badhega**.

Lekin problem: K badhane se **Precision hit leta hai** — kyunki ab zyada chunks aayenge, jinme se zyada noise bhi honge. Jaise-jaise Recall 100% ki taraf jaata hai, Precision 0 ki taraf move karta hai.

Key insight: Recall aur Precision ke beech **tension/trade-off** hota hai, especially jab aap K ki value badalte ho. Dono ko ek saath badhaana thoda tricky kaam hai.

5. Ye Evals Reference-Based Hain (Important Distinction)

Quick check: Kya Recall aur Precision reference-based evaluations hain ya reference-free? — **Answer: Reference-based.**

Kyun: Recall/Precision measure karne ke liye aapko pata hona chahiye ki har query ke liye **“correct context” kaisa dikhta hai**. Isliye aapko ek **golden dataset** aur **golden context** chahiye. Jahan bhi “sahi answer/context kya hai” batana padta hai, wo evaluation **reference-based** ban jaata hai.

6. Golden Dataset Banane Ka GALAT Tareeka (Aur Kyun)

Galat Tareeka — Chunk ID Approach

Ek naive approach ye lagti hai: Golden dataset me 2 columns rakho — **Question** aur **Document/Chunk ID(s)**.

Example: | Question | Correct Chunk IDs | |—|—| | What is regression testing? | 72, 89, 100 | | What is RAG Triad? | 120, 111 | | What are online evals? | 151, 121, 130 |

Isse Recall/Precision calculate karte: Retriever ko query bhejo, jo chunk IDs wo laata hai unhe golden chunk IDs se compare karo.

Ye Tareeka Kyun GALAT Hai (Deep Reasoning)

Ye approach **kuch applications ke liye chalta hai** — jaise agar aapke documents ekdum **cleanly separated** hain (Document 1, Document 2 completely independent, unrelated) aur aap apne **chunking parameters kabhi change nahi karte**.

Lekin humare case me ye kaam nahi karega, 2 reasons se:

1. **Manually banana बहुत hectic hai**: Jisko ye task diya jaayega, use **800+ chunks** ko manually padh ke, har ek 50 questions ke liye batana padega ki kaun-kaun se chunks relevant hain. 50 questions × 800 chunks — koi ye kaam khushi se nahi karega. **Not scalable**.
2. **Chunking parameters change karte hi puri golden dataset VOID ho jaati hai** (asli bada reason): Jaise hi aap **chunk size ya overlap change karte ho** (recall improve karne ke liye, jo ek common technique hai), aapke chunks bhi change ho jaate hain — same information ab **alag chunk numbers** me chala jaata hai. 800 chunks the, ab shayad 700 ho gaye. Purani chunk-ID-based golden dataset ab **meaningless** ho gayi — usse dobara banana padega. Aur agar aap baar-baar tuning karte ho (jo normal hai), to har baar dobara golden dataset banana padega — **too much effort, not sustainable**.

Interview-worthy insight: “We should look at the actual content/transcript, not chunk-ID numbers” — iska asli reason ye NAHI hai ki humne numbers use kiye. Asli reason ye hai ki **chunk boundaries khud unstable hain** — chunking parameters tune karte hi wo shift ho jaate hain, isliye unpe dependency banana fragile hai.

7. Golden Dataset Banane Ka SAHI Tareeka — “Ideal Answer” Approach

Naya Golden Dataset Format

Do columns: **Question** aur **Ideal Answer** (chunk IDs nahi).

Important clarification: “Ideal Answer” ka matlab Google se liya hua answer NAHI hai. Ye wahi answer hai jo hamare **vector database ke content (transcripts) me actually taught kiya gaya** — usi content ko combine karke ek answer banaya gaya hai. Sahi/galat hona zaroori nahi, bas ye **course content ka faithful representation** hona chahiye.

Example: | Question | Ideal Answer | |—|—| | What is regression testing? | (3 chunks — 72, 89, 100 — combine karke bana answer) | | What is RAG Triad? | (2 chunks — 120, 111 — combine karke bana answer) |

Fayda: Chunk-Independence

Chunking parameters change karne se chunks reshuffle/change ho sakte hain — **lekin Ideal Answer nahi badlega**, kyunki wo already-written text hai, chunk numbers pe dependent nahi hai. Isliye **ek baar ki mehnat se baar-baar chunking-parameter-tuning ke saath golden dataset dobara banane ki zaroorat khatam** ho jaati hai.

DeepEval me is technique ka naam: Contextual Recall (aur Contextual Precision). Pehle “Recall @ K” wala simple concept padhaya gaya tha — ye alag hai, ye LLM-as-a-judge flavor wala version hai.

8. Contextual Recall — Kaise Calculate Hota Hai (Step-by-Step)

1. Question retriever ko bhejo → retriever K=5 chunks return karta hai (jaise 72, 81, 89, 99, 100).
2. Ek **LLM-as-a-judge** lao. Usse bolo: *"Ye Ideal Answer hai — isko **atomic claims** me break karo."* Jaise agar answer 3 sentences ka hai, to LLM 3 claims banayega (Claim 1, Claim 2, Claim 3).
3. Ab isi LLM ko bolo: *"Ab har retrieved chunk me jaao aur batao ki in 3 claims me se kaun-kaun se claims is chunk me exist karte hain."*
4. Jaise: Chunk 72 → Claim 1 mila. Chunk 81 → koi claim nahi. Chunk 89 → Claim 2 mila. Chunk 99 → koi nahi. Chunk 100 → Claim 3 mila.
5. Result: Ideal Answer ke **3 claims me se 3 hi kahin-na-kahin retrieved chunks me mil gaye** → **Recall = 3/3 = 1 (100%)**.

Doosra example: 2 claims the, sirf 1 hi retrieved chunks me mila → **Recall = 1/2 = 0.5**.

Har question ke liye ye process repeat karo, phir sab ka average nikalo → wahi **average Contextual Recall** hai.

Isme 2 Design Choices Chhupi Hain (Reliability Ke Liye)

- Golden dataset banane wala insaan aise likhega ki answer **atomic, easily-separable claims** se bana ho (taaki LLM-judge asaani se claims me tod sake).
- Ek **achhe quality ka LLM judge** use karna, jiska system instruction achhe se likha ho aur jo usse achhe se follow bhi kare.

9. Contextual Precision — Kaise Calculate Hota Hai (Rank-Aware)

Basic Version

Har retrieved chunk ke liye LLM-judge ko system prompt diya jaata hai:

"Here is the question. Here is the ideal answer. Here is one retrieved chunk. Is this chunk relevant? Does it contain information that helps produce that expected answer? Answer yes/no and give a reason."

Har chunk ke liye yes/no milta hai — jitne "yes" hain, unhe count karo → jaise 3 sahi, 2 noise → **Precision = 3/5**.

Advanced Version — Rank-Aware Precision (DeepEval Ka Actual Tareeka)

Important insight jo transcript me discuss hua: Sirf "kitne sahi hain" kaafi nahi — **rank/order bhi matter karta hai**.

Do cases compare karo: Dono me 5 chunks hain, dono me 2 sahi + 3 noisy. Simple precision formula se dono ka precision **same (2/5) aayega**. Lekin:

- **Case A:** Sahi 2 chunks **upar (rank 1, 2)** hain, noise neeche.
- **Case B:** Noise **upar** hai, sahi 2 chunks sabse **neeche (rank 4, 5)** hain.

Case A obviously ek behtar retriever hai — kyunki relevant content ko upar rank kiya. Lekin plain precision formula ye differentiate nahi kar paata. Isiliye **rank-aware calculation** chahiye.

Rank-Aware Calculation Kaise Hota Hai

Har position tak "running precision" nikalo — matlab **abhi tak dekhe gaye chunks me se kitne sahi hain**:

Case A (sahi-sahi-noise-noise-noise):

- Position 1: 1 chunk dekha, sahi → $1/1 = 1.0$
- Position 2: 2 dekhe, dono sahi → $2/2 = 1.0$
- Position 3: 3 dekhe, 2 sahi → $2/3 = 0.67$
- Position 4: 4 dekhe, 2 sahi → $2/4 = 0.5$
- Position 5: 5 dekhe, 2 sahi → $2/5 = 0.4$

- **Average** = $(1.0+1.0+0.67+0.5+0.4)/5 \approx 0.71$

Case B (noise-noise-noise-sahi-sahi):

- Position 1: $0/1 = 0$
- Position 2: $0/2 = 0$
- Position 3: $0/3 = 0$
- Position 4: $1/4 = 0.25$
- Position 5: $2/5 = 0.4$
- **Average** = $(0+0+0+0.25+0.4)/5 = 0.13$

Result: Case A ka precision (0.71) Case B (0.13) se **kaafi zyada aata hai** — jo sahi reflect karta hai ki Case A ek behtar retriever hai. Yehi hai **Contextual Precision** ka rank-aware calculation, jo DeepEval use karta hai.

Core idea: Precision ka basic concept to same hai (kitne useful hain), lekin isme ek dusra concept bhi incorporate kiya gaya hai — **kya sahi wale chunks upar rank ho rahe hain ya nahi**.

10. Golden Dataset Banane Ke 4 Tareeke

1. Hand-authored (khud se banao) — *Best method* Apko poora data ka knowledge hona chahiye. Instructor khud is position me tha (usne saari classes padhaayi thi), isliye khud manually questions + ideal-answers bana saka — session number yaad tha to us session ke relevant chunks me jaake answer construct kiya.

- **Pros:** Human judgment use hota hai, galti ka chance kam.
- **Cons: Not scalable** — 50 questions ke baad thak jaoge, ya kisi aur ko paisa dena padega.

2. LLM-Assisted Drafting — *Instructor ne is session ke liye yahi use kiya* Apna data Claude/LLM ko upload karo, usse bolo golden dataset banane ko (question + ideal answer format me). Fir **carefully human review** karo.

- **Pros:** Cost/time/effort kam.
- **Cons:** Galti hone ke chances badh sakte hain (LLM apni internet-knowledge se kuch add kar sakta hai jo class me nahi padhaya gaya) — isliye human review zaroori hai.
- Instructor ne 15 questions ek-ek karke banaye (batch me nahi — taaki quality control ho sake).

3. DeepEval Synthesizer Module DeepEval library ke andar ek **Synthesizer** class hai jo random chunks uthaake, khud se golden dataset generate kar sakti hai.

- **Instructor ka experience:** Achha nahi tha is particular application ke liye. Kai generated questions **student ki language jaisi nahi lagti thi** (jaise overly-academic/research-style phrasing), ya **course ke scope se bahar ke topics** (jaise ek specific case-study ke internal mechanics) pe focus kar rahe the. DeepEval ko ye pata nahi hota ki kahan emphasis dena hai — bas transcript se kuch bhi utha leta hai.
- Isliye **is current project ke liye use nahi kiya gaya**, lekin ye ek valid technique hai jo option available hai.

4. Production Logs Se App deploy hone ke baad, jin interactions me **positive signals** mile (jaise thumbs-up), unhe uthaake golden dataset me add karte jao — kyunki wahan retriever + chatbot sahi kaam kar raha tha.

- **Limitation:** Ye pehla method nahi ho sakta — shuru me bhi kuch entries chahiye hoti hain.

Is session ke liye use kiya gaya tareeka: Method 2 (LLM-Assisted). 15 questions banaye gaye (`retriever_goldens.json`), manually review kiye — questions “student-language” jaise lage (jaise *“What is an online eval and how is it different from offline eval?”*, *“Faithfulness vs groundedness kya hota hai?”*) aur answers transcripts se hi liye gaye the.

11. DeepEval Library — Core Structure (3 Cheezein)

Kisi bhi DeepEval code me primarily **3 cheezein** hoti hain:

(1) LLMTestCase

Ye aapke **golden dataset ki ek row** ko represent karta hai. Agar dataset me 15 rows hain, to 15 `LLMTestCase` objects banenge. Har test case me fields hoti hain:

- `input` — question
- `expected_output` — ideal/golden answer
- `retrieval_context` — retriever se aaye chunks (list)
- `actual_output` — generator ka output (agar generator involved nahi hai, to placeholder text daal dete hain)

(2) Metric

Kis metric pe evaluate karna hai — jaise `AnswerRelevancyMetric`, `ContextualPrecisionMetric`, `ContextualRecallMetric`. Har metric ke andar parameters set hote hain:

- `model` — LLM-as-a-judge kaunsa use hoga (jaise `gpt-4.1`)
- `threshold` — is score se neeche score aane par test case **fail** maana jaata hai
- `include_reason=True` — pass/fail ka reason bhi diya jaata hai

(3) evaluate() Function

Sab test cases aur metrics ko is function me pass karo — ye har test case ko har metric pe run karta hai aur score deta hai.

```
# General DeepEval pattern (illustrative)
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric
from deepeval import evaluate

test_case_1 = LLMTestCase(
    input="What is the capital of France?",
    actual_output="The capital of France is Paris."
)
test_case_2 = LLMTestCase(
    input="...",
    actual_output="..."
)

answer_relevancy_metric = AnswerRelevancyMetric(
    threshold=0.7,
    model="gpt-4.1",
    include_reason=True
)

evaluate(
    test_cases=[test_case_1, test_case_2],
    metrics=[answer_relevancy_metric]
)
```

12. Retriever Eval Code — Real Repo Code (`evals/eval_retriever.py`)

Ye **exact code hai jo repo me hai** (verbatim, koi change nahi):

```

import json

from dotenv import load_dotenv

from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import ContextualRecallMetric, ContextualPrecisionMetric

from src.retriever import build_retriever

load_dotenv()

GOLDEN_PATH = "goldens/retriever_goldens.json"
JUDGE_MODEL = "gpt-4.1-mini"
THRESHOLD = 0.7

# 1. LOAD the golden set --- the fixed, human-authored truth
with open(GOLDEN_PATH) as f:
    goldens = json.load(f)

# 2. RUN THE RETRIEVER on each question to fill retrieval_context,
#     then build one test case per golden.
retriever = build_retriever()

test_cases = []

for g in goldens:
    retrieved = retriever.invoke(g["query"])
    retrieval_context = [doc.page_content for doc in retrieved]

    test_cases.append(
        LLMTestCase(
            input=g["query"],
            expected_output=g["ideal_answer"],
            retrieval_context=retrieval_context,
            actual_output="(generator not evaluated in this run)",
        )
    )

# 3. THE METRICS --- recall (did we miss?) and precision (ranked well?)
metrics = [
    ContextualRecallMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
    ContextualPrecisionMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
]

# 4. EVALUATE --- every metric on every case, batched + parallel, with a printed report
evaluate(
    test_cases=test_cases,
    metrics=metrics,
    hyperparameters={
        "retriever": "base_k5", # vs "reranked" when you swap it in
        "embedding_model": "text-embedding-3-small",
        "chunk_size": 1000,
        "chunk_overlap": 150,
        "top_k": 5,
        "judge_model": JUDGE_MODEL,
        "golden_set": GOLDEN_PATH,
    },
)

```

Structure breakdown (matches wahi 3-part DeepEval pattern jo humne discuss kiya):

1. **Golden set load karo** — `retriever_goldens.json` se questions + ideal answers.
2. **LLMTestCase banao** — har golden ke liye retriever ko live call karo, `retrieval_context` fill karo. `actual_output`

placeholder hai kyunki generator abhi evaluate nahi ho raha.

3. **2 metrics** — `ContextualRecallMetric` aur `ContextualPrecisionMetric`, dono `threshold=0.7`, judge model = `gpt-4.1-mini`.
4. `evaluate()` — test cases + metrics pass kiye, saath me `hyperparameters` dict bhi diya — isse DeepEval ke report/dashboard me current run ki config (embedding model, chunk size/overlap, top_k, judge model) automatically log ho jaati hai — pehle jo humne manual `print()` socha tha, uski jagah ye zyada clean, built-in tareeka hai.

Note: Is file me `chunk_size=1000, chunk_overlap=150` already set hai — matlab ye repo ka snapshot **already-tuned baseline** (improvement Round 1 ke baad wali config) reflect karta hai, na ki bilkul shuru wali 750/100 wali baseline. Judge model bhi `gpt-4.1-mini` hai (`gpt-4.1` nahi).

Run karne ka command (module import errors avoid karne ke liye, `src / evals` folders me `__init__.py` files add karke):

```
python3 -m evals.eval_retriever
```

Reranker Version — `evals/eval_retriever_with_reranker.py`

Improvement Round 2 (reranker add karna) ke liye ye separate file hai — sirf **ek import aur ek line** ka fark hai baaki sab same:

```

import json

from dotenv import load_dotenv

from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import ContextualRecallMetric, ContextualPrecisionMetric

from src.reranker import RerankingRetriever

load_dotenv()

GOLDEN_PATH = "goldens/retriever_goldens.json"
JUDGE_MODEL = "gpt-4.1-mini"
THRESHOLD = 0.7

# 1. LOAD the golden set --- the fixed, human-authored truth
with open(GOLDEN_PATH) as f:
    goldens = json.load(f)

# 2. RUN THE RETRIEVER on each question to fill retrieval_context,
#     then build one test case per golden.
retriever = RerankingRetriever()

test_cases = []

for g in goldens:
    retrieved = retriever.invoke(g["query"])
    retrieval_context = [doc.page_content for doc in retrieved]

    test_cases.append(
        LLMTestCase(
            input=g["query"],
            expected_output=g["ideal_answer"],
            retrieval_context=retrieval_context,
            actual_output="(generator not evaluated in this run)",
        )
    )

# 3. THE METRICS --- recall (did we miss?) and precision (ranked well?)
metrics = [
    ContextualRecallMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
    ContextualPrecisionMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
]

# 4. EVALUATE --- every metric on every case, batched + parallel, with a printed report
evaluate(
    test_cases=test_cases,
    metrics=metrics,
    hyperparameters={
        "retriever": "base_k5",          # vs "reranked" when you swap it in
        "embedding_model": "text-embedding-3-small",
        "chunk_size": 1000,
        "chunk_overlap": 150,
        "top_k": 5,
        "judge_model": JUDGE_MODEL,
        "golden_set": GOLDEN_PATH,
    },
)

```

Fark sirf itna hai:

- Import: `from src.retriever import build_retriever` → `from src.reranker import RerankingRetriever`
- Retriever object: `build_retriever()` → `RerankingRetriever()`

- Baaki poora structure (test case building, metrics, evaluate call) **hooba-hoo same hai**.

Design insight: Ye dikhata hai ki DeepEval ka structure kitna **modular/swappable** hai — sirf retriever-object ka source change karke, poora eval pipeline dobara likhne ki zaroorat nahi padi. Yehi approach embedding model badalne (small → large) ya K badalne ke liye bhi kaam karta — sirf `src/retriever.py` ke andar ka config change hota, eval file same rehti.

13. Live Results — Baseline Aur Iterative Improvements

Baseline Run

Settings: `chunk_size=750`, `chunk_overlap=100`, `k=5`, embedding model = `text-embedding-3-small`.

Result: Recall = 80%, Precision = 80% — 15 me se 10 test cases pass, 5 fail (threshold 0.7 ke against).

Improvement Round 1 — Chunk Size Badhaya

Change: `chunk_size` 750→1000, `chunk_overlap` 100→150.

- **Reasoning:** Bade chunks me zyada context/information hoti hai, isliye claims milne ke chances badh jaate hain.
- **Zaroori step:** Chunking parameters badalne ka matlab hai chunks khud change ho jaayenge — isliye **existing** `chroma_store` **delete karke dobara embed karna padta hai**.

Result: Recall = 97%, Precision = 83% — dono improve hue, ab sirf 3 fail ho rahe (pehle 5 the).

Improvement Round 2 — Reranker Add Kiya

Reranker kya karta hai: Retriever ke fetched chunks ko leta hai aur unhe **re-rank** karta hai — sabse **meaningful** chunks ko upar le aata hai, non-meaningful (noise) ko neeche.

Kyun help karta hai: Jaisa humne discuss kiya, Contextual Precision **rank-aware** hai — sahi chunks upar honge to precision automatically badhega.

Code change minimal tha — `build_retriever()` ki jagah `build_reranking_retriever()` use kiya gaya (implementation same repo me `reranker.py` ke through available hai, HuggingFace ka ek sentence-transformer model power karta hai).

Result: Recall = 92%, Precision = 85% — precision thoda aur badha (83→85), sirf 2 fail ho rahe (pehle 3 the). Recall thoda gira (97→92) — trade-off expected tha.

Improvement Round 3 — Bada Embedding Model

Change: `text-embedding-3-small` → `text-embedding-3-large`.

- **Trade-off:** Cost badhega, lekin quality potentially better.

Result: Recall = 99%, Precision = 85% — recall aur zyada badh gaya, precision same reh gaya. 3 fail ho rahe (thoda variance — chhote dataset (sirf 15 rows) ki wajah se run-to-run thoda fluctuation aata hai, jo normal hai).

Improvement Round 4 — K Ghataya (Try Kiya)

Change: `k=5` → `k=3`.

- **Reasoning:** Kam chunks fetch karne se noise kam aayega, precision improve ho sakta hai (recall thoda hit lega).

Result: Precision **84%** (koi significant improvement nahi) — is particular setup me fayda nahi hua.

Final Summary Table

Iteration	Chunk Size/Overlap	Reranker	Embedding Model	K	Recall	Precision	Fails (of 15)
Baseline	750/100	No	text-embedding-3-small	5	80%	80%	5
Round 1	1000/150	No	text-embedding-3-small	5	97%	83%	3
Round 2	1000/150	Yes	text-embedding-3-small	5	92%	85%	2
Round 3	1000/150	Yes	text-embedding-3-large	5	99%	85%	3
Round 4	1000/150	Yes	text-embedding-3-large	3	~lower	84%	—

Final achieved: ~95%+ Recall aur **~85% Precision** — dono “kaafi achhe” mane gaye.

Aage aur improve karne ke potential raaste: Behtar quality ka reranking model use karna, chunking parameters (jaise 1500/200) ke saath aur experiment karna.

14. Generator — Kya Hai Aur Kaise Banta Hai

Recap — Aaj Ka Plan

Pichhle session me **Retriever** complete ho gaya tha (Recall + Precision, dono achhe scores pe). **Aaj** ka plan hai: **Generator** ko build + evaluate karna, aur phir **RAG Pipeline** (Retriever + Generator connected) ko bhi build + evaluate karna — dono ek hi session me.

Same principle continue: pehle Generator banao, phir isolation me evaluate karo — poora application ek saath nahi banate.

Generator Kya Hai

Ek RAG pipeline me Retriever ka kaam hai query ko embed karke vector database se top-K relevant docs fetch karna. Uske baad **Generator** ka kaam shuru hota hai: usi query aur retrieved docs ko lekar ek answer generate karna.

Simple definition: Generator ek function hai jisko input me 2 cheezein milti hain — **question** aur **relevant documents (context)** — aur output me 1 cheez milti hai — **answer**. Andar bas ek LLM call hai.

Generator Code (`src/generator.py`)

Model: **GPT-4o mini** (sasta), **temperature = 0** (evaluation ke waqt hamesha 0 rakha jaata hai, deterministic output ke liye).

System prompt (starting/basic version): > *“You are a helpful teaching assistant for a course on LLM evaluations. Answer the student’s question ONLY from the context provided below. Use only information present in the context — do not add outside knowledge. If the context does not contain enough information to answer, say ‘I don’t have enough information in the course material to answer that.’ Keep the answer clear and concise.”*

Key design choices explicitly baked into the prompt:

- Sirf context se answer do, apni training-knowledge se nahi.
- Agar context me answer nahi hai, to **explicitly bol do** “nahi pata” — **hallucinate mat karo**, chahe kuch bhi ho jaaye.
- Answer clear aur concise rakho.

Code structure: prompt template + LLM + LangChain chain (`prompt | llm | StrOutputParser()`), aur ek simple `generate(query, context)` function jo dono inputs leta hai aur answer string return karta hai.

Sanity test: Ek dummy context aur question (“*What is online eval?*”) manually bana ke `generate()` ko call kiya — answer aaya (“*Online eval means evaluating your system on live production traffic after deployment. It works without an answer key, unlike offline eval...*”). Isse pata chala ki setup kaam kar raha hai — lekin ye **koi proof nahi hai ki generator achha kaam kar raha hai**, sirf ye pata chala ki wo *chal* raha hai. Achha/bura kaam kar raha hai, ye

evaluate karne se pata chalega.

15. Generator Ke 2 Failure Modes → Faithfulness & Answer Relevancy

Kisi bhi system ko evaluate karne ka **first-principles approach**: pehle socho wo system **kaise-kaise fail ho sakta hai** — failure modes dhundo, unse metrics nikalte hain.

Failure Mode 1 — Unfaithful Response (Hallucination)

Generator ko question + context diya jaata hai, use bola jaata hai *“is context ke basis pe answer karo”* — lekin generator context to padhta hai, saath me apni taraf se thodi extra information bhi add kar deta hai. Isse **unfaithful response** bolte hain.

Example (transcript se, live illustration):

Query:

Query

Does the CampusX AI Engineering program include live classes?

Query

Retrieved Context (jo generator ko diya gaya):

Retrieved context

The AI Engineering program includes recorded lessons, coding assignments, projects, and weekly doubt-solving sessions.

Retrieved Context

Context me kahin explicitly nahi likha ki “live classes” hoti hain ya nahi — sirf recorded lessons, coding assignments, projects, aur weekly doubt-solving sessions mention hain.

Generator ka (galat) answer:

Yes. The program includes two live classes every week along with weekly doubt-solving sessions.

Unfaithful Answer

Ye answer **context me kahin nahi likha tha** — generator ne khud se bana diya (“hallucinate” kar diya) jab usse context me seedha jawab nahi mila.

Danger: Ye bahut hi dangerous scenario hai. Isi tarah ka real-world example: **Air Canada case study** (jahan ek airline chatbot ne khud se bol diya “ticket book kar lo, refund baad me kar denge” — jo policy me nahi tha) — wahan bhi ye **generator ki galti** thi, retriever ki nahi. Context ko ignore karke khud se information create karke user ko de diya.

Metric jo isse test karti hai: Faithfulness — generated answer apne context ke against kitna faithful hai.

Important nuance: Faithful hone ka matlab ye NAHI hai ki answer **correct** bhi ho. Agar retriever galat context laaya, aur generator usi galat context se answer bana de (chahe wo answer completely galat ho), to bhi wo **faithful** maana jaayega — kyunki usne apni taraf se koi extra info add nahi ki, jo context mila usi se poora answer banaya.

Faithfulness = context ke against loyalty, correctness nahi.

Failure Mode 2 — Irrelevant Response

Doosra failure mode: generator context se hi answer banata hai (faithful hai), lekin wo answer khud **question ko relevantly address hi nahi karta**.

Example: Same query, aur is baar generator ka answer:

The program includes coding assignments, projects, recorded lessons, and weekly doubt-solving sessions.

Faithful Answer

Ye answer poori context se hi bana hai (faithful hai) — lekin **kya ye question (“kya live classes hain?”) ka jawab de raha hai?** Nahi — ye sirf context repeat kar raha hai, seedha jawab nahi de raha.

Ideal, sahi tarah relevant answer aisa hota:

The provided context does not confirm that the program includes live classes. It only mentions recorded lessons, coding assignments, projects, and weekly doubt-solving sessions.

Ideal Relevant Answer

Ye answer **relevant bhi hai** (question ko directly address kar raha hai) **aur faithful bhi hai** (koi extra info nahi add ki) — dono properties ek saath.

Metric jo isse test karti hai: Answer Relevancy — generated answer, poochhe gaye question ke liye kitna relevant hai.

Note: Aur bhi metrics hote hain jo generator pe test kiye ja sakte hain (Citation Accuracy, Completeness, Correctness) — lekin wo **application level** pe test honge, is session me nahi.

Aaj ka goal: Generator ko 2 cheezon pe evaluate karna — **Faithfulness** aur **Answer Relevancy**.

16. Faithfulness — Kaise Calculate Hota Hai

Golden Dataset (Faithfulness Ke Liye)

2 columns: **Question** aur **Golden Context** (Retriever session wale “Ideal Answer” column se **alag** hai — ab dobara **context** chahiye, answer nahi).

Kisi insaan ne (ya sahayak process se) vector database ke saare chunks chhaan ke, kisi particular question ke relevant chunks dhoondh ke unhe golden context ke roop me daal diya. 10-15-50 questions ke liye ye repeat kiya jaata hai.

Calculation Process (Step-by-Step)

1. Question uthao, **generator ko bhejo saath me golden context ke** (isolation testing — retriever se context nahi aa raha, direct golden dataset se aa raha hai).
2. Generator ek answer generate karta hai.
3. Is generated answer ko ek **LLM-as-a-judge** ke paas bhejo. Judge answer ko **atomic claims** me todta hai (jaise: Claim 1, Claim 2, Claim 3...).
4. Har claim ko ek-ek karke check karo: **kya ye claim golden context me exist karta hai?**
5. Jitne claims golden context me mile, unko count karo → **Faithfulness Score = (claims context me mile) / (total claims)**.
6. Har question ke liye ye repeat karo, phir average nikalo → final Faithfulness score.

Worked example (transcript ke graphic se):

Query: what does it mean for a benchmark to get saturated?

Context (what the generator was given)

Saturation = scores cluster high & close together (95%, 94%, 92%)
→ can't tell models apart → benchmark no longer useful

Generated answer (this is what we score)

"A benchmark gets saturated when it can no longer distinguish between models. This happens once models all score very high on it, and the models show a wide range of scores across the benchmark."

↓ decompose into claims, check each against context

Claim 1 — a saturated benchmark no longer distinguishes models

context says exactly this ("can't differentiate the models")

✓

Claim 2 — it happens once models all score very high

context supports this (95% / 94% / 92% clustering)

✓

Claim 3 — models show a wide range of scores

CONTRADICTS context — saturation is clustering, not spreading

✗

supported ÷ total = 2 ÷ 3

Faithfulness = 0.67

Faithfulness Calculation

- **Query:** "What does it mean for a benchmark to get saturated?"
- **Context jo generator ko diya gaya:** Saturation = scores cluster high & close together → can't tell models apart → benchmark no longer useful.
- **Generated answer:** 2 claims context se match karte hain (saturation = models ko differentiate na kar paana; ye tab hota hai jab sab models high score karte hain), lekin 3rd claim ("models show a **wide range** of scores") **context se CONTRADICT** karta hai — context to bolta hai scores **close/cluster** karte hain, wide-range nahi.
- **Result:** 2 supported / 3 total claims → **Faithfulness = 0.67**

17. Answer Relevancy — Kaise Calculate Hota Hai

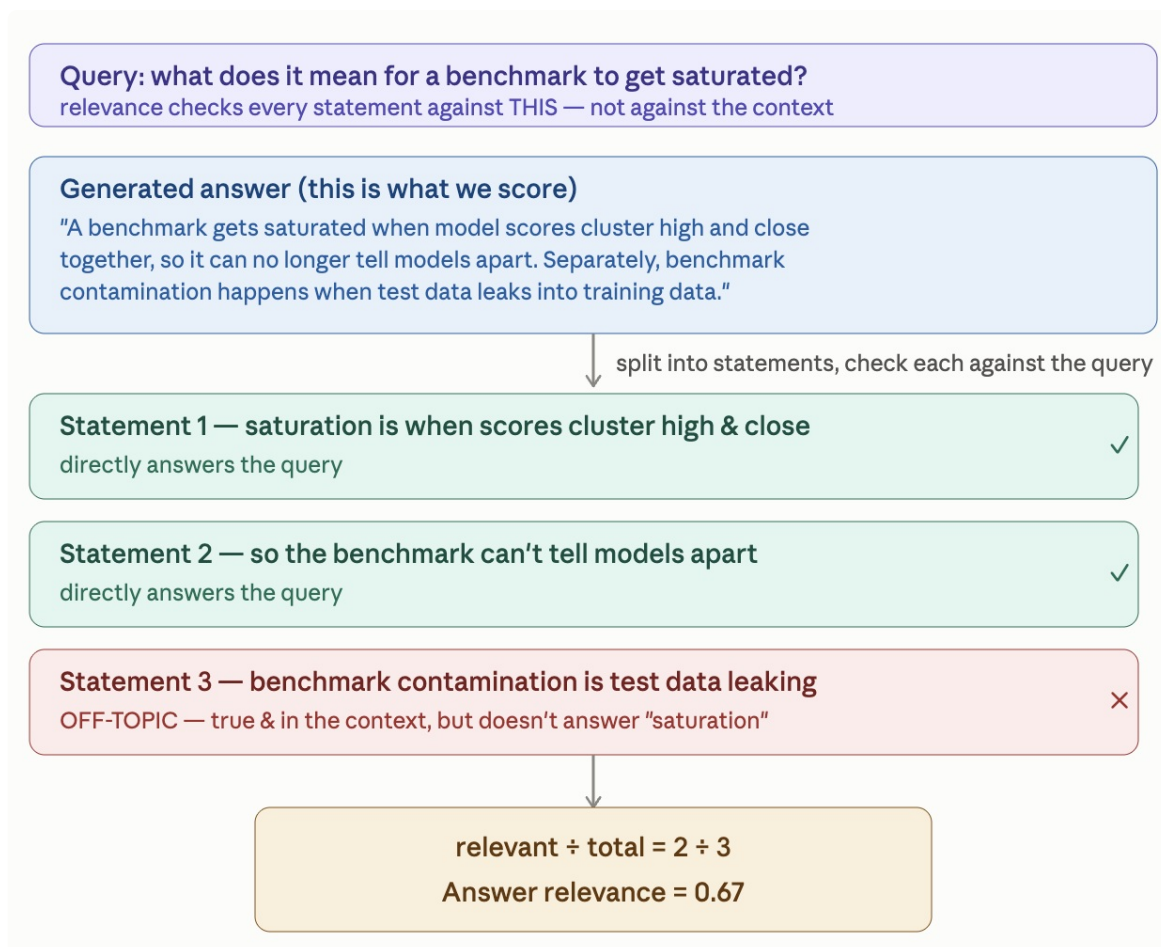
Faithfulness se thoda aasan hai — **isko golden dataset ki zaroorat bhi nahi hoti**. Isliye Answer Relevancy ek **reference-free evaluation** hai (koi golden/ideal reference ki zaroorat nahi).

Calculation Process (Step-by-Step)

1. Question + context generator ko bhejo, ek answer generate hota hai.
2. Is answer ko LLM-judge se **claims me todo**.
3. Har claim ko **question ke against** check karo: *"kya ye claim is question ko answer karne me help kar raha hai, ya irrelevant hai?"*
4. Jitne claims relevant nikle, unhe count karo → **Answer Relevancy Score = (relevant claims) / (total claims)**.
5. Multiple questions pe repeat karo, average nikalo.

Important — Faithfulness ka bilkul opposite comparison point hai ye: Faithfulness me answer ko **context** ke against check karte hain. Answer Relevancy me answer ko **question** ke against check karte hain. Do bilkul alag comparisons hain, dono claims-based approach use karte hain lekin different anchor ke against.

Worked example (transcript ke graphic se):



Answer Relevancy Calculation

- **Query:** "What does it mean for a benchmark to get saturated?"
- **Generated answer:** 3 statements — (1) saturation = scores cluster high & close, (2) isliye benchmark models ko differentiate nahi kar paata, (3) benchmark contamination = test data training data me leak hona.
- Statement 1 aur 2 seedha question ko answer karte hain (relevant). Statement 3 **context me sahi hai** (true fact hai), lekin **question ko answer nahi kar raha** — ye off-topic hai ("saturation" ke baare me poocha tha, "contamination" ke baare me nahi).
- **Result:** 2 relevant / 3 total → **Answer Relevancy = 0.67**

Key takeaway: Answer Relevancy **factually correct hone se concern nahi hai** — sirf ye check karta hai ki statement **question ke liye topically relevant** hai ya nahi. Factually correct lekin off-topic statement bhi "irrelevant" maana jaata hai.

18. Generator Eval Code — Real Repo Code ([evals/eval_generator.py](#))

Ye **exact code hai jo repo me hai**:

```

"""
evals/eval_generator.py
=====
Component-level evaluation of the GENERATOR, in isolation.

Faithfulness: of the claims in the generated answer, how many are supported
by the context it was given? (Did the generator make things up?)

ISOLATION: we feed the generator the GOLDEN context (the known-good chunks
from the faithfulness dataset), NOT the retriever's output. So a low score
is purely the generator's fault — the context was already correct.

    python -m evals.eval_generator
"""

import json
from dotenv import load_dotenv

from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import FaithfulnessMetric, AnswerRelevancyMetric

from src.generator import generate # your generator: generate(query, context) -> answer

load_dotenv()

GOLDEN_PATH = "goldens/faithfulness_dataset.json"
JUDGE_MODEL = "gpt-4o-mini"
THRESHOLD = 0.7

# 1. LOAD the faithfulness golden set (query + ideal_context)
with open(GOLDEN_PATH) as f:
    goldens = json.load(f)

# 2. RUN THE GENERATOR on the GOLDEN context (isolation), build one test case each
test_cases = []
for g in goldens:
    context = g["ideal_context"] # known-good context (list of chunk strings)
    answer = generate(g["query"], context) # RUN the generator -> actual_output

    test_cases.append(
        LLMTestCase(
            input=g["query"],
            actual_output=answer, # the generated answer we're judging
            retrieval_context=context, # faithfulness checks the answer against THIS
            # no expected_output — faithfulness never reads it
        )
    )

# 3. THE METRIC — decomposes actual_output into claims, attributes each to context
metrics = [FaithfulnessMetric(
    threshold=THRESHOLD,
    model=JUDGE_MODEL,
    include_reason=True, # prints WHY each score — shows which claims were unsupported
),
AnswerRelevancyMetric(
    threshold=THRESHOLD,
    model=JUDGE_MODEL,
    include_reason=True
)]

# 4. EVALUATE — runs the metric on every case, prints a report
evaluate(test_cases=test_cases, metrics=metrics)

```

Structure bilkul retriever wale eval jaisa hi hai (same 3-part DeepEval pattern):

- `LLMTestCase` : `input` = golden dataset se question, `actual_output` = generator ka answer, `retrieval_context` = golden dataset ka context (`ideal_context`) — `expected_output` **nahi diya**, kyunki Faithfulness aur Answer Relevancy dono ismko read hi nahi karte.
- **2 Metrics**: `FaithfulnessMetric` aur `AnswerRelevancyMetric` — dono DeepEval ke built-in metrics.
- `evaluate()` : sab test cases + metrics ko run karta hai.

Run command: `python -m evals.eval_generator`

Golden dataset kaise bana: Poora vector database ke saare chunks (862 chunks) ek JSON file me export kiye gaye, phir puri file Claude ko di gayi is instruction ke saath — **“ek baar me sirf ek question+context pair banao”** (step-by-step, sab ek saath nahi). Har generated question ka context manually review kiya gaya (instructor ko khud pata tha kis session me kya padhaya) — is tarah 15 questions ka `goldens/faithfulness_dataset.json` bana.

19. Generator Results — Baseline Aur Improvement

Baseline Run

Basic system prompt ke saath:

Metric	Score
Faithfulness	91%
Answer Relevancy	73%

Interesting question discuss hui class me: Faithfulness itna high (91%) pehli hi try me kyun aa gaya, bina kisi tweaking ke?

Answer: Faithfulness me actually zyada “mehnat” LLM ko karni hi nahi padti — hum generator ko explicitly bol hi rahe hain *“isi context se answer karo”*, aur aaj ke LLMs ka **instruction-following capability** kaafi high hai. Isliye is baat ki high probability hai ki jo bhi answer generate hoga wo usi context se bana hoga — Faithfulness naturally high aane ki tendency hoti hai. **Answer Relevancy is comparatively “harder”** kyunki usme LLM ko genuinely samajhna padta hai ki question kya poocha gaya hai aur uska seedha jawab dena padta hai (sirf context repeat nahi karna).

Generator Ko Improve Kaise Karein?

Retriever ke paas improve karne ke kai levers the (chunk size, embedding model, reranker, K). **Generator ke paas comparatively kam options hain**, mainly 2:

1. **Better model pe switch karo** — jo behtar instruction-following de.
2. **System prompt ko tweak karo** — ye **bahut zyada matter karta hai**.

Approach jo use kiya gaya: Multiple runs liye, har run me jo test cases fail ho rahe the unke reasons dekhe (DeepEval `include_reason=True` se milte hain), un failure-reasons ko ek LLM (Claude) ko diya aur system prompt ko **3-4 iterations** me refine kiya.

Final improved prompt me naye rules add hue (failures dekh-dekh ke):

- *“Do not strengthen or overstate claims”* — agar context 2 alag cheezein bolta hai, unhe upgrade karke ek combined/stronger method mat bana do.
- *“The context is an informal lecture transcript — synthesize and rephrase what is there, do not require the question’s exact wording to appear.”*
- (Pehle wale rules bhi retained: sirf context se answer, hallucinate mat karo, clear/concise raho.)

Improved Results

Metric	Baseline	After Prompt Tuning
Faithfulness	91%	96%
Answer Relevancy	73%	92%

Overfitting ka risk: Agar system prompt ko bahut zyada apne **golden dataset ke hisaab se** tweak kar doge, to ek tarah ki **overfitting** ho sakti hai — test data pe achha result, lekin naya real-world data (jaise retriever se aane wala context) aane pe performance gir sakta hai. Isko validate karne ka tareeka: **pipeline level pe dobara test karna** (agla section) — jahan context ab golden dataset se nahi, real retriever se aayega.

DeepEval parallel execution note: Jab `evaluate()` chalta hai, saare test cases **parallel** me run hote hain (sequential nahi) — isliye evaluation kaafi fast hota hai.

Bonus — Confident AI Dashboard (Experiment Tracking)

DeepEval ki parent company **Confident AI** pe bhi run results save kiye ja sakte hain (`deepeval view` command se) — isse ek dashboard milta hai jahan pass/fail count, config (system prompt, chunk size, etc.) sab log ho jaata hai, jaisa MLflow karta hai. Ye ek **LLMOps/experiment-tracking** feature hai, core evaluation ka part nahi, lekin useful hai.

20. Component Level — Complete! (Checkpoint)

Is point pe **Component Level Eval Suite complete** ho gaya:

- **Retriever:** Recall, Precision
- **Generator:** Faithfulness, Answer Relevancy

4 naye metrics seekh liye, aur DeepEval me implement bhi kar liye. Ab agla level: **Pipeline Level**.

21. RAG Pipeline — Retriever + Generator Ko Connect Karna

Ab jab dono components (Retriever, Generator) independently achhe se kaam kar rahe hain, unhe **connect** karte hain — yehi RAG Pipeline banata hai.

Code (`src/rag_pipeline.py`)

Bahut simple “glue code” — ek `RagPipeline` class:

1. `RerankingRetriever` (reranker wala retriever, pichhle session ka improved version) se context fetch karo.
2. Context ko ek string me convert karo.
3. Context + question ko **generator** ko bhejo.
4. Generator ka answer return karo.

```
# src/rag_pipeline.py (structure)
class RagPipeline:
    def __init__(self):
        self.retriever = RerankingRetriever()

    def invoke(self, query):
        docs = self.retriever.invoke(query)
        context = [doc.page_content for doc in docs]
        answer = generate(query, context)
        return {"answer": answer, "context": context}
```

Test run: Query — *“What is drift and why does it matter after deployment?”* — is baar query **real retriever** ke paas gayi, real context fetch hua, phir wo context + query **real generator** ke paas gaye, aur answer aaya. Pipeline error-free chal raha — matlab pipeline working hai.

22. Pipeline-Level Evaluation — RAG Triad

RAG Triad hi standard tareeka hai RAG pipeline evaluate karne ka — agar aap internet pe search karo “how do you evaluate a RAG pipeline”, yehi keyword milega.

3 entities hote hain: **Question**, **Context** (retriever se), **Answer** (generator se). In teeno ke beech 3 pairs, 3 metrics:

Pair	Metric
Answer ↔ Context	Faithfulness
Question ↔ Answer	Answer Relevancy
Question ↔ Context	Contextual Relevancy (<i>naya</i>)

Bahut important distinction — Component Level vs Pipeline Level:

Component Level pe humne bhi Faithfulness aur Answer Relevancy calculate kiya tha jab Generator isolation me test kar rahe the. Ab Pipeline Level pe **wahi 2 metrics phir se calculate ho rahe hain** — to fark kya hai?

Fark sirf itna hai: context kahan se aa raha hai. Component level pe context **golden dataset** se aa raha tha (hand-picked, guaranteed-correct). Pipeline level pe ab context **real retriever** se aa raha hai — jo perfect nahi hai. Isliye same calculation method hone ke bawajood, **scores different aayenge** — ye zyada realistic, end-to-end test hai.

23. Contextual Relevancy — Kaise Calculate Hota Hai

Ye teesra, naya metric hai jo abhi tak discuss nahi hua tha.

Simple definition: Retriever ne jo context pull karke diya hai, wo **question ko answer karne ke liye kitna relevant hai** — ye check karta hai.

Calculation Process (Step-by-Step)

1. Retriever ko sirf **question** do (bas itna hi chahiye retriever ko).
2. Retriever K contexts return karta hai (jaise $K=5 \rightarrow 5$ chunks).
3. Ek LLM-judge lo. **Har chunk ko individually claims me break karo** — Chunk 1 se 3 claims, Chunk 2 se 2 claims, waghera. Maan lo total 15 claims mile 5 chunks se.
4. Ab har ek claim ko **question ke against** check karo: “*kya ye claim is question se related/relevant hai?*” — yes/no.
5. Jitne claims relevant nikle (jaise 15 me se 10) → **Contextual Relevancy = 10/15**.
6. Multiple questions pe repeat + average karo.

Reference-free evaluation: Contextual Relevancy ke liye bhi golden dataset sirf **questions provide karne ke liye** chahiye — koi reference/answer-key ki zaroorat nahi. Isliye ye bhi **reference-free** hai.

24. RAG Pipeline Eval Code — Real Repo Code (`evals/eval_rag_pipeline.py`)

```

import json
from dotenv import load_dotenv

from deepeval import evaluate
from deepeval.test_case import LLMTestCase
from deepeval.metrics import (
    FaithfulnessMetric,
    AnswerRelevancyMetric,
    ContextualRelevancyMetric,
)

from src.rag_pipeline import RagPipeline

load_dotenv()

GOLDEN_PATH = "goldens/faithfulness_dataset.json" # reuse the queries
JUDGE_MODEL = "gpt-4o-mini"
THRESHOLD = 0.7

# 1. LOAD queries (we only need the queries — context comes from the pipeline now)
with open(GOLDEN_PATH) as f:
    goldens = json.load(f)

# 2. RUN THE FULL PIPELINE per query, build a test case from LIVE output
rag = RagPipeline()
test_cases = []
for g in goldens:
    result = rag.invoke(g["query"]) # retrieve → rerank → generate

    test_cases.append(
        LLMTestCase(
            input=g["query"],
            actual_output=result["answer"], # what the generator produced
            retrieval_context=result["context"], # what the RETRIEVER returned
        )
    )

# 3. THE THREE TRIAD METRICS
metrics = [
    ContextualRelevancyMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
    FaithfulnessMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
    AnswerRelevancyMetric(threshold=THRESHOLD, model=JUDGE_MODEL, include_reason=True),
]

# 4. EVALUATE
evaluate(test_cases=test_cases, metrics=metrics)

```

Key difference vs Generator eval code: `GOLDEN_PATH` wahi hai (`faithfulness_dataset.json`) — lekin ab **sirf** `query` field use ho rahi hai, `ideal_context` nahi (comment: *“we only need the queries — context comes from the pipeline now”*). `retrieval_context` ab `result["context"]` se aa raha hai — matlab real retriever ka output, golden context nahi. `expected_output` **yahan bhi nahi diya** — RAG Triad ke teeno metrics reference-free hain.

Run command: `python -m evals.eval_rag_pipeline`

25. Pipeline Results — Ek “Curious Case”

Results

Metric	Score
Faithfulness	~93%
Answer Relevancy	~86%
Contextual Relevancy	~42% 

Faithfulness aur Answer Relevancy dono achhe reh gaye (matlab jo system prompt tune kiya tha wo genuinely robust tha — sirf golden-context pe overfit nahi tha, real retriever-context pe bhi kaam kar raha hai). Lekin **Contextual Relevancy sirf 42%** — ye kaafi kam hai.

Curious Case: Retriever Ka Recall/Precision Achha, Phir Contextual Relevancy Itna Kam Kyun?

Retriever ke apne independent metrics (pichhle session se): **Recall ~99%, Precision ~89%** — dono bahut achhe. Lekin isi retriever ka context jab pipeline me Contextual Relevancy se test kiya, to sirf **42%** aaya.

Question: Ek retriever “achha” hone ke saath-saath “bura” kaise ho sakta hai? Ye teeno metrics (Recall, Precision, Contextual Relevancy) alag kya measure kar rahe hain?

Resolution (transcript se, key insight):

- **Recall** batata hai: jitne total sahi chunks the, unme se kitne retriever la paaya (99% — matlab bahut kam miss ho raha hai).
- **Precision** batata hai: jitne chunks laaya, unme se kitne overall **useful** the (89% — matlab zyadatar laaye gaye chunks kaam ke hain).
- **Contextual Relevancy** batata hai: retrieved content **kitna relevant hai** question ko answer karne ke liye — lekin ye check **chunk-level pe nahi, claim-level pe** hota hai.

Asli reason: Har chunk ke andar bhi noise ho sakta hai. Precision sirf ye dekhta hai “is chunk me se kam-se-kam kuch useful hai ya nahi” — agar chunk ka ek chhota sa hissa bhi relevant hai, to poora chunk “useful” gin liya jaata hai. Lekin agar ek chunk ki **5 lines** hain aur usme se sirf **1-2 lines relevant** hain, baaki **3 lines off-topic/noise** hain — to jab is chunk ko claims me todoge, to 5 claims banenge jisme se sirf 1-2 relevant honge. Yehi **per-chunk internal noise** hai jo Contextual Relevancy pakadta hai, lekin Precision nahi pakadta (kyunki Precision sirf chunk ko boolean useful/not-useful maanta hai, chunk ke andar ke content ki granularity nahi dekhta).

One-liner: “Precision checks whether each retrieved chunk is useful overall. Contextual Relevancy checks how much of each chunk’s actual content is useful — it catches noise WITHIN a chunk that Precision cannot see.”

Fix karne ka lever: Chunk size ghatao — chhote chunks me proportionally kam noise hone ka chance hota hai (ek chota chunk single-topic hone ki zyada sambhavna rakhta hai).

Practical judgment call: Agar Faithfulness, Answer Relevancy, Recall, Precision — sab achhe hain, aur sirf Contextual Relevancy thoda kam hai, to **end-user ke liye answers phir bhi achhe aa rahe hain** — is case me thoda kam Contextual Relevancy **acceptable** bhi ho sakta hai (business judgment call), ya aap further tune kar sakte ho (chunk size kam karke), jisme trade-off ka risk hai — baaki metrics thode neeche bhi aa sakte hain.

26. Session Ka Outcome — Component + Pipeline Level Complete

Is session me humne:

1. **Generator** build kiya (query + context → answer, simple LLM call + prompt).
2. Generator ke **2 failure modes** samjhe — unfaithful response (hallucination) aur irrelevant response — jinse **Faithfulness** aur **Answer Relevancy** metrics nikalte hain.
3. Dono metrics ka **claim-based calculation** seekha (LLM-as-judge se claims me todna, phir context/question ke against verify karna).
4. Golden dataset (question + golden context) banaya, DeepEval code likha, aur **system prompt tuning** se scores improve kiye (91%→96% Faithfulness, 73%→92% Answer Relevancy).
5. **RAG Pipeline** banaya — Retriever + Generator connect kiya.
6. **RAG Triad** seekha — Faithfulness, Answer Relevancy (dobara, ab real retrieved context ke against), aur **naya**

metric: Contextual Relevancy.

7. Ek **curious case** solve kiya — achha Recall/Precision hone ke bawajood Contextual Relevancy kam kyun aa sakta hai (per-chunk internal noise).

Poora Component + Pipeline Level Eval Suite ab complete hai — 5 core RAG metrics cover ho chuke: Recall, Precision (Retriever) + Faithfulness, Answer Relevancy (Generator) + Contextual Relevancy (Pipeline/Triad). *(Ye wahi 5 metrics hain jo DeepEval documentation me “RAG” category ke andar official listed hain.)*

Baaki bacha hai: Application Level evaluation (Correctness, Completeness, Style — custom **GEval**-based metrics), phir **Safety evals**, **Ops evals**, aur finally **Regression Testing + Online Evaluation** — ye sab agli classes me.

Confidence statement: Ab humare paas ek RAG pipeline hai jiska Faithfulness, Answer Relevancy, Recall, Precision sab achhe hain — Contextual Relevancy thoda weak hai lekin acceptable range me. Is confidence ke saath hum aage **Application Level** evaluation (agla session) ki taraf badhenge.

27. Code Reference Appendix — Application-Level Eval (Session Not Yet Covered)

Important: Is file ka **class transcript abhi tak cover nahi hua hai** — isliye neeche sirf **real code + code ke apne comments ke basis pe** short explanation di gayi hai (koi instructor-commentary fabricate nahi ki gayi). Jab is session ka detailed transcript milega, to poore conceptual notes (jaisa Retriever/Generator/Pipeline ke liye upar diye gaye hain) alag se add kiye jaayenge.

Kya karta hai: Poore pipeline ke against 3 **application-level quality metrics** — Correctness, Completeness, Style — sab **GEval** (custom rubric-based LLM judge) use karke.

```
import json
from dotenv import load_dotenv

from deepeval import evaluate
from deepeval.test_case import LLMTestCase, LLMTestCaseParams
from deepeval.metrics import GEval
from deepeval.metrics.g_eval import Rubric

from src.rag_pipeline import RagPipeline

load_dotenv()

GOLDEN_PATH = "goldens/correctness_goldens.json"
JUDGE_MODEL = "gpt-4o-mini"
THRESHOLD = 0.7

# 1. LOAD queries + ideal answers
with open(GOLDEN_PATH) as f:
    goldens = json.load(f)

# 2. RUN THE FULL PIPELINE per query, build a test case from LIVE output
rag = RagPipeline()
test_cases = []
for g in goldens:
    result = rag.invoke(g["question"]) # retrieve → rerank → generate

    test_cases.append(
        LLMTestCase(
            input=g["question"],
            actual_output=result["answer"],
            expected_output=g["ideal_answer"],
        )
    )
```

```

    )

# 3. THREE APPLICATION-LEVEL QUALITY METRICS

# 3a. CORRECTNESS — reference-based, judges TRUTH (not coverage or length)
correctness = GEval(
    name="Correctness",
    evaluation_steps=[
        "Compare only the factual claims in the actual output against the expected output.",
        "A claim is wrong only if it CONTRADICTS the expected output or is factually false. Judge truth, not completeness.",
        "A factually accurate answer must score at least 0.9 even if it is shorter or covers fewer points than the expected output.",
        "Do NOT deduct for brevity, missing elaboration, or omitted points — omissions are not errors here.",
        "Additional correct information must NEVER lower the score.",
    ],
    rubric=[
        Rubric(score_range=(9, 10), expected_outcome="All stated claims are factually correct and consistent. No contradictions. Brevity is fine."),
        Rubric(score_range=(5, 8), expected_outcome="Mostly correct but one minor inaccuracy."),
        Rubric(score_range=(0, 4), expected_outcome="Contains a clear factual error or a claim that contradicts the expected output."),
    ],
    evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT, LLMTestCaseParams.EXPECTED_OUTPUT],
    threshold=THRESHOLD,
    model=JUDGE_MODEL,
    strict_mode=False,
)

# 3b. COMPLETENESS — reference-based, judges COVERAGE (not correctness)
completeness = GEval(
    name="Completeness",
    evaluation_steps=[
        "Identify the key points contained in the expected output.",
        "Check how many of those key points are addressed in the actual output.",
        "Penalize the actual output for each key point from the expected output that it omits or only partially covers.",
        "Judge coverage only. Do NOT lower the score because a covered point is stated incorrectly — factual correctness is judged separately.",
        "Do NOT penalize the actual output for adding extra information beyond the expected output.",
    ],
    rubric=[
        Rubric(score_range=(9, 10), expected_outcome="Addresses essentially all key points in the expected output."),
        Rubric(score_range=(5, 8), expected_outcome="Covers the main key points but misses one or more."),
        Rubric(score_range=(0, 4), expected_outcome="Misses several key points; only partially covers the expected output."),
    ],
    evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT, LLMTestCaseParams.EXPECTED_OUTPUT],
    threshold=THRESHOLD,
    model=JUDGE_MODEL,
    strict_mode=False,
)

# 3c. STYLE — reference-free, judges TONE only (note: no EXPECTED_OUTPUT)
style = GEval(
    name="Style",
    evaluation_steps=[
        "Judge only the teaching style and tone of the actual output, not whether it is factually correct or complete.",
        "Reward an intuitive, explanatory tone: plain language, the idea explained before any formula or jargon, and technical terms briefly unpacked when used.",
        "Reward a direct, conversational register written in prose, as a CampusX lecture would explain it out loud, rather than a dry, formal, or bullet-list tone.",
        "An analogy or concrete example is a BONUS when the concept is abstract, but a clear, direct, well-explained answer is fully acceptable and must NOT be penalized for not having one.",
    ],

```

```

        "Penalize answers that are stiff, bureaucratic, structured as a bare list with no explanation, or that
        use unexplained jargon.",
        "Do NOT reward or penalize based on correctness, completeness, or length — only on style and tone.",
    ],
    rubric=[
        Rubric(score_range=(9, 10), expected_outcome="Clearly in a CampusX teaching voice: intuitive,
        conversational prose that explains before it formalizes."),
        Rubric(score_range=(7, 8), expected_outcome="Clear, conversational, and well-explained in prose. Fully
        acceptable even without an analogy or example."),
        Rubric(score_range=(4, 6), expected_outcome="Understandable but somewhat flat, formal, or list-heavy in
        places."),
        Rubric(score_range=(0, 3), expected_outcome="Dry, stiff, bare-list, jargon-heavy, or robotic; does not
        read like a teaching explanation."),
    ],
    evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT],
    threshold=THRESHOLD,
    model=JUDGE_MODEL,
    strict_mode=False,
)

# 4. EVALUATE — all three together
evaluate(test_cases=test_cases, metrics=[correctness, completeness, style])

```

Key observations (code se):

- Teeno metrics `GEval` use karte hain — DeepEval ka custom LLM-judge builder jisme aap khud **evaluation_steps** (judge ko diye jaane wale instructions) aur **rubric** (score-range → expected-outcome mapping) define karte ho.
- **Correctness** — sirf **truth** judge karta hai, coverage/length ke liye penalize nahi karta (`"Do NOT deduct for brevity, missing elaboration, or omitted points"`). `expected_output` use hota hai — **reference-based**.
- **Completeness** — sirf **coverage** judge karta hai, correctness ke liye penalize nahi karta (wo Correctness ka kaam hai — dono metrics deliberately ek doosre ke concern se overlap nahi karte). Ye bhi **reference-based**.
- **Style** — `evaluation_params` me `EXPECTED_OUTPUT` **include nahi hai** — matlab ye **reference-free** metric hai, sirf tone/voice judge karta hai (CampusX teaching style ke against), correctness/completeness se bilkul independent.
- Ye teeno metrics exactly wahi hain jo Strategy Overview session me discuss hue the — **Correctness, Completeness, Style** — application-level quality metrics ke roop me.

Quick Recap (Interview Cheat-Sheet)

Concept	One-liner
Retriever	
Retriever failure modes	(1) Relevant context miss karna, (2) Noisy context la lena
Recall	Total correct contexts me se kitne retrieve hue
Precision	Retrieved contexts me se kitne genuinely useful the
Recall vs Precision trade-off	K badhane se Recall ↑ lekin Precision ↓
Golden dataset — galat tareeka	Chunk IDs store karna — chunking params change hote hi void ho jaata hai
Golden dataset — sahi tareeka	Question + Ideal Answer (chunk-independent, reusable)
Contextual Recall	Ideal answer ko claims me todo, dekho retrieved chunks me kitne claims mile
Contextual Precision	Rank-aware — sirf “kitne sahi” nahi, “sahi wale upar hain ya nahi” bhi check karta hai
Generator	
Generator failure modes	(1) Unfaithful/hallucinated response, (2) Irrelevant response
Faithfulness	Answer ke claims context me support hote hain ya nahi (correctness nahi, sirf context-loyalty)
Answer Relevancy	Answer ke claims question ko address karte hain ya nahi (reference-free)
Pipeline (RAG Triad)	
RAG Triad	Faithfulness (Answer↔Context) + Answer Relevancy (Question↔Answer) + Contextual Relevancy (Question↔Context)
Component vs Pipeline level	Same metrics, fark sirf itna — context golden-dataset se aata hai vs real retriever se
Contextual Relevancy	Retrieved chunks ke claims question ke against relevant hain ya nahi — per-chunk internal noise pakadta hai
Curious case	Achha Recall/Precision + kam Contextual Relevancy = chunks ke andar noise (Precision ye nahi pakadta)
General	
Reference-based vs free	Recall/Precision/Faithfulness = reference-based (golden data chahiye); Answer Relevancy/Contextual Relevancy = reference-free
DeepEval core 3 things	<code>LLMTestCase</code> (ek row), Metric (kya measure karna), <code>evaluate()</code> (run karna)
Improvement levers (Retriever)	Chunk size/overlap, Reranker, Embedding model, K value
Improvement levers (Generator)	Better model, System prompt tuning

One-liner for interviews: “A RAG pipeline’s core evaluation rests on 5 metrics — Recall and Precision for the retriever (does it find the right chunks without too much noise), Faithfulness and Answer Relevancy for the generator (does it stick to the context and actually answer the question), and Contextual Relevancy once the two are wired together as the RAG Triad (is the retrieved content itself relevant to the question, down to the claim level). The same Faithfulness and Answer Relevancy metrics get recalculated at the pipeline level — the only difference is the context now comes from the live retriever instead of a hand-picked golden dataset, which is what makes pipeline-level testing a true end-to-end check.”